



Specifica Tecnica

The Bitless (Gruppo 18):

Lorenzo Battistella (2103116), Edoardo De Piccoli (2101055), Davide Facco (2087852),
Anna Marini (2110986), Andrea Menegaldo (2116426), Samuele Vendramin (2111934),
Simone Zecchinato (2113189)

Informazioni documento			
Versione	Data	Stato	Destinatari
0.14.0	2026-08-18	Stesura	The Bitless, prof. T. Vardanega, prof. R. Cardin, Zucchetti

Contatti: thebitless.swe@gmail.com

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.14.0	2026-08-18	A. Marini		-	Unione delle stesure parallele delle sezioni Architettura e Flusso dei dati; aggiornamento dei riferimenti a Norme di Progetto _G v2.0.0, Glossario v2.0.0 e Analisi dei Requisiti _G della baseline _G di Product Baseline _G ; integrazione _G della generazione a partire da un collegamento nello scopo del prodotto
0.13.1	2026-08-16	E. De Piccoli		-	Allineamento dell'Architettura del backend allo spostamento dell'adattatore di trasporto fra gli adattatori primari

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.13.0	2026-08-16	A. Marini		-	Inserimento dei sei diagrammi e delle figure corrispondenti, allineamento dell'enumerazione dei contratti di dipendenza e della definizione di adattatore primario nelle sezioni Tecnologie di test _G e analisi e Panoramica architeturale, e delle annotazioni di struttura della sezione Architettura del backend, a seguito dello spostamento dell'adattatore di trasporto fra gli adattatori primari; riclassificazione della copertura di R-97-F-De nella sezione Persistenza delle note, con la distinzione fra dati posseduti e copertura del requisito e la rimozione dei rimandi al sorgente dal corpo del testo; revisione redazionale di titoli dei paragrafi, grassetti e incisi
0.12.1	2026-08-13	E. De Piccoli		-	Revisione delle sezioni Architettura del backend, Idiomi e convenzioni di codifica e dei relativi flussi: correzioni dei riferimenti ai casi d'uso _G , condensazione del testo e riordino delle figure
0.12.0	2026-08-13	E. De Piccoli		-	Stesura dei flussi di generazione a partire da un collegamento e di propagazione degli errori

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.11.0	2026-08-13	E. De Piccoli		-	Stesura di Idiomi e convenzioni di codifica
0.10.0	2026-08-13	E. De Piccoli		-	Stesura dell'Architettura del backend
0.9.0	2026-08-12	A. Marini		-	Stesura delle sezioni sul flusso di generazione in streaming e sull'annullamento
0.8.0	2026-08-12	A. Marini		-	Stesura della sezione Design pattern adottati
0.7.2	2026-08-11	A. Menegaldo		-	Inserimento dei diagrammi UML _G nella sezione Salvataggio e caricamento di una nota
0.7.1	2026-08-11	A. Menegaldo		-	Inserimento dei diagrammi UML _G nella sezione Architettura del frontend _G
0.7.0	2026-08-11	A. Menegaldo		-	Stesura della sezione Architettura del frontend _G
0.6.0	2026-08-11	A. Marini		-	Stesura della sezione Architettura di deployment
0.5.0	2026-08-11	A. Marini		-	Allineamento della sezione Tecnologie al repository _G di prodotto, stesura della Panoramica architetturale e riorganizzazione dello scheletro delle sezioni di Architettura
0.4.0	2026-07-27	A. Marini	A. Menegaldo	-	Integrazione _G continua e analisi statica del backend, tracciamento dei requisiti _G nella sezione Tecnologie, riorganizzazione della struttura delle sezioni Architettura, Flusso dei dati e Tracciamento

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.3.0	2026-07-27	A. Marini	A. Menegaldo	-	Stesura della sezione Tecnologie
0.2.0	2026-07-27	A. Marini	A. Menegaldo	-	Riorganizzazione della struttura del documento e stesura della sezione Introduzione
0.1.0	2026-07-18	A. Menegaldo	A. Marini	-	Struttura del documento #1

Indice

1	Introduzione	8
1.1	Scopo del documento	8
1.2	Scopo del prodotto	8
1.3	Glossario	9
1.4	Riferimenti	9
1.4.1	Riferimenti normativi	9
1.4.2	Riferimenti informativi	9
2	Tecnologie	12
2.1	Criteri di scelta	12
2.2	Frontend	13
2.3	Backend	16
2.4	Persistenza delle note	19
2.5	Tecnologie di test e analisi	22
2.5.1	Analisi statica	22
2.5.2	Analisi dinamica	24
2.6	Infrastruttura	25
3	Architettura	29
3.1	Panoramica architetturale	29
3.1.1	Visione d'insieme	29
3.1.2	Stile architetturale del servizio applicativo	30
3.1.3	Stile architetturale dell'applicazione web	31
3.1.4	Vocabolario	31
3.2	Architettura di deployment	32
3.2.1	Unità di esecuzione e configurazione	32
3.2.2	Confronto con l'architettura a microservizi	35
3.2.3	Confronto con il monolite a strati	36
3.3	Architettura del frontend	37
3.3.1	Organizzazione dei sorgenti	38
3.3.2	Scomposizione in componenti	39
3.3.3	Gestione dello stato	45
3.3.4	Integrazione dell'editor	48
3.3.5	Accesso al servizio applicativo	51
3.3.6	Persistenza lato client	53
3.4	Architettura del backend	55
3.4.1	Struttura a esagono e contratti di dipendenza	56
3.4.2	Adattatori primari	57
3.4.3	Il nucleo	59
3.4.4	Adattatori secondari	62
3.4.5	Composition root e configurazione	63
3.5	Design pattern adottati	64
3.5.1	Object Adapter	65
3.5.2	Facade	67
3.5.3	Observer	68
3.5.4	Dependency Injection	70
3.5.5	Pattern valutati e non adottati	71

3.6	Idiomi e convenzioni di codifica	73
3.6.1	Generatori asincroni e propagazione dell'annullamento	73
3.6.2	Tipizzazione dei confini fra i livelli	74
3.6.3	Denominazione, struttura dei file e gestione degli errori	75
4	Flusso dei dati	76
4.1	Generazione assistita da LLM_G con risposta in streaming	76
4.2	Annullamento di un'operazione in corso	80
4.3	Generazione a partire da un collegamento	81
4.4	Salvataggio e caricamento di una nota	83
4.4.1	Persistenza su file system	83
4.4.2	Continuità della sessione	86
4.4.3	Recupero dei metadati dall'oggetto File	89
4.5	Propagazione e presentazione degli errori	90
5	Tracciamento dei requisiti$_G$	92
5.1	Criteri di tracciamento	92
5.2	Requisiti $_G$ funzionali	92
5.3	Requisiti $_G$ di qualità	92
5.4	Requisiti $_G$ di vincolo $_{GG}$	92
5.5	Grafici riassuntivi	92

Elenco delle tabelle

1	Tecnologie adottate per il frontend _G	14
2	Tecnologie adottate per il backend	17
3	Strumenti di analisi statica	23
4	Strumenti di analisi dinamica	24
5	Tecnologie di infrastruttura	26
6	Vocabolario architetturale	32
7	Contratti di dipendenza dichiarati in <code>api/.importlinter</code>	57

Elenco delle figure

1	Diagramma di deployment dell'ambiente di sviluppo	35
2	Diagramma UML 3.3-1 - Componenti UI e Store	37
3	Diagramma UML 3.3-2 -Hooks, Librerie e Dipendenze Esterne	38
4	Diagramma UML 3.3.1 - Organizzazione dei sorgenti del frontend	39
5	Diagramma UML 3.3.2-1 - Sidebar	40
6	Diagramma UML 3.3.2-2 - TopBar	41
7	Diagramma UML 3.3.2-3 - ViewToggle	41
8	Diagramma UML 3.3.2-4 - EditorToolbar	42
9	Diagramma UML 3.3.2-5 - Editor	43
10	Diagramma UML 3.3.2-6 - Preview	43
11	Diagramma UML 3.3.2-7 - MarkdownView	44
12	Diagramma UML 3.3.2-8 - AiActionDialog	45
13	Diagramma UML 3.3.3-1 - useEditorStore	47
14	Diagramma UML 3.3.3-2 - notes	48
15	Diagramma UML 3.3.4-1 - Sincronizzazione store → editor	49
16	Diagramma UML 3.3.4-2 - Sincronizzazione editor → store	51
17	Diagramma UML 3.3.5 - Accesso al servizio applicativo	53
18	Diagramma UML 3.3.6 - Persistenza lato client	55
19	Architettura del servizio applicativo	56
20	Diagramma delle classi dell'Object Adapter	67
21	Diagramma della facciata di accesso al servizio applicativo	68
22	Diagramma soggetto-osservatori degli store e dei selettori	69
23	Diagramma della risoluzione della dipendenza e della sostituzione con un doppio	71
24	Diagramma di sequenza di un'operazione assistita con risposta in streaming	80
25	Diagramma UML 4.4.1-1 - fileSystem	84
26	Diagramma UML 4.4.1-2 - saveNoteToFile	85
27	Diagramma UML 4.4.1-3 - openNoteFromFile	86
28	Diagramma UML 4.4.2-1 - salvataggio automatico in localStorage	87
29	Diagramma UML 4.4.2-2 - lettura all'avvio	88
30	Diagramma UML 4.4.2-3 - continuità della sessione	89
31	Diagramma UML 4.4.3 - nota creata nel prodotto vs nota importata da disco	90

1 Introduzione

1.1 Scopo del documento

Il presente documento descrive l'architettura del sistema_{GG} **Second Brain**, prodotto realizzato dal gruppo **The Bitless** in risposta al capitolato_{GG} **C6** proposto da **Zucchetti S.p.A.**. Mentre l'Analisi dei Requisiti_{GG} stabilisce *che cosa* il sistema_{GG} deve fare, la Specifica Tecnica stabilisce *come* il sistema_{GG} è costruito per farlo.

Nello specifico, il documento persegue i seguenti obiettivi:

- **Motivazione delle scelte tecnologiche:** espone le tecnologie adottate per ciascun livello del prodotto, i bisogni tecnici che ne hanno determinato la selezione e le alternative valutate e scartate, con le relative motivazioni.
- **Descrizione dell'architettura:** illustra la scomposizione del sistema_{GG} nelle sue componenti, le responsabilità attribuite a ciascuna di esse e le interazioni che ne regolano la collaborazione, sia in termini logici sia in termini di distribuzione delle parti in esecuzione.
- **Documentazione dei design pattern:** riporta i pattern architetturali e di progettazione adottati, motivandone l'impiego rispetto al problema specifico che risolvono all'interno del prodotto.
- **Tracciamento dei requisiti_G:** correla le componenti architetturali ai requisiti_{GG} individuati nell'Analisi dei Requisiti_{GG}, così da rendere verificabile la copertura del perimetro funzionale concordato con la proponente_{GG}.

Il documento costituisce il riferimento progettuale per le successive fasi di realizzazione e di manutenzione del prodotto: ogni intervento sul codice deve risultare coerente con quanto qui descritto e, qualora introduca una variazione architetturale, deve essere preceduto dall'aggiornamento della presente specifica.

La Specifica Tecnica dà continuità al **Proof of Concept_G** sviluppato per la **Requirements and Technology Baseline_G**, nel quale le tecnologie qui adottate sono state selezionate e sperimentate. Il documento non descrive però quell'artefatto_{GG}: **describe il repository_G di prodotto**, che da allora è stato realizzato e nel quale quelle scelte sono state consolidate, in parte riviste e portate al livello di dettaglio necessario allo sviluppo_{GG} completo. Ogni affermazione fattuale che segue è verificata contro quel repository_{GG}, che è l'unico termine di confronto del documento.

Come gli altri documenti del gruppo, la stesura segue un approccio **incrementale_G**: il documento viene raffinato a ogni iterazione, in modo da riflettere costantemente lo stato effettivo del prodotto.

1.2 Scopo del prodotto

Il capitolato_{GG} **C6** di **Zucchetti S.p.A.** richiede la realizzazione di **Second Brain**, un'applicazione web per la scrittura e la gestione di note in formato **Markdown_G**, assistita da modelli linguistici di grandi dimensioni (**LLM_G**).

L'interfaccia si articola in due pannelli affiancati: a sinistra un editor in cui l'utente compone il testo nella sintassi **Markdown_G**, a destra un'anteprima che ne mostra il rendering aggiornato in tempo reale. Su questa base si innestano le funzioni assistite dall'**LLM_{GG}**, che operano sul testo selezionato o sull'intera nota: riassunto, riscrittura e adattamento del tono, correzione, traduzione multilingue, analisi critica secondo la tecnica dei **Sei Cappelli per Pensare_G** di

Edward de Bono e generazione di testo secondo il paradigma del **Distant Writing_G**, in cui l'utente descrive sinteticamente il contenuto desiderato e delega al modello la stesura. A queste si aggiunge la generazione a partire da un collegamento, in cui l'utente indica l'indirizzo di una pagina web e il sistema_{GG} ne estrae il contenuto per sottoporlo al modello. Le note prodotte sono salvate e ricaricate sul file system locale dell'utente in formato Markdown_G.

Il valore del prodotto non risiede quindi nella sola redazione del testo, ma nel ruolo attivo assunto dal sistema_{GG}, che affianca l'utente nelle fasi di ideazione, revisione e riorganizzazione dei contenuti.

Per la trattazione completa del perimetro funzionale, dei casi d'uso_{GG} e della classificazione dei requisiti_{GG} in obbligatori, desiderabili e opzionali si rimanda all'*Analisi dei Requisiti*, di cui il presente documento non intende costituire una duplicazione.

1.3 Glossario

La creazione e lo sviluppo_{GG} di un sistema_{GG} software richiedono una complessa operazione di progettazione e analisi del dominio_{GG}, attività che deve necessariamente precedere la stesura del codice. Il gruppo si impegna a raccogliere e organizzare tali informazioni in modo che siano facilmente accessibili, al fine di favorire la massima asincronia ed efficienza nelle attività di progetto.

La principale fonte di ambiguità nello svolgimento delle attività è rappresentata dall'incomprensione dei termini tecnici o specialistici adottati. A tale scopo, la nomenclatura di riferimento viene raccolta nel *glossario*, un documento incrementale_{GG} volto a definire ogni termine rilevante per il dominio_{GG} del progetto.

Il gruppo si impegna a contrassegnare ogni occorrenza di un termine presente nel glossario mediante l'apposizione di una lettera **G** a pedice, come esemplificato di seguito:

parola_G

Al fine di garantire una corretta comprensione del dominio_{GG}, è fondamentale che ogni membro del gruppo consulti periodicamente il glossario, assicurando così il costante allineamento sulle definizioni e sui concetti chiave.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato C6 - Second Brain:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
Ultimo accesso: 20 maggio 2026.
- **Regolamento del Progetto Didattico:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
Ultimo accesso: 23 maggio 2026.
- **Norme di Progetto v2.0.0:**
https://thebitless.live/PB/doc_interna/norme-di-progetto/norme-di-progetto.pdf
Ultimo accesso: 18 agosto 2026.

1.4.2 Riferimenti informativi

Documentazione di progetto

- **Glossario v2.0.0:**
https://thebitless.live/PB/doc_interna/glossario/glossario.pdf
Ultimo accesso: 18 agosto 2026.
- **Analisi dei Requisiti v2.0.0:**
https://thebitless.live/PB/doc_esterna/analisi-dei-requisiti/analisi-dei-requisiti.pdf
Ultimo accesso: 18 agosto 2026.
- **Verbale interno VI_2026-05-12 – Scelta delle tecnologie per il PoC_G:**
https://thebitless.live/RTB/doc_interna/verbali/VI_2026-05-12_quarto-sprint/VI_2026-05-12_quarto-sprint.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale interno VI_2026-05-20 – Progettazione del PoC_G:**
https://thebitless.live/RTB/doc_interna/verbali/VI_2026-05-20_progettazione-PoC/VI_2026-05-20_progettazione-PoC.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale esterno VE_2026-05-14 – Approvazione delle tecnologie da parte della proponente_G:**
https://thebitless.live/RTB/doc_esterna/verbali/VE_2026-05-14_AdR-scelta-tecnologie/VE_2026-05-14_AdR-scelta-tecnologie.pdf
Ultimo accesso: 27 luglio 2026.

Documentazione delle tecnologie adottate Documentazione ufficiale delle tecnologie descritte nella sezione 2. *Ultimo accesso* a tutte le risorse elencate di seguito: 27 luglio 2026.

- **TypeScript:** <https://www.typescriptlang.org/docs/>
- **React:** <https://react.dev/>
- **Zustand:** <https://zustand.docs.pmnd.rs/>
- **react-markdown:** <https://github.com/remarkjs/react-markdown>
- **Tailwind CSS:** <https://tailwindcss.com/docs>
- **shadcn/ui:** <https://ui.shadcn.com/>
- **Radix UI Primitives:** <https://www.radix-ui.com/primitives>
- **CodeMirror 6:** <https://codemirror.net/docs/>
- **Vite:** <https://vite.dev/>
- **Python 3.12:** <https://docs.python.org/3.12/>
- **FastAPI:** <https://fastapi.tiangolo.com/>
- **Pydantic:** <https://docs.pydantic.dev/>
- **httpx:** <https://www.python-httpx.org/>
- **Uvicorn:** <https://www.uvicorn.org/>
- **Tavily:** <https://docs.tavily.com/>
- **LiteLLM:** <https://docs.litellm.ai/>
- **Server-Sent Events (MDN):** https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events

- **File System Access API (MDN):** https://developer.mozilla.org/en-US/docs/Web/API/File_System_API
- **Specifica OpenAPI:** <https://spec.openapis.org/oas/latest.html>
- **openapi-typescript:** <https://openapi-ts.dev/>
- **ESLint:** <https://eslint.org/docs/latest/>
- **Ruff:** <https://docs.astral.sh/ruff/>
- **import-linter:** <https://import-linter.readthedocs.io/>
- **Vitest:** <https://vitest.dev/>
- **Copertura del codice in Vitest:** <https://vitest.dev/guide/coverage>
- **Testing Library:** <https://testing-library.com/docs/react-testing-library/intro/>
- **pytest:** <https://docs.pytest.org/>
- **Docker:** <https://docs.docker.com/>
- **Docker Compose:** <https://docs.docker.com/compose/>
- **GitHub Actions:** <https://docs.github.com/actions>
- **pnpm:** <https://pnpm.io/>
- **uv:** <https://docs.astral.sh/uv/>
- **Node.js – calendario di supporto delle versioni:** <https://github.com/nodejs/release>
- **File API – interfaccia File (MDN):** <https://developer.mozilla.org/en-US/docs/Web/API/File>

Riferimenti metodologici

- **E. Gamma, R. Helm, R. Johnson, J. Vlissides,** *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- **C4 model for visualising software architecture:**
<https://c4model.com/>
Ultimo accesso: 27 luglio 2026.

2 Tecnologie

2.1 Criteri di scelta

Le tecnologie descritte in questa sezione non sono state selezionate in astratto, sulla base della sola diffusione o popolarità, ma in risposta a bisogni tecnici precisi derivati dal capitolato_{GG} e dall'Analisi dei Requisiti_{GG}. Ciascuno dei bisogni elencati di seguito vincola una o più delle scelte successive, e costituisce il criterio rispetto al quale tali scelte vanno valutate.

Risposta progressiva Un riassunto o una generazione di testo richiedono al modello linguistico un tempo di elaborazione dell'ordine dei secondi. Mantenere l'utente davanti a un semplice indicatore di attesa per tutta la durata dell'operazione non è accettabile sul piano dell'usabilità_G: il testo deve comparire progressivamente mentre viene prodotto (*streaming*). Questo bisogno determina la scelta di un livello server asincrono, di un client HTTP che supporti la lettura incrementale_G della risposta e di un protocollo di trasporto adatto al flusso unidirezionale di dati testuali. Il criterio discende dal requisito_G **R-109-F-De**, che impone un indicatore di avanzamento per l'intera durata delle operazioni con attesa percepibile, e da **R-79-F-De**, che richiede la possibilità di annullare un'elaborazione in corso: entrambi presuppongono che il server mantenga il controllo dell'operazione mentre questa procede, e non si limiti a restituirne l'esito finale.

Riservatezza delle credenziali dei fornitori esterni L'accesso ai modelli linguistici avviene tramite una chiave di autenticazione fornita dalla proponente_{GG}, in attuazione del requisito di vincolo_{GG} **R-2-V-Ob**, che prescrive l'interfacciamento con i servizi LLM_{GG} tramite API_{GG}. Tale chiave non deve in alcun caso raggiungere il browser, dove sarebbe leggibile da chiunque ispezioni il traffico o il codice dell'applicazione. È quindi necessaria una componente server intermediaria che detenga la chiave e sia l'unico soggetto a comunicare con il fornitore; la medesima componente risolve anche il vincolo_{GG} **R-3-V-Ob** sulla *same-origin policy*, poiché è essa a esporre al browser un'origine controllata, configurata mediante la politica CORS.

Il criterio si estende a ogni fornitore esterno contattato dal prodotto, non al solo fornitore dei modelli. La funzione di generazione a partire da un collegamento (**R-74-F-De**) richiede l'estrazione del contenuto di una pagina web indicata dall'utente, operazione affidata a un secondo servizio esterno dotato di una propria chiave: anche questa è custodita dal backend e non transita dal browser. Va dichiarato, per trasparenza verso l'utente e verso la proponente_{GG}, quali dati lasciano il perimetro del sistema_{GG}: verso il fornitore dei modelli linguistici transitano il testo sottoposto all'operazione (la porzione selezionata o l'intera nota) e le istruzioni costruite dall'applicazione; verso il servizio di estrazione transita il solo indirizzo fornito dall'utente. Nessuno dei due riceve dati identificativi dell'utente, che il prodotto non raccoglie.

Nessun file è trasmesso automaticamente: una nota aperta da disco resta nel browser finché l'utente non ne affida il contenuto a un'operazione assistita. Aprire un file non comporta quindi alcuna trasmissione, chiederne il riassunto sì.

Rendering sicuro del Markdown_G Il requisito_G **R-113-F-Ob** prevede una componente di rendering che trasformi il testo Markdown_G in una presentazione grafica formattata. Il testo restituito dal modello linguistico non è però contenuto fidato: è generato da un sistema_{GG} esterno a partire da input_{GG} che l'utente controlla solo in parte. Deve essere mostrato all'utente come HTML formattato senza che ciò apra la strada a vulnerabilità di tipo *cross-site scripting* (XSS). La tecnologia di rendering deve quindi essere sicura per costruzione, non affidarsi a una sanificazione applicata a posteriori. Al medesimo criterio si riconduce **R-110-F-Ob**, che impone

messaggi di errore in linguaggio naturale privi di dettagli tecnici: la gestione degli errori del modello non deve riversare nell'interfaccia contenuto proveniente dall'esterno.

Editor ricco ma stabile L'editor deve offrire evidenziazione della sintassi, cronologia delle modifiche con annullamento e ripristino (**R-7-F-De**, **R-8-F-De**), e un comportamento accessibile da tastiera e da lettore di schermo; **R-111-F-Ob** ne prescrive la realizzazione come componente capace di ospitare testo su più righe. Deve inoltre mantenere invariati cursore e posizione di scorrimento mentre la parte di interfaccia che presenta l'esito in arrivo si aggiorna in streaming, condizione che esclude soluzioni in cui la ricostruzione della vista coinvolge l'intera area di lavoro.

Manutenibilità_G con un gruppo di sette persone Il prodotto è sviluppato in parallelo da sette persone con livelli di esperienza eterogenei. Ciò richiede codice tipizzato, in cui gli errori di interfaccia emergano in fase di compilazione anziché in esecuzione, confini netti fra i livelli dell'applicazione e verifica_{GG} automatica delle modifiche prima della loro integrazione_{GG}. Il criterio è vincolato da tre requisiti_{GG} di qualità: **R-4-Q-Ob** richiede che ogni pull request_{GG} sia sottoposta a revisione tra pari e superi i test_{GG} automatici prima dell'integrazione_{GG} nel ramo principale, **R-2-Q-Ob** richiede una copertura del codice_G conforme alle soglie del Piano di Qualifica_G, **R-10-Q-Ob** impone il rispetto delle Norme di Progetto_{GG}. Serve quindi un'infrastruttura di *Continuous Integration_G* che applichi questi controlli in modo automatico e uniforme sui due livelli.

Perimetro tecnologico ammesso dal capitolato_G Il requisito di vincolo_{GG} **R-7-V-Op** stabilisce che la componente server-side, se realizzata, possa essere sviluppata utilizzando Java o Python. La scelta di Python ricade quindi all'interno del perimetro tecnologico ammesso dalla proponente_{GG}. Sul versante client, **R-1-V-Ob** circoscrive il perimetro alle Tecnologie Web_{GG} e ne fissa gli ambienti di esecuzione di riferimento: le versioni recenti di Google Chrome, Mozilla Firefox e Microsoft Edge. Questo vincolo_{GG} ha conseguenza diretta sulla strategia di persistenza descritta nella sezione 2.4, poiché le tre famiglie di browser non offrono le medesime interfacce di accesso al file system.

Un criterio comune Sotto le scelte che seguono c'è un solo criterio: l'equilibrio fra controllo e velocità di sviluppo_{GG}. Il gruppo ha preferito tecnologie che lasciano visibile e modificabile ciò che accade, evitando sia quelle che nascondono il comportamento dietro astrazioni ampie e prescrittive, sia quelle che obbligano a riscrivere da zero funzionalità già disponibili. A parità di adeguatezza tecnica ha pesato la maturità dell'ecosistema, con documentazione estesa e molte risorse reperibili: per un gruppo che affronta quasi tutte queste tecnologie per la prima volta è un fattore decisivo.

2.2 Frontend

Il frontend_{GG} è un'applicazione a pagina singola eseguita interamente nel browser. Le tecnologie sono presentate nell'ordine con cui si sovrappongono: il linguaggio, la libreria che struttura la vista, l'ecosistema di librerie che ne realizza le funzioni specifiche e, infine, gli strumenti che ne governano la costruzione.

Tabella 1: Tecnologie adottate per il frontend_G

Tecnologia	Versione	Descrizione e ruolo nel progetto
TypeScript	6.0.3	Linguaggio di sviluppo _{GG} dell'intero frontend _{GG} . Estende JavaScript con un sistema _G di tipi statici rimossi in fase di compilazione (<i>type erasure</i>): spostano in compilazione errori che altrimenti emergerebbero in produzione, soprattutto sulle interfacce fra componenti. Non sopravvivendo alla compilazione, non possono però validare i dati che arrivano dall'esterno: quel controllo resta al backend.
React	19.2.7	Libreria per la costruzione dell'interfaccia, organizzata a componenti. Editor, anteprima, barra degli strumenti e pannelli delle funzioni assistite dall'LLM _{GG} sono componenti React distinti. Il meccanismo di <i>reconciliation</i> applica al DOM le sole modifiche necessarie a ogni cambiamento di stato: durante lo streaming, dove lo stato cambia decine di volte al secondo, evita di ricostruire l'intera pagina. L'applicazione non impiega alcun router: l'interazione si svolge in una sola vista, e un instradamento lato client sarebbe una complessità non giustificata.
Zustand	5.0.14	Gestione dello stato applicativo condiviso fra i componenti (testo della nota, selezione corrente, testo in arrivo dallo streaming, stato di errore, modalità di visualizzazione). Espone selettori granulari, ciascuno dei quali osserva una singola porzione di stato: durante lo streaming la ridipintura interessa la sola parte di interfaccia che presenta l'esito in arrivo, mentre l'editor resta immobile e conserva cursore e posizione di scorrimento.
react-markdown	10.1.0	Rendering del Markdown _G nel pannello di anteprima. Il testo viene analizzato e trasformato in un albero sintattico astratto, dal quale la libreria costruisce direttamente gli elementi React corrispondenti. In nessun punto della catena viene iniettato HTML grezzo nel documento: la libreria è quindi sicura per costruzione rispetto agli attacchi XSS, senza necessità di un passaggio di sanificazione. Il requisito è critico, poiché il testo da rendere proviene in larga parte dal modello linguistico.
remark-gfm	4.0.1	Estensione della pipeline di analisi con le costruzioni <i>GitHub Flavored Markdown</i> (tabelle, elenchi di attività, testo barrato, collegamenti automatici), attese dall'utenza di riferimento del prodotto. Le tabelle in particolare non appartengono al Markdown _G di base: l'estensione è la condizione perché i requisiti _{GG} R-34-F-De ... R-38-F-De (inserimento di una tabella e gestione di righe e colonne, UC44 ... UC48) trovino una resa nell'anteprima.
remark-math rehype-katex KaTeX	6.0.0 7.0.1 0.17.0	Riconoscimento delle espressioni matematiche nel sorgente Markdown _G e loro composizione tipografica nell'anteprima. Realizzano i requisiti _{GG} R-28-F-De e R-29-F-De (inserimento e rimozione di una formula scientifica o matematica, UC35 e UC37), che senza un motore di composizione dedicato non sarebbero verificabili _G sul lato dell'anteprima. Operano come estensioni della stessa pipeline ad albero sintattico, preservandone la proprietà di non iniettare HTML grezzo.

Tecnologia	Versione	Descrizione e ruolo nel progetto
rehype-highlight highlight.js	7.0.2 11.11.1	Evidenziazione della sintassi all'interno dei blocchi di codice mostrati nell'anteprima. Discende dai requisiti _{GG} R-26-F-De , R-27-F-De e R-119-F-De (inserimento, rimozione e modifica di un blocco di codice sorgente, UC32, UC34 e UC33), che presuppongono una resa del blocco di codice distinta dal testo ordinario.
Tailwind CSS	4.3.1	Sistema _G di stile <i>utility-first</i> : la presentazione è espressa con classi elementari applicate direttamente al marcatore. L'insieme chiuso di valori che ne risulta (spaziature, colori, dimensioni tipografiche) mantiene uniforme l'aspetto fra parti dell'interfaccia scritte da persone diverse. Il plugin <code>@tailwindcss/typography</code> (0.5.20) fornisce gli stili applicati al risultato del rendering Markdown _G .
shadcn/ui Radix UI	— 1.5.0	Insieme di componenti di interfaccia (pulsanti, aree di testo, finestre di dialogo, avvisi). shadcn/ui non è una dipendenza installata ma un catalogo da cui il codice sorgente dei componenti viene copiato nel progetto: resta quindi interamente leggibile e modificabile dal gruppo, senza vincolo _G di adozione verso un fornitore esterno. I componenti poggiano su Radix UI, che fornisce le primitive di comportamento accessibile (navigazione da tastiera, gestione del focus, attributi per i lettori di schermo), la cui realizzazione manuale corretta sarebbe onerosa e soggetta a errori.
CodeMirror 6	6.0.2	Editor di testo del pannello sinistro. Fornisce nativamente evidenziazione della sintassi Markdown _G (modulo <code>@codemirror/lang-markdown</code> 6.5.0), cronologia delle modifiche con annullamento e ripristino (<code>@codemirror/commands</code> 6.10.3) e supporto all'accessibilità, a fronte di un ingombro dell'ordine del centinaio di kilobyte. L'architettura modulare consente di includere le sole estensioni effettivamente utilizzate.
lucide-react	1.18.0	Insieme di icone vettoriali impiegate nella barra degli strumenti e nei comandi dell'interfaccia.
Vite	8.0.16	Strumento di sviluppo _{GG} e costruzione del frontend _{GG} . In sviluppo _{GG} aggiorna i soli moduli modificati mantenendo lo stato dell'applicazione, senza ricostruire l'intero pacchetto; in produzione genera il pacchetto ottimizzato. La sua configurazione è riutilizzata dall'infrastruttura di test _G descritta nella sezione 2.5.

Alternative considerate

- **Angular, Vue e Svelte al posto di React**

Il confronto documentato nel verbale interno del 12 maggio 2026 ha contrapposto React e Svelte. Svelte adotta un approccio a compilatore, con prestazioni superiori e pacchetto più leggero, ma ha un ecosistema più giovane: meno librerie mature per le esigenze del prodotto e meno risorse di apprendimento, fattore che pesa per un gruppo alla prima esperienza con la tecnologia. Vue è maturo, ma la via più diffusa per il rendering del Markdown_G nel suo ecosistema produce HTML e lo inietta nel documento, esattamente il rischio_G che il criterio del rendering sicuro impone di evitare. Angular è completo e fortemente prescrittivo, sovradimensionato per un'applicazione a vista singola. A decidere è stato il fatto che le librerie di cui il prodotto ha bisogno (rendering Markdown_G sicuro, componenti accessibili, integrazione_{GG} dell'editor) nascono tutte pensando a React come primo ambiente.

- **Redux, Context API e Jotai al posto di Zustand**

Redux impone una quantità di codice di supporto (azioni, riduttori, selettori memorizzati) sproporzionata rispetto alla dimensione dello stato da gestire. La Context API_G, disponibile nativamente in React, non offre sottoscrizioni granulari: ogni variazione del valore del contesto provoca la ridipintura di tutti i componenti che vi accedono, comportamento penalizzante durante lo streaming, in cui lo stato cambia con frequenza elevata e ridipingere l'editor comporterebbe la perdita della posizione del cursore. Jotai adotta un modello ad atomi tecnicamente adeguato, ma richiede di scomporre lo stato in unità elementari fin dall'inizio: Zustand ottiene il medesimo risultato con un modello concettuale più semplice e una superficie di apprendimento minore.

- **marked e markdown-it al posto di react-markdown**

Entrambe le librerie convertono il Markdown_G in una stringa HTML, che dovrebbe poi essere inserita nel documento tramite un meccanismo di iniezione diretta. Servirebbe allora un passaggio esplicito di sanificazione, la cui correttezza dipenderebbe dalla configurazione scelta e andrebbe verificata_G a parte. Con react-markdown il problema non si pone.

- **MUI e Chakra UI al posto di shadcn/ui**

Offrono un numero maggiore di componenti già pronti, ma impongono un linguaggio visivo marcato, la cui personalizzazione richiede di intervenire contro le impostazioni predefinite della libreria; inoltre il codice dei componenti resta all'interno della dipendenza, non modificabile. L'alternativa opposta, i CSS Modules, non aggiunge alcuna dipendenza ma non fornisce né un sistema_G di valori condiviso né componenti accessibili, che andrebbero realizzati integralmente dal gruppo.

- **Monaco e textarea nativa al posto di CodeMirror**

Monaco è l'editor impiegato in Visual Studio Code: è progettato per la scrittura di codice sorgente e la sua dotazione di funzioni (completamento automatico, analisi del linguaggio, riquadro di anteprima) eccede quanto richiesto dalla redazione di note, a fronte di un ingombro di oltre un ordine di grandezza superiore. L'elemento `textarea` nativo non comporta alcuna dipendenza, ma è privo di evidenziazione della sintassi e di una cronologia delle modifiche strutturata.

- **Webpack e Create React App al posto di Vite**

Ricostruiscono l'intero pacchetto a ogni modifica, con tempi di attesa in sviluppo_{GG} sensibilmente superiori, e richiedono una configurazione più estesa e frammentata. Create React App non è inoltre più oggetto di manutenzione attiva.

2.3 Backend

Il backend è una componente server che espone alcune API_{GG} tramite HTTP. Le sue responsabilità sono quattro:

1. custodire le chiavi di accesso ai fornitori esterni, così che non raggiungano mai il browser (criterio della riservatezza delle credenziali);
2. comporre le istruzioni da inviare al modello linguistico;
3. ritrasmettere al browser il testo prodotto man mano che diviene disponibile (criterio della risposta progressiva);
4. acquisire il contenuto testuale di una pagina web indicata dall'utente, quando la generazione avviene a partire da un collegamento.

La quarta responsabilità discende dal requisito_G **R-74-F-De** (UC67.2 e UC67.5) e non è riducibile alle altre tre: comporta un secondo fornitore esterno, distinto da quello dei modelli, e per

questo è trattata separatamente nel paragrafo *Estrazione del contenuto delle pagine web*. Come per il frontend_{GG}, le tecnologie sono presentate nell'ordine linguaggio, framework_{GG}, librerie, strumenti.

Tabella 2: Tecnologie adottate per il backend

Tecnologia	Versione	Descrizione e ruolo nel progetto
Python	3.12	Linguaggio di sviluppo _{GG} del backend. Rientra nel perimetro tecnologico ammesso dal requisito _G R-7-V-Op. Dispone del supporto nativo alla programmazione asincrona (async/await), condizione necessaria per gestire attese di rete prolungate senza bloccare il processo, e dell'ecosistema più esteso per l'integrazione _{GG} con i modelli linguistici.
FastAPI	0.136.1	Framework _{GG} web su cui è costruita l'API _{GG} . È asincrono per impostazione, condizione senza la quale lo streaming non sarebbe possibile; deriva la validazione _G dei corpi delle richieste direttamente dalle annotazioni di tipo, senza codice di controllo scritto a mano; genera automaticamente lo schema OpenAPI dell'interfaccia esposta. La risposta in streaming è realizzata mediante StreamingResponse , che consuma un generatore asincrono e gestisce la trasmissione a blocchi lungo la connessione HTTP.
Pydantic	2.13.4	Validazione _G e serializzazione dei dati in ingresso e in uscita. Gli schemi Pydantic costituiscono la fonte unica di verità dei contratti dell'API _{GG} : da essi FastAPI deriva lo schema OpenAPI e, da quest'ultimo, vengono generati i tipi TypeScript utilizzati dal frontend _{GG} (si veda la sezione 2.6). Il contratto è definito quindi una volta sola, in un solo punto del sistema _{GG} .
pydantic-settings	2.14.1	Lettura e validazione _G della configurazione applicativa (indirizzo del gateway dei modelli, identificativo del modello, chiave di accesso, origini ammesse dalla politica CORS) a partire dalle variabili d'ambiente. Le impostazioni sono esposte tramite un'unica istanza per processo.
httpx	0.28.1	Client HTTP asincrono impiegato per la comunicazione verso il provider dei modelli linguistici. Legge la risposta a blocchi man mano che arriva, senza attenderne il completamento. La risposta vive entro un blocco a gestione automatica della risorsa: all'annullamento il codice chiude esplicitamente il solo generatore del dominio _{GG} , e l'uscita da quel blocco chiude la connessione al provider per comportamento documentato della libreria.
tavily-python	0.7.26	Estrazione del contenuto testuale delle pagine web nella funzione di generazione a partire da un collegamento (requisito _G R-74-F-De , UC67.2 e UC67.5). Restituisce il solo testo della pagina, già ripulito dal marcatore e dagli elementi accessori, e ne consente il troncamento a una lunghezza massima prima dell'invio al modello. È un client verso un servizio esterno: si veda il paragrafo <i>Estrazione del contenuto delle pagine web</i> per il perimetro dei dati trasmessi e per l'alternativa scartata.
Uvicorn	0.47.0	Server ASGI sul quale l'applicazione FastAPI è posta in esecuzione. Costituisce l'implementazione dell'interfaccia asincrona fra il server HTTP e il codice applicativo.

Tecnologia	Versione	Descrizione e ruolo nel progetto
Server-Sent Events	—	Protocollo di trasporto dei blocchi testuali dal backend al browser. È un formato testuale unidirezionale che viaggia su una normale connessione HTTP mantenuta aperta, in cui ogni evento è una riga con prefisso data: e la fine del flusso è segnalata da un marcatore convenzionale. Qui i dati vanno in una direzione sola, dal server al client, ed è esattamente quello che il protocollo offre.

Accesso ai modelli linguistici L'accesso ai modelli avviene attraverso un **gateway LiteLLM**, componente esterna che espone un'interfaccia compatibile con quella di OpenAI e uniforma l'accesso a fornitori diversi. Poiché si tratta di un servizio contattato via HTTP e non di una libreria installata nel progetto, non figura fra le dipendenze del backend e non è associato a una versione dichiarata nei file di progetto.

La comunicazione con il gateway è confinata dietro l'astrazione **LLMClient**, una classe astratta che dichiara la sola operazione di produzione incrementale_G del testo. L'implementazione concreta che dialoga con LiteLLM, interpretandone il formato di risposta e traducendone gli errori, ne è l'unico realizzatore, secondo il pattern Adapter descritto nella sezione dedicata all'architettura. Né i router né la logica di dominio_{GG} invocano direttamente LiteLLM: conoscono unicamente l'interfaccia astratta. La sostituzione del gateway con un fornitore diverso comporta pertanto la scrittura di una nuova implementazione dell'interfaccia, e nessuna modifica al resto del backend.

Estrazione del contenuto delle pagine web Il requisito_G **R-74-F-De** consente all'utente di indicare un URL come fonte di contesto per la generazione automatica di testo (UC67.2). Poiché il modello linguistico non accede alla rete, il contenuto della pagina deve essere acquisito dal prodotto e passato al modello come parte delle istruzioni. L'operazione è affidata a **Tavily**, servizio esterno che restituisce il testo di una pagina già ripulito dal marcatore, dalla navigazione e dagli elementi accessori.

Si tratta del secondo fornitore esterno del sistema_{GG}, dotato di una propria chiave di accesso. Coerentemente con il criterio della riservatezza delle credenziali, la chiave è custodita dal backend e non raggiunge il browser; verso il servizio transita esclusivamente l'indirizzo indicato dall'utente, non il testo della nota. Il contenuto restituito è troncato a una lunghezza massima prima di essere inoltrato al modello, così da mantenere prevedibile la dimensione delle istruzioni.

L'alternativa considerata è l'estrazione lato server con una libreria locale: **trafilatura** e le realizzazioni Python dell'algoritmo *Readability* scaricano la pagina con il client HTTP già presente nel progetto e ne isolano il contenuto principale senza coinvolgere alcun terzo. Eliminerebbero un fornitore, una chiave e un punto di uscita dei dati (vantaggio non trascurabile), ma sposterebbero sul gruppo la qualità dell'estrazione, che su pagine costruite dinamicamente tramite JavaScript risulta sensibilmente inferiore, e richiederebbero di gestire in proprio reindirizzamenti, codifiche e tempi di risposta dei siti contattati. Dato che R-74-F-De è desiderabile e non obbligatorio, il gruppo ha preferito la soluzione che ne rende prevedibile il completamento, accettando la dipendenza esterna.

Le due dipendenze esterne sono trattate allo stesso modo: come l'accesso al gateway passa dall'interfaccia astratta **LLMClient**, così l'estrazione passa da **ContentExtractor**, dichiarata anch'essa fra le porte del dominio_{GG} con il proprio tipo di errore e raggiunta dalle rotte per iniezione della dipendenza (sezione 3.4.5). Sostituire il servizio di estrazione richiede quindi un nuovo adattatore e una modifica al solo modulo di composizione, senza toccare il dominio_{GG} né le rotte. Ciò che distingue le due dipendenze non è la sostituibilità, ma la criticità: senza la chiave del gateway il processo non parte, perché nessuna funzione assistita sarebbe servibile,

mentre senza la chiave del servizio di estrazione viene meno la sola generazione a partire da un collegamento, segnalata per singola richiesta.

Alternative considerate

- **Node.js con TypeScript e Go al posto di Python**

Entrambi sarebbero tecnicamente adeguati: Node.js consentirebbe di adottare un unico linguaggio sui due livelli, Go offrirebbe prestazioni superiori nella gestione di connessioni concorrenti prolungate. Nessuno dei due rientra però nel perimetro tecnologico ammesso dal requisito_G R-7-V-Op, che indica Java e Python. Java rientra nel perimetro ed è stato valutato, ma è stato scartato per la minore immediatezza del suo ecosistema nell'integrazione_{GG} con i modelli linguistici e per la maggiore verbosità rispetto alla dimensione contenuta del backend, che espone poche interfacce e non contiene logica di dominio_{GG} estesa.

- **Flask e Django al posto di FastAPI**

Flask è sincrono per impostazione: il supporto allo streaming asincrono richiederebbe estensioni aggiuntive e una configurazione non prevista dal suo modello di base, oltre a un livello di validazione_G scritto separatamente. Django è orientato ad applicazioni complete, con mappatore relazionale a oggetti, sistema_G di autenticazione e pannello di amministrazione integrati: componenti che il prodotto non utilizza, dato che non prevede persistenza su base di dati (si veda la sezione 2.4), e che ne renderebbero l'adozione sproporzionata.

- **requests e aiohttp al posto di httpx**

La libreria `requests` è lo standard di fatto per le richieste HTTP in Python, ma è esclusivamente sincrona: bloccherebbe il processo per l'intera durata della risposta del modello, vanificando la scelta di un framework_{GG} asincrono. La libreria `aiohttp` è asincrona e supporta lo streaming, ma espone un'ergonomia meno lineare nella gestione della chiusura anticipata delle connessioni, aspetto centrale per la funzione di annullamento della generazione.

- **SDK diretto del fornitore e LangChain al posto del gateway**

L'impiego dell'SDK proprietario di un singolo fornitore legherebbe il codice a quel fornitore, in un ambito in cui modelli e condizioni di accesso cambiano con frequenza elevata. LangChain, all'estremo opposto, introduce un livello di astrazione ampio e fortemente opinato (catene, agenti, memoria, strumenti) di cui il prodotto non utilizzerebbe che una frazione minima, a fronte di un aumento sensibile della superficie di dipendenza e di una minore prevedibilità del comportamento. Il gateway, unito all'interfaccia `LLMClient`, offre l'indipendenza dal fornitore senza alcuno dei due inconvenienti.

- **WebSocket e polling al posto dei Server-Sent Events**

I WebSocket stabiliscono un canale bidirezionale, con un protocollo di negoziazione dedicato e la necessità di gestire esplicitamente riconessioni e stato della connessione: capacità superflue, visto che nel prodotto i dati scorrono in una sola direzione. Il *polling* periodico richiederebbe di accumulare il testo prodotto lato server in attesa di essere richiesto, introducendo latenza e complessità di gestione dello stato intermedio.

2.4 Persistenza delle note

Il gruppo ha deliberato di non realizzare la persistenza delle note su base di dati. Le note non lasciano dunque la macchina dell'utente: sono salvate e ricaricate sul file system locale per il tramite del browser, e conservate fra una sessione e l'altra nell'archivio locale del browser stesso.

Entrambi i meccanismi sono lato client; nessun dato dell'utente raggiunge il servizio applicativo per esservi memorizzato.

Meccanismo adottato I meccanismi di persistenza lato client sono due, distinti per scopo e non alternativi. L'accesso al file system serve all'esportazione e all'importazione deliberate: è l'utente a chiedere esplicitamente di salvare una nota su disco o di aprirne una, e il file prodotto è un documento che gli appartiene e che può usare con altri strumenti. La persistenza nel browser serve invece alla continuità della sessione: senza di essa chiudere la scheda comporterebbe la perdita delle note in lavorazione, poiché non esiste alcun archivio server-side che le conservi. Il primo meccanismo è descritto di seguito, il secondo nel paragrafo *Continuità della sessione*.

Le operazioni di salvataggio e di caricamento su file sono realizzate con una strategia a due vie, selezionata a tempo di esecuzione in base alle capacità del browser ospite.

Quando il browser espone la **File System Access API**, verificata mediante la presenza dei metodi `showOpenFilePicker` e `showSaveFilePicker` sull'oggetto globale della finestra, l'applicazione apre la finestra di selezione file nativa del sistema_{GG} operativo. Il caricamento ottiene un riferimento al file scelto e ne legge il contenuto testuale; il salvataggio ottiene un riferimento al percorso di destinazione e vi scrive un flusso di byte. L'annullamento da parte dell'utente è riconosciuto e trattato come esito legittimo, non come errore.

Quando l'interfaccia non è disponibile (condizione che ricorre su Firefox e Safari, che non la implementano), l'applicazione ricade su meccanismi supportati universalmente. La doppia via non è una precauzione generica: il requisito di vincolo_{GG} **R-1-V-Ob** include Firefox fra gli ambienti di esecuzione di riferimento, quindi una realizzazione fondata sulla sola File System Access API_G lascerebbe scoperto uno dei tre browser prescritti. In caricamento crea un elemento `input` di tipo file e ne legge il contenuto tramite un lettore di file; in salvataggio costruisce un oggetto `Blob` con il contenuto della nota, ne ricava un indirizzo temporaneo e lo assegna a un collegamento di scaricamento attivato per via programmatica, delegando al browser la scelta della destinazione secondo le impostazioni dell'utente. L'indirizzo temporaneo viene rilasciato al termine dell'operazione.

A decidere è la presenza effettiva dei metodi, non l'identità del browser: l'applicazione non interroga l'agente utente. La funzione resta quindi disponibile ovunque, con l'unica differenza che dove l'interfaccia nativa esiste l'utente sceglie il percorso di destinazione, altrove il file segue le impostazioni di scaricamento predefinite.

Continuità della sessione Il secondo meccanismo non richiede alcuna azione dell'utente. Lo store delle note è avvolto in un middleware di persistenza che ne riversa lo stato nel `localStorage` del browser sotto la chiave `notes_persistence`, e lo rilegge all'avvio dell'applicazione: alla riapertura l'elenco delle note e la nota corrente sono quelli con cui la sessione precedente si era chiusa, e il contenuto della nota corrente è ricaricato nell'editor. Ciò che viene conservato è l'elenco delle note con i rispettivi identificativo, titolo, contenuto e istanti di creazione e di ultima modifica, oltre all'indicazione di quale sia la nota corrente.

Conta però *quando* il testo in lavorazione raggiunge quell'archivio, altrimenti al meccanismo si attribuisce una copertura che non ha: il contenuto dell'editor finisce nella nota corrente al cambio di nota e alla creazione di una nuova nota, non a ogni modifica del testo né alla chiusura della scheda. I metadati, assegnati alla creazione, sono invece persistiti immediatamente. Le modifiche successive all'ultimo cambio di nota non sono quindi conservate: è un limite noto della realizzazione attuale, non una proprietà del supporto.

I limiti sono quelli dell'archiviazione locale del browser. Lo spazio disponibile è soggetto a una quota, e superarla fa fallire la scrittura: l'applicazione se lo aspetta e ripristina lo stato precedente, invece di lasciare l'interfaccia incoerente. I dati sono confinati all'origine e al profilo

del browser in uso: non sono raggiungibili da un altro browser, da un altro dispositivo o da una finestra di navigazione anonima, non vi è alcuna sincronizzazione fra dispositivi, e la cancellazione dei dati di navigazione li rimuove. Questo meccanismo non è un sostituto della persistenza server-side: resta interamente lato client, e non copre alcuno dei requisiti_{GG} opzionali elencati più avanti in questa sezione.

Formato dei file Le note sono salvate in Markdown_G, con estensione `.md` e tipo MIME `text/markdown`; in caricamento sono accettate anche le estensioni `.txt`. La scelta del testo semplice attua il requisito di vincolo_{GG} **R-4-V-Ob**, che prescrive la memorizzazione delle note salvate localmente come file di testo. Il file contiene il solo testo della nota, senza intestazioni o marcatori aggiuntivi: resta quindi leggibile e modificabile con qualsiasi altro strumento, e non introduce alcun vincolo_{GG} verso il prodotto.

Metadati della nota I metadati che l'applicazione associa alla nota (identificativo, titolo, istante di creazione e istante di ultima modifica) non sono scritti nel file, che resta un documento Markdown_G puro. Il titolo è veicolato dal nome del file: in salvataggio il nome proposto deriva dal titolo della nota, in caricamento il titolo è ricavato dal nome del file privato dell'estensione. La derivazione avviene in entrambi i rami di caricamento, quello nativo e quello di ripiego, con un titolo convenzionale riservato al solo caso in cui il file scelto non esponga un nome utilizzabile: la parità fra i due rami è necessaria, poiché quello di ripiego è il percorso primario su Firefox, incluso fra i browser prescritti dal requisito_G di vincolo_{GG} **R-1-V-Ob**.

Sul requisito_G **R-97-F-De** (UC84), che chiede di mostrare informazioni e metadati di una nota selezionata, i casi da distinguere sono due, e non hanno a disposizione gli stessi dati.

- **Nota creata nel prodotto**

Identificativo, titolo e istanti di creazione e di ultima modifica sono assegnati alla creazione e affidati alla persistenza locale descritta sopra, che li scrive immediatamente: sopravvivono alla chiusura della sessione e sono disponibili alla riapertura, alle condizioni proprie di quel supporto (stessa origine, stesso profilo del browser). Per queste note i dati richiesti dal caso d'uso_G sono quindi già posseduti dall'applicazione, senza che nulla debba essere ricavato dal file system.

- **Nota importata da disco**

Il file contiene il solo testo, quindi i metadati andrebbero ricostruiti da ciò che il browser espone sull'oggetto **File**. Il titolo lo è, come appena descritto, perché deriva dal nome del file; gli istanti no: all'importazione entrambi sono posti all'istante di apertura, che descrive quando la nota è entrata nell'applicazione e non quando il file è stato scritto o modificato. L'attributo `lastModified`, che il browser espone su quell'oggetto in entrambi i rami di caricamento, non è letto.

In nessuno dei due casi però il requisito_G è soddisfatto: chiede la *visualizzazione* dei metadati, e il prodotto non espone alcun pannello che li mostri, per cui i dati che possiede restano interni all'applicazione. I fronti scoperti sono due: in importazione l'istante di ultima modifica che il browser espone sull'oggetto **File** non viene letto, e nell'interfaccia non esiste alcuna vista che presenti i metadati della nota. Trattandosi di un requisito_G desiderabile, la scopertura è una decisione consapevole del gruppo e non una dimenticanza, ed è tracciata come tale nel capitolo 5.

L'istante di creazione fa eccezione, perché è il solo dato che non dipende da una scelta realizzativa: per le note create nel prodotto è conservato, per quelle importate da disco resta non ricostruibile anche leggendo l'istante di ultima modifica, dato che le interfacce del browser non lo espongono. Andrebbe quindi mostrato come non disponibile, non con un valore convenzionale.

Motivazione dell'esclusione della base di dati La persistenza server-side è prevista dal capitolato_{GG} come estensione facoltativa. I requisiti_{GG} funzionali che ne derivano sono classificati come opzionali nell'Analisi dei Requisiti_{GG} e riguardano quattro capacità distinte:

- **operazioni sull'archivio remoto:** R-83-F-Op (creazione di una nota nella base di dati, UC74.2), R-86-F-Op (caricamento, UC76.2), R-89-F-Op (salvataggio, UC78.2), R-92-F-Op (rimozione, UC80.2), R-95-F-Op (rinomina, UC82.2). Sono il nucleo di ciò che l'esclusione comporta: le quattro operazioni di base sulle note, più la rinomina;
- **ricerca e filtro:** R-98-F-Op (ricerca per titolo), R-99-F-Op (ricerca full-text), R-100-F-Op e R-101-F-Op (filtro per data di creazione e di ultima modifica);
- **autenticazione:** R-102-F-Op (accesso con credenziali), R-103-F-Op (gestione degli errori di autenticazione), R-108-F-Op (disconnessione);
- **gestione dell'account:** R-104-F-Op (registrazione), R-105-F-Op (unicità dello username), R-106-F-Op (conferma della password), R-107-F-Op (rimozione dell'account e dei dati associati).

Sono opzionali anche i requisiti_{GG} di vincolo_{GG} corrispondenti: **R-5-V-Op** (memorizzazione delle note in una base di dati raggiungibile dal sistema_{GG}), **R-6-V-Op** (componente server raggiungibile tramite chiamate HTTP) e **R-7-V-Op** (linguaggio della componente server-side). Si segnala che dei tre soltanto R-7-V-Op è già soddisfatto dall'architettura attuale, poiché la componente server esiste per le funzioni assistite da LLM_{GG} indipendentemente dalla persistenza; R-5-V-Op e R-6-V-Op restano scoperti insieme alla funzionalità che li condiziona.

Il gruppo ha scelto di concentrare le risorse della presente iterazione sul completamento dei requisiti_{GG} obbligatori e desiderabili, in particolare sulle funzionalità assistite da LLM_{GG}, che costituiscono il nucleo del prodotto richiesto dalla proponente_{GG}. Lo stato di copertura di ciascun requisito_G è riportato nella sezione 5.

Sul margine di estensione conviene essere precisi, per non attribuire all'architettura una predisposizione che non ha. Nel perimetro attuale la persistenza delle note sta tutta nel frontend_{GG}: le note non raggiungono mai il backend, e nel suo dominio_{GG} non esiste una porta per le note in attesa di essere implementata. Introdurre la persistenza su base di dati comporterebbe quindi lavoro nuovo e non marginale: entità di dominio_{GG} per la nota e per l'utente, le porte corrispondenti, gli endpoint che le espongono, un adattatore verso il sistema_{GG} di gestione della base di dati e l'intero blocco di autenticazione richiesto da R-102-F-Op ... R-108-F-Op. Ciò che l'architettura esagonale garantisce è più circoscritto, ma sufficiente: quel lavoro si aggiungerebbe come nuova porta e nuovo adattatore, senza toccare il dominio_{GG} delle funzioni assistite da LLM_{GG} già realizzato.

2.5 Tecnologie di test e analisi

Le attività di verifica_{GG} previste dalle Norme di Progetto_{GG} si articolano in analisi statica_G, condotta sul codice sorgente senza porlo in esecuzione, e analisi dinamica_G, condotta eseguendo il codice su casi di prova predisposti.

2.5.1 Analisi statica

L'analisi statica_G intercetta gli errori prima che il codice venga eseguito, spostandone il rilevamento dalla fase di prova a quella di scrittura.

Tabella 3: Strumenti di analisi statica

Tecnologia	Versione	Descrizione e ruolo nel progetto
Compilatore TypeScript	6.0.3	Verifica _{GG} della coerenza dei tipi sull'intero codice del frontend _{GG} . La compilazione è eseguita in modo esplicito come primo passo del comando di costruzione (<code>tsc -b && vite build</code>): un errore di tipo interrompe la costruzione e impedisce la produzione del pacchetto. La configurazione attiva inoltre la segnalazione di variabili e parametri dichiarati e non utilizzati, dei casi non terminati nei costrutti di selezione multipla e delle costruzioni non rimovibili per sola cancellazione dei tipi.
ESLint	10.5.0	Analisi statica _G del codice del frontend _{GG} rispetto a un insieme di regole configurate. Individua costrutti sintatticamente validi ma problematici, che il compilatore non segnala.
typescript-eslint	8.61.0	Estensione di ESLint alla sintassi TypeScript e alle regole specifiche del linguaggio.
eslint-plugin-react-hooks	7.1.1	Verifica _{GG} del rispetto delle regole di impiego degli <i>hook</i> di React, in particolare della completezza degli elenchi di dipendenze. Si tratta di una classe di errori che non produce alcun messaggio in esecuzione ma si manifesta come stato dell'interfaccia non aggiornato, di difficile diagnosi.
eslint-plugin-react-refresh	0.5.2	Verifica _{GG} che i moduli rispettino le condizioni necessarie all'aggiornamento a caldo dei componenti durante lo sviluppo _{GG} .
Ruff	0.16.1	Analisi statica _G e formattazione automatica del codice del backend. Riunisce in un solo strumento le regole di più analizzatori dell'ecosistema Python: gli insiemi selezionati sono quelli di <code>pyflakes</code> , <code>pycodestyle</code> (errori e avvisi), <code>isort</code> , <code>pyupgrade</code> e <code>flake8-bugbear</code> , che segnalano importazioni e variabili non utilizzate, ordinamento delle importazioni, costrutti sintatticamente validi ma problematici e violazioni delle convenzioni di stile. Il formattatore integrato riproduce lo stile di <code>black</code> , indicato dalle Norme di Progetto _{GG} per la formattazione del codice Python: la conformità è quindi mantenuta senza introdurre un secondo strumento. La configurazione risiede in <code>ruff.toml</code> , accanto ai file di progetto del backend.
import-linter	2.13	Verifica _{GG} dei contratti di dipendenza del backend: quale package possa importare quale altro. Ispeziona il grafo delle importazioni senza eseguire il codice e segnala come errore ogni importazione che violi un contratto. I quattro contratti dichiarati in <code>.importlinter</code> fissano l'architettura descritta nella sezione 3.4.1: il nucleo del dominio _{GG} non importa il framework _{GG} web, il client HTTP, il client del servizio di estrazione, la libreria di configurazione né i package degli adattatori; non importa il modulo di configurazione del prodotto; i tre livelli procedono dall'esterno verso il centro; i due adattatori dell'infrastruttura non si importano fra loro. Rende verificabile _G un'architettura che altrimenti resterebbe una convenzione affidata alla disciplina di chi scrive.

L'analisi statica_G è quindi applicata a entrambi i livelli del prodotto, come richiesto dalle Norme di Progetto_{GG} e, per il loro tramite, dal requisito_G di qualità **R-10-Q-Ob**. La ripartizione fra i due livelli non è però simmetrica: sul frontend_{GG} si verificano_G i tipi, con il compilatore

TypeScript, e le regole di scrittura, con ESLint; sul backend si verificano_G le regole di scrittura, con Ruff, e i contratti di dipendenza, con import-linter. Ogni livello ha quindi un controllo che all'altro manca: i tipi sono verificati_G solo sul frontend_{GG}, perché il backend non adotta un analizzatore di tipi, mentre i confini architetturali sono verificati_G solo sul backend, dove i package sono organizzati a esagono e la direzione delle dipendenze è ciò che serve proteggere.

Non è una lacuna della pianificazione, ma una conseguenza di come sono fatti i due livelli: il backend è tipizzato ma piccolo, e al suo interno la cosa più facile da violare senza accorgersene non è la coerenza di una firma, è la direzione di un'importazione.

La validazione_G degli schemi Pydantic non rientra in questa categoria e non la sostituisce: Pydantic controlla i *dati* che attraversano i confini dell'applicazione mentre il programma gira, l'analisi statica_G controlla il *codice* senza eseguirlo. I difetti che intercettano non si sovrappongono (un corpo di richiesta malformato da una parte, un'incoerenza fra firme di funzione dall'altra): sono controlli complementari.

2.5.2 Analisi dinamica

L'analisi dinamica_G è organizzata su test_{GG} di unità e di integrazione_{GG} dei due livelli, eseguiti separatamente dalle rispettive catene di strumenti.

Tabella 4: Strumenti di analisi dinamica

Tecnologia	Versione	Descrizione e ruolo nel progetto
Vitest	4.1.8	Esecutore dei test _{GG} di unità e di integrazione _{GG} del frontend _{GG} . Riutilizza integralmente la configurazione di Vite (risoluzione dei moduli, alias dei percorsi, trasformazione di TypeScript e JSX), evitando la manutenzione di una seconda catena di strumenti allineata alla prima.
@vitest/ coverage-v8	4.1.8	Misurazione della copertura del codice _G raggiunta dai test _{GG} del frontend _{GG} . Si appoggia allo strumento di copertura nativo del motore V8, senza richiedere una strumentazione del codice sorgente. Le soglie minime sono dichiarate nella configurazione di Vitest, così che il superamento sia verificato _G dall'esecutore stesso e non affidato alla lettura del rapporto; l'insieme dei file misurati vi è dichiarato esplicitamente, così che un sorgente privo di test _G pesi sul denominatore anziché sparire dal calcolo. È la controparte di pytest-cov sul versante frontend _{GG} : senza di essa il Piano di Qualifica _G non potrebbe riportare una metrica _G di copertura su questo livello, come richiesto da R-2-Q-Ob .
jsdom	29.1.1	Realizzazione del DOM in ambiente Node.js, che consente di eseguire i test _{GG} dei componenti senza avviare un browser.
Testing Library (React)	16.3.2	Interrogazione dei componenti resi attraverso ruoli, etichette e testo visibile, così come vi accederebbe una persona che utilizza il prodotto, anziché attraverso la struttura interna del marcatore. I test _{GG} restano quindi validi a fronte di riorganizzazioni interne dei componenti e verificano indirettamente l'accessibilità dell'interfaccia.
jest-dom user-event	6.9.1 14.6.1	Asserzioni specifiche per gli elementi del DOM e simulazione delle interazioni dell'utente (digitazione, selezione, attivazione di comandi) con la sequenza di eventi effettivamente prodotta da un browser.

Tecnologia	Versione	Descrizione e ruolo nel progetto
pytest	9.0.3	Esecutore dei test _{GG} del backend. Il sistema _G delle <i>fixture</i> consente di comporre e riutilizzare le precondizioni dei test _{GG} in modo esplicito.
pytest-asyncio	1.4.0	Esecuzione dei test _{GG} su funzioni asincrone, condizione necessaria data la natura asincrona dell'intero backend. La modalità automatica configurata nel progetto rende superflua l'annotazione di ogni singolo test _{GG} .
pytest-cov	7.1.0	Misurazione della copertura del codice _G raggiunta dai test _{GG} del backend.
respx	0.23.1	Interposizione sulle richieste HTTP effettuate tramite httpx. Consente di verificare _G il comportamento del client verso il provider dei modelli (flusso corretto, risposta malformata, errore di connessione, scadenza del tempo massimo) senza contattare alcun servizio esterno, rendendo i test _{GG} deterministici e indipendenti dalla rete.

Alternative considerate

- **Jest al posto di Vitest**

Jest è lo standard storico dei test_{GG} in ambiente JavaScript, con la comunità più ampia e la maggiore disponibilità di documentazione. Richiederebbe però una configurazione separata e mantenuta in parallelo a quella di Vite, in particolare per la trasformazione di TypeScript e per i moduli ES, che Jest non supporta nativamente. Il disallineamento fra le due configurazioni è una fonte di errori ricorrente, e non è compensato da alcun vantaggio funzionale: l'interfaccia di programmazione di Vitest è deliberatamente compatibile con quella di Jest, e i test_{GG} risultano pressoché identici.

- **unittest al posto di pytest**

Il modulo `unittest` è incluso nella libreria standard di Python e non aggiunge dipendenze. Impone però una struttura a classi che rende i test_{GG} più verbosi e, soprattutto, non dispone né di un sistema_G di *fixture* componibili paragonabile a quello di pytest né dell'ecosistema di estensioni sul quale il progetto si appoggia per l'esecuzione asincrona, la misurazione della copertura del codice_G e l'interposizione sulle richieste HTTP.

2.6 Infrastruttura

Le tecnologie di questa sezione non concorrono al comportamento del prodotto in esecuzione, ma governano il modo in cui esso viene costruito, avviato, verificato_G e mantenuto coerente fra i due livelli.

Organizzazione della repository_G Le tecnologie che seguono dipendono tutte da una decisione organizzativa presa prima: il gruppo ha adottato un'unica repository_{GG} (*mono-repo*) per i due livelli del prodotto. Il codice del frontend_{GG} risiede in `/web` e quello del backend in `/api`, ciascuno con il proprio gestore di pacchetti, il proprio file di dipendenze e il proprio file di lock. Restano quindi due progetti distinti, con costruzioni e dipendenze proprie: condividono la repository_{GG} e la cronologia delle modifiche, non divengono un unico artefatto_{GG}.

La scelta ha due conseguenze dirette sulle tecnologie descritte di seguito. La prima è che una variazione del contratto dell'API_{GG} e il corrispondente adeguamento del client possono essere integrati_G con una sola pull request_{GG}, che li sottopone alla medesima verifica_{GG} automatica:

non esiste un intervallo in cui i due livelli risultino disallineati. La seconda è che la configurazione di verifica_{GG} è definita una volta sola, in un solo insieme di flussi di lavoro.

Tabella 5: Tecnologie di infrastruttura

Tecnologia	Versione	Descrizione e ruolo nel progetto
Docker Docker Compose	29.4.3 5.1.4	Ambiente di sviluppo _{GG} riproducibile, non una configurazione di esercizio, per le ragioni esposte nella sezione 3.2.1. Il file <code>docker-compose.yml</code> descrive i due servizi <code>web</code> e <code>api</code> , la rete che li collega, le porte esposte verso la macchina ospite e le variabili d'ambiente, e li avvia con un solo comando. Le immagini di base sono fissate dai rispettivi <code>Dockerfile</code> : <code>node:22-alpine</code> per il frontend _{GG} e <code>python:3.12-slim</code> per il backend. La linea di Node.js non è scelta soltanto come limite inferiore: è fissata al di sopra e al di sotto dal campo <code>engines</code> di <code>package.json</code> (<code>>=22 <23</code>) e dal file <code>.nvmrc</code> , che l'ambiente locale e l'integrazione _{GG} continua leggono entrambi. Il limite superiore ha una ragione misurata e documentata nel repository _{GG} : su linee successive parte della suite di test _{GG} del frontend _{GG} non è eseguibile.
GitHub Actions	—	Infrastruttura di <i>Continuous Integration</i> _G . Su ogni pull request _{GG} , e su ogni integrazione _{GG} nei rami principali, esegue in processi indipendenti l'analisi statica _G dei due livelli, la verifica _{GG} dei contratti di dipendenza del backend, il controllo che il contratto dell'API _{GG} e i tipi da esso derivati siano allineati al codice, le due suite di test _{GG} con misurazione della copertura del codice _G e la costruzione del pacchetto di produzione del frontend _{GG} . È qui che gli strumenti delle sezioni 2.5.1 e 2.5.2 vengono applicati davvero: senza, la loro esecuzione dipenderebbe dalla buona volontà del singolo. Trattandosi di un servizio della piattaforma e non di una dipendenza installata, non è associato a una versione dichiarata nei file di progetto; le versioni fissate sono quelle delle singole azioni richiamate dai flussi di lavoro.
pnpm	11.18.0	Gestore dei pacchetti del frontend _{GG} . Memorizza ogni versione di ogni pacchetto una sola volta e la collega per riferimento, riducendo occupazione di spazio e tempi di installazione. Il file <code>pnpm-lock.yaml</code> fissa le versioni risolte dell'intero albero delle dipendenze; la versione dello strumento stesso è vincolata dal campo <code>packageManager</code> di <code>package.json</code> , che l'integrazione _{GG} continua legge per approvvigionarsi, così che questa e lo sviluppo _{GG} locale adoperino il medesimo risolutore. Il vincolo _{GG} non si estende all'immagine di sviluppo _{GG} del frontend _{GG} , che installa lo strumento senza indicarne la versione: chi lavora nel container può quindi risolvere una versione diversa da quella dichiarata.
uv	0.11.16	Gestore dei pacchetti e dell'ambiente virtuale del backend. Risolve le dipendenze dichiarate in <code>pyproject.toml</code> e ne fissa le versioni in <code>uv.lock</code> . Nell'ambiente composto con Docker è la sincronizzazione eseguita all'avvio del container, non il passo di costruzione dell'immagine, a rendere identico l'ambiente fra macchine diverse: il volume di sviluppo _{GG} copre infatti quanto installato in costruzione (sezione 3.2.1). Il vincolo _{GG} esplicito al file di lock è dichiarato nella costruzione dell'immagine e nell'integrazione _{GG} continua, non nel comando di avvio.

Tecnologia	Versione	Descrizione e ruolo nel progetto
openapi-typescript	7.13.0	Generazione dei tipi TypeScript del client a partire dallo schema OpenAPI prodotto da FastAPI. Il frontend _{GG} non dichiara autonomamente la forma dei dati scambiati con il backend: la deriva dal contratto pubblicato dal backend stesso. Una modifica di uno schema Pydantic si propaga così ai tipi del client e, se incompatibile con l'uso che il frontend _{GG} ne fa, si manifesta come errore di compilazione anziché come guasto in esecuzione. Il file generato è escluso dall'analisi di ESLint, non essendo codice scritto a mano.

Nota sulle versioni degli strumenti di sviluppo Le versioni di Docker, Docker Compose e uv hanno uno statuto diverso da tutte le altre del documento. Quelle delle librerie sono fissate dai file di lock e risultano identiche su ogni macchina; queste tre no, perché nessun file del progetto le vincola: Docker e Compose sono presupposti dall'ambiente anziché dichiarati, e uv è installato dall'immagine del backend senza indicazione di versione e risolto alla versione corrente dall'integrazione_{GG} continua. I valori riportati sono quindi quelli in uso sull'ambiente di sviluppo_G del gruppo alla data del documento, e non un vincolo_{GG} verificabile_G: su un'altra macchina, o in un altro momento, l'installazione può risolvere versioni diverse. Per Docker il numero è quello di CLI e motore riportato da `docker --version`, che non coincide con la versione di Docker Desktop, la quale segue una numerazione propria. Fa eccezione pnpm, che il repository_{GG} vincola: il valore riportato è quello dichiarato in `package.json` e non quello rilevato dall'ambiente.

Verifica automatica delle pull request_G I controlli eseguiti dall'integrazione_{GG} continua sono la condizione che ogni modifica deve soddisfare prima di essere integrata_G nel ramo principale. Sono organizzati in tre processi indipendenti, ciascuno con i propri passi:

- **frontend_G**: installazione vincolata al file di lock, ESLint, rigenerazione dei tipi dallo schema OpenAPI e confronto con il file versionato, compilazione dei tipi con `tsc`, esecuzione di Vitest con misurazione della copertura del codice_G e verifica_{GG} delle soglie, costruzione del pacchetto di produzione con Vite;
- **backend**: installazione vincolata a `uv.lock`, Ruff nella modalità di controllo delle regole, riesportazione dello schema OpenAPI e confronto con il file versionato, esecuzione di pytest con `pytest-cov` e verifica_{GG} della soglia;
- **contratti di dipendenza**: installazione vincolata a `uv.lock` e controllo dei contratti dichiarati per l'architettura del backend, descritti nella sezione 3.4.1.

La soglia di copertura verificata dai primi due processi è quella fissata dal Piano di Qualifica_G per il valore accettabile della metrica_G corrispondente; entrambi conservano il rapporto prodotto come artefatto_{GG} del processo, così che il dato sia consultabile a posteriori e non solo nel registro di esecuzione.

L'esito negativo di un qualunque passo impedisce l'integrazione_{GG}. Questi controlli soddisfano il requisito_G **R-4-Q-Ob**, che subordina l'integrazione_{GG} nel ramo principale al superamento dei test_{GG} automatici oltre che alla revisione tra pari. Il controllo sul contratto dell'API_{GG} è doppio e simmetrico: il processo del backend riesporta lo schema dal codice e pretende che coincida con `openapi.json` versionato, quello del frontend_{GG} rigenera i tipi da quello schema e pretende che coincidano con il file versionato. Se fallisce il primo confronto, il contratto pubblicato non corrisponde più al codice che lo produce; se fallisce il secondo, lo schema è cambiato

senza che il client sia stato aggiornato. In entrambi i casi la verifica_{GG} si ferma prima che il disallineamento raggiunga il ramo principale.

Alternative considerate

- **Configurazione manuale dell'ambiente al posto di Docker**

Richiederebbe a ciascun membro del gruppo di installare e allineare manualmente le versioni di Node.js, di Python e delle rispettive dipendenze di sistema_{GG} su tre famiglie di sistemi operativi differenti. Le configurazioni divergerebbero comunque, e un comportamento osservato su una macchina non sarebbe riproducibile sulle altre: diagnosticare un guasto costerebbe molto di più e i test_{GG} perderebbero valore di prova.

- **Podman al posto di Docker**

Podman è compatibile con i formati e i comandi di Docker e presenta un vantaggio di sicurezza_G non trascurabile: non richiede un processo demone privilegiato in esecuzione permanente e consente di eseguire i container senza privilegi amministrativi. È stato scartato per ragioni di diffusione, ampiezza della documentazione e familiarità pregressa dei membri del gruppo, elementi che riducono il tempo speso in attività non produttive.

- **Poly-repo al posto del mono-repo**

La separazione di frontend_{GG} e backend in repository_{GG} distinte è appropriata quando i due componenti sono sviluppati da gruppi diversi e rilasciati in modo indipendente. Non è il caso del progetto: le due parti sono sviluppate dallo stesso gruppo e rilasciate congiuntamente. La separazione imporrebbe due pull request $_{GG}$ coordinate a ogni variazione del contratto dell' API_{GG} , con una finestra temporale in cui le due repository_{GG} risultano disallineate, oltre alla duplicazione della configurazione di verifica_{GG} .

- **Tipi TypeScript scritti a mano al posto della generazione da OpenAPI**

È l'alternativa che non aggiunge alcuno strumento, ma introduce una duplicazione del contratto dell' API_{GG} in due punti del sistema_{GG} mantenuti separatamente. Le due definizioni divergono appena il backend cambia senza che il frontend_{GG} venga aggiornato di conseguenza; e siccome i tipi TypeScript spariscono in compilazione, la divergenza non produce alcun errore e si manifesta come guasto silenzioso in esecuzione. Generare i tipi dallo schema toglie di mezzo la duplicazione.

3 Architettura

3.1 Panoramica architetturale

La presente sezione fornisce la visione d'insieme del sistema_{GG}: le unità che lo compongono, i confini che le separano e lo stile architettuale adottato all'interno di ciascuna. Ne fissa inoltre il vocabolario, che le sezioni successive impiegano senza introdurre sinonimi. Il dettaglio di ciascuna parte è rimandato a quelle sezioni: qui si stabilisce soltanto che cosa esiste, come è delimitato e secondo quale criterio è organizzato al proprio interno.

3.1.1 Visione d'insieme

Il sistema_{GG} è composto da due unità di deployment indipendenti:

- un'**applicazione web a pagina singola** (*Single Page Application*), eseguita interamente nel browser dell'utente;
- un **servizio applicativo** che espone un'API_{GG} HTTP.

Le due unità risiedono in altrettante cartelle della repository_{GG} (**web/** e **api/**), ciascuna con il proprio gestore di pacchetti, le proprie dipendenze e la propria catena di costruzione: restano quindi due progetti distinti, costruiti ed eseguiti separatamente, e non due parti di un unico artefatto_{GG}. Il modo in cui vengono poste in esecuzione è trattato nella sezione dedicata all'architettura di deployment.

La comunicazione fra le due unità avviene su HTTP secondo uno stile REST_G, ed è asimmetrica: l'applicazione web è l'unico soggetto che avvia le richieste, il servizio applicativo il solo che risponde. Le risposte delle operazioni assistite dal modello linguistico sono restituite in streaming: il testo raggiunge il browser a frammenti, man mano che il modello lo produce, e compare progressivamente nell'interfaccia anziché tutto insieme al termine dell'elaborazione. Il servizio non trattiene il testo per consegnarlo completo, ma lo inoltra appena disponibile; il formato con cui i frammenti viaggiano sulla connessione appartiene alla sezione sul flusso dei dati.

Il servizio applicativo è a sua volta client di due servizi esterni, entrambi contattati via HTTP e nessuno dei due realizzato dal gruppo:

- un **gateway LiteLLM**, che espone un'interfaccia compatibile con quella di OpenAI e per il cui tramite avviene l'inferenza del modello linguistico;
- **Tavily**, che restituisce il contenuto testuale di una pagina web il cui indirizzo è indicato dall'utente.

Il perimetro del sistema_{GG} comprende quindi due unità realizzate dal gruppo e due dipendenze esterne raggiunte dalla sola unità server: l'applicazione web non contatta in alcun caso i due fornitori.

Non esiste persistenza lato server. Il servizio applicativo non possiede dati: non dispone di una base di dati, non conserva stato applicativo fra una richiesta e la successiva e il suo dominio_{GG} non dichiara alcuna porta per le note o per gli utenti. I dati dell'utente restano nel browser. Il meccanismo con cui vi permangono è descritto altrove; qui rileva la sola conseguenza architettuale: il confine del sistema_{GG} non racchiude alcun archivio, e le due unità non condividono alcuno stato persistente.

3.1.2 Stile architetturale del servizio applicativo

Il servizio applicativo adotta l'**architettura esagonale** (*Ports & Adapters*). Il criterio che la caratterizza è la direzione delle dipendenze: al centro stanno le regole del prodotto, attorno gli adattatori che le collegano al mondo esterno, e la dipendenza punta sempre verso il centro, mai in senso contrario.

Il nucleo Il package `app/core/` contiene le regole del prodotto (i casi d'uso_G, i valori del dominio_{GG} e le porte) e non importa nulla dagli altri package. Non conosce il framework_{GG} web, non conosce il client HTTP, non conosce i fornitori esterni né il modulo che legge la configurazione. Un caso d'uso è una funzione che riceve i dati dell'operazione e, come ultimo parametro, la porta di cui si serve: non sa chi la implementi e non ha modo di scoprirlo.

Le porte Le porte dichiarate sono due, entrambe in `app/core/ports/`. `LLMClient` dichiara la produzione incrementale_G di testo da parte di un modello linguistico; `ContentExtractor` dichiara l'estrazione del contenuto testuale di un indirizzo web. Sono interfacce astratte: descrivono ciò di cui il dominio_{GG} ha bisogno, non il modo in cui un fornitore lo realizza. Accanto a ciascuna è dichiarato il tipo di errore che essa può sollevare, così che chi la consuma possa trattarne il fallimento senza conoscere l'implementazione che lo produce.

I due lati dell'esagono Gli adattatori si dispongono su due lati opposti. In `app/api/` vive ciò che serve il chiamante esterno, tanto la traduzione della sua richiesta in una chiamata al dominio_{GG} quanto la formattazione dell'esito nella risposta che gli viene restituita: è il lato da cui l'applicazione viene invocata. In `app/infrastructure/` vive ciò che il dominio_{GG} chiama per parlare con il mondo: è il lato che l'applicazione invoca. Le due direzioni non si toccano direttamente: un adattatore del primo lato non istanzia un adattatore del secondo, ma dichiara la porta di cui ha bisogno e la riceve dall'esterno. Il modulo che stabilisce quale adattatore soddisfa quale porta è uno solo, il *composition root*, ed è l'unico punto dell'applicazione in cui i due adattatori concreti sono nominati fuori dai moduli che li definiscono. Sostituire un fornitore esterno significa perciò scrivere un nuovo adattatore e cambiare una riga in quel modulo, non intervenire sul dominio_{GG}.

Il vincolo_G non è affidato alla disciplina dei redattori È il punto qualificante di questa architettura, e va detto esplicitamente: la direzione delle dipendenze non è una convenzione che il gruppo si impegna a rispettare, ma un vincolo_{GG} dichiarato e verificato_G automaticamente. I contratti sono scritti in `api/.importlinter` e controllati staticamente dallo strumento `import-linter`, che ispeziona il grafo delle importazioni senza eseguire il codice. Sono quattro:

- il nucleo non importa il framework_{GG} web, il client HTTP, il client del servizio di estrazione, la libreria che legge la configurazione, né i package dei due lati di adattatori;
- il nucleo non importa il modulo del prodotto che dichiara lo schema della configurazione. Il contratto è distinto dal precedente e non ne è un caso particolare: quello vieta una libreria di terzi, questo un modulo scritto dal gruppo, e nessuno dei due implica l'altro;
- i tre livelli sono ordinati dall'esterno verso il centro: ciascuno può importare quelli che gli stanno sotto, mai quelli che gli stanno sopra;
- i due adattatori dell'infrastruttura non si importano fra loro.

Il controllo è un passo dell'integrazione_{GG} continua: un'importazione che invertisse la dipendenza farebbe fallire la verifica_{GG} automatica prima ancora di arrivare alla revisione tra pari.

Il beneficio ottenuto Ciò che l'esagono restituisce, e che è verificabile_G nel codice, è la provabilità del dominio_{GG} in isolamento: un caso d'uso si esercita senza montare HTTP e senza rete, passandogli al posto della porta un doppio che ne rispetta il contratto. I test_{GG} dei casi d'uso_G del prodotto sono scritti esattamente così, e non istanziano alcun adattatore. La proprietà non è un corollario teorico dello stile adottato: discende dal fatto che il tipo del parametro sia la porta e mai l'adattatore, condizione che i contratti di dipendenza rendono impossibile violare inavvertitamente.

Lo stile qui dichiarato (un servizio applicativo unico, organizzato a esagono) è stato adottato in alternativa alla scomposizione in microservizi e al monolite a strati tradizionale. L'argomentazione del confronto, con i costi e i benefici di ciascuna alternativa, è riportata nelle sezioni 3.2.2 e 3.2.3.

3.1.3 Stile architetturale dell'applicazione web

L'applicazione web è organizzata a componenti, con stato condiviso esterno all'albero e flusso dei dati unidirezionale.

I componenti React compongono l'interfaccia per annidamento: ciascuno riceve dal genitore i dati che deve presentare e ne restituisce la rappresentazione. Lo stato realmente condiviso non risiede però nell'albero dei componenti: vive fuori di esso, in due store distinti, che i componenti interrogano direttamente senza doverlo trasmettere lungo la gerarchia. Il flusso resta unidirezionale: un'interazione dell'utente invoca un'azione dello store, l'azione aggiorna lo stato, i componenti sottoscritti alla porzione modificata si ridisegnano. Non esiste un percorso inverso in cui un componente modifichi la vista di un altro.

La regola seguita nel collocare lo stato è graduale: stato locale al componente per difetto, sollevato al genitore comune quando due componenti vicini devono concordare, promosso a store soltanto quando componenti lontani nell'albero devono dividerlo. Lo store non è quindi il deposito di tutto lo stato dell'applicazione: le condizioni puramente locali di un componente (l'apertura di un pannello, il testo in corso di digitazione in un campo, la voce dell'elenco in fase di rinomina) restano nel componente che le possiede, e non se ne ha traccia altrove.

I componenti si sottoscrivono a singole porzioni di stato, non allo stato intero: ciascuno store espone un selettore per ogni porzione osservabile, e il componente che ne dichiara uno viene ridisegnato solo quando quella porzione cambia. È la proprietà che rende sostenibile lo streaming: mentre il testo arriva a frammenti e la porzione che lo raccoglie cambia molte volte al secondo, si ridisegna la sola parte di interfaccia che presenta l'esito in arrivo, mentre l'editor, sottoscritto a una porzione distinta (il testo della nota), non viene toccato. Una sottoscrizione allo stato nel suo complesso, o a un oggetto che ne aggrega più porzioni, riporterebbe il ridisegno sull'intera area di lavoro a ogni frammento ricevuto.

I due store si dividono le responsabilità in questo modo: `useEditorStore` custodisce lo stato della sessione di scrittura, cioè il testo corrente, la selezione, l'esito in arrivo dal servizio applicativo, la condizione di errore e di avanzamento e la modalità di visualizzazione; `useNotesStore` custodisce la raccolta delle note e l'indicazione di quale sia quella corrente. La loro composizione interna appartiene alla sezione sull'architettura logica dell'applicazione web.

3.1.4 Vocabolario

I termini fissati nella tabella 6 sono impiegati con questo significato in tutto il capitolo. Le sezioni successive vi si attengono e non ne adottano sinonimi.

Tabella 6: Vocabolario architetturale

Termine	Definizione
Porta	Interfaccia astratta dichiarata dal nucleo, che descrive ciò di cui il dominio _{GG} ha bisogno dall'esterno senza indicare come venga realizzato.
Adattatore primario	Modulo che sta sul lato dell'esagono da cui l'applicazione è invocata: traduce la richiesta proveniente dall'esterno in una chiamata al dominio _{GG} , oppure ne formatta l'esito nella risposta restituita al chiamante; risiede in <code>app/api/</code> .
Adattatore secondario	Modulo che realizza una porta traducendo la chiamata del dominio _{GG} nel protocollo di un servizio esterno; risiede in <code>app/infrastructure/</code> .
Nucleo (dominio_G)	Insieme delle regole del prodotto, contenuto in <code>app/core/</code> , che non importa nulla dagli altri package del servizio applicativo.
Caso d'uso	Funzione del nucleo che realizza una singola operazione del prodotto, ricevendo i dati dell'operazione e, come ultimo parametro, la porta di cui si serve.
Composition root	Unico modulo che stabilisce quale adattatore soddisfa quale porta, e unico punto in cui gli adattatori concreti sono nominati fuori dai moduli che li definiscono.
Store	Contenitore di stato condiviso dell'applicazione web, esterno all'albero dei componenti, che espone lo stato in lettura e le azioni che lo modificano.
Selettore	Funzione che estrae da uno store una singola porzione di stato e alla quale un componente si sottoscrive; determina quando quel componente viene ridisegnato.

3.2 Architettura di deployment

La sezione 3.1 ha stabilito che il sistema_{GG} è composto da due unità di deployment indipendenti. La presente sezione descrive come quelle due unità sono messe in esercizio: quali processi esistono, in quale forma sono isolati, attraverso quale indirizzo si raggiungono, da dove ricevono la propria configurazione e che cosa accade loro all'avvio e allo spegnimento.

Il perimetro è deliberatamente ristretto a ciò che si osserva dall'esterno dei due processi. L'organizzazione interna dell'applicazione web e del servizio applicativo appartiene alle sezioni 3.3 e 3.4; il contenuto degli scambi fra le due unità è trattato nella sezione 4; le ragioni per cui sono state adottate le tecnologie qui nominate sono esposte nella sezione 2, e i controlli automatici applicati a ogni modifica nella sezione 2.6. Qui esse compaiono come fatti.

Le sottosezioni 3.2.2 e 3.2.3 raccolgono infine l'argomentazione sulle alternative architetturali complessive, richiamata per riferimento dalla sezione 3.1.2.

3.2.1 Unità di esecuzione e configurazione

I due container Il file `docker-compose.yml` descrive due servizi, `web` e `api`, uno per ciascuna delle due unità. Nessuno dei due riferisce un'immagine pubblicata in un registro: entrambi sono costruiti dal contesto della rispettiva cartella della repository_{GG}, così che l'immagine in esecuzione corrisponda sempre al codice della copia di lavoro. Ciascun servizio pubblica una porta verso la macchina ospite: la 5173 per l'applicazione web, la 8000 per il servizio applicativo. Sono i due soli punti di ingresso del sistema_{GG}, e li usa entrambi il browser dell'utente. Il servizio `web` dichiara inoltre una dipendenza di avvio dal servizio `api`: l'ordine riguarda l'avvio dei container, non la disponibilità dell'API_{GG}, che il primo non attende.

Le immagini di base Le due immagini sono fissate dai rispettivi `Dockerfile`: `node:22-alpine` per l'applicazione web e `python:3.12-slim` per il servizio applicativo. Entrambe installano il proprio gestore di pacchetti e risolvono le dipendenze in fase di costruzione a partire dal file di lock versionato. Le due linee non sono intercambiabili con altre: quella di Node.js è vincolata al di sopra e al di sotto, come argomentato nella sezione 2.6.

Va osservato, perché la lettura del `Dockerfile` da sola condurrebbe a una conclusione errata, che per il servizio applicativo l'ambiente costruito nell'immagine non è quello effettivamente adoperato: il volume che monta la copia di lavoro sul medesimo percorso lo copre, e il comando di avvio sincronizza l'ambiente a ogni accensione del container, materializzandolo nella cartella montata. La riproducibilità fra macchine diverse discende quindi da quella sincronizzazione all'avvio, non dal passo di costruzione, il cui esito viene sostituito.

La differenza fra i due momenti non è però soltanto di collocazione. La sincronizzazione eseguita durante la costruzione è dichiarata vincolata al file di lock, e rifiuta di procedere se questo non corrisponde alle dipendenze dichiarate; quella eseguita all'avvio del container non porta il medesimo vincolo_{GG}, e davanti a un disallineamento fra i due file aggiornerebbe il lock anziché arrestarsi. L'allineamento è perciò garantito a monte, dalla costruzione dell'immagine e dai controlli automatici descritti nella sezione 2.6, e non nell'istante dell'esecuzione. È una conseguenza coerente con la natura dell'ambiente: sincronizzare a ogni avvio ha senso in sviluppo_{GG}, dove i sorgenti e le dipendenze cambiano di continuo, e non ne avrebbe in esercizio.

La rete I due servizi sono collegati da una rete dedicata, di tipo *bridge*, dichiarata nello stesso file. La rete attribuisce a ciascun container un nome risolvibile dall'altro, rendendo possibile una chiamata da container a container senza passare per la macchina ospite. Va però osservato, per non attribuire alla rete un ruolo che oggi non ha, che il traffico fra l'applicazione web e il servizio applicativo non la attraversa: il codice dell'applicazione web è eseguito dal browser sulla macchina ospite, ed è il browser a contattare la porta pubblicata dal servizio applicativo. La rete resta quindi predisposta per comunicazioni fra container che il prodotto, allo stato attuale, non effettua.

Il punto di accoppiamento fra le due unità L'applicazione web non conosce staticamente l'indirizzo del servizio applicativo: lo legge da una variabile d'ambiente, fornita al processo del frontend_{GG} al momento dell'avvio e valorizzata dal file di composizione con l'indirizzo pubblicato sulla macchina ospite. In sua assenza ripiega sulla stringa vuota, che produce indirizzi relativi e quindi richieste alla medesima origine da cui l'applicazione è stata servita.

Che quell'indirizzo sia configurazione e non codice discende da una distinzione precisa: esso dipende da *dove* il sistema_{GG} è messo in esercizio, non da *che cosa* il sistema_{GG} fa. Le due unità dietro la medesima origine o su origini distinte, come avviene qui, richiedono valori diversi e nessuna modifica al codice; il ripiego sulla stringa vuota non è un predefinito arbitrario, ma la codifica del primo dei due casi.

La provenienza della configurazione Il servizio applicativo non legge variabili d'ambiente sparse nel codice: le raccoglie in un unico schema validato, che ne dichiara nomi, tipi ed eventuali valori predefiniti. Le voci sono cinque: l'indirizzo del gateway dei modelli linguistici, l'identificativo del modello da impiegare, la chiave di accesso al gateway, la chiave del servizio di estrazione e l'elenco delle origini ammesse dalla politica CORS. Le prime due e l'ultima hanno un valore predefinito nello schema; le due chiavi no, e il loro predefinito vuoto è precisamente ciò che rende riconoscibile una configurazione incompleta. I valori sono forniti da un file d'ambiente non versionato, di cui la repository_{GG} conserva un esemplare con i soli nomi delle variabili e

segnaposto al posto dei valori: nessun segreto è quindi versionato, e chi predispone un ambiente sa esattamente che cosa deve fornire.

Il confine fra configurazione e codice passa di qui: è configurazione ciò che cambia fra un ambiente e l'altro senza cambiare il comportamento del prodotto (indirizzi, identificativi, credenziali, origini ammesse), ed è codice tutto il resto. Lo schema descrive *che cosa* sia la configurazione; il *come* la si ottenga appartiene al modulo di composizione trattato nella sezione 3.4.5.

La politica CORS Il requisito_G di vincolo_{GG} **R-3-V-Ob**, argomentato nella sezione 2.1, deriva dal fatto che il browser applica la *same-origin policy*: un'applicazione servita da un'origine non può contattarne un'altra se questa non lo consente esplicitamente. Poiché nella configurazione descritta sopra le due unità occupano origini distinte, il servizio applicativo registra all'avvio un componente intermedio che dichiara le origini ammesse. Tali origini provengono dalla configurazione, non dal codice: sono una delle cinque voci dello schema, con l'indirizzo dell'applicazione web come valore predefinito. Metodi e intestazioni sono invece ammessi senza restrizione, mentre l'elenco delle origini resta esplicito ed è il solo controllo effettivo.

Il ciclo di vita del processo Il processo del servizio applicativo ha due momenti dichiarati, e nessuno dei due contiene logica di prodotto.

All'avvio, prima che una qualunque richiesta possa essere accettata, viene verificata la presenza delle chiavi senza le quali il servizio non è in grado di svolgere il proprio compito. Se ne manca una, il processo non parte: solleva un errore che nomina la variabile mancante e ne registra il messaggio nel registro degli eventi. La collocazione è motivata: una chiave assente non è una condizione che sopravviene durante l'esercizio, è nota nell'istante in cui il processo viene avviato, e scoprirla al primo utente significherebbe accettare richieste sapendo di non poterle soddisfare.

Che il nome esatto della variabile compaia nel messaggio non costituisce una deroga al requisito_G **R-110-F-Ob**, che prescrive messaggi di errore privi di dettagli tecnici: quel requisito_G governa le risposte rivolte all'utente, e il registro di avvio non è una di esse. Il destinatario qui è chi predispone l'ambiente, per il quale il nome della variabile è l'unica informazione che rende l'errore azionabile. Il vincolo_{GG} non è quindi allentato: è fuori ambito.

Allo spegnimento vengono rilasciate le risorse di trasporto detenute dagli adattatori verso i servizi esterni, che vivono quanto il processo. Anche questa collocazione è motivata: il rilascio appartiene a chi possiede la risorsa, e il modulo che governa il ciclo di vita si limita a invocare due funzioni senza conoscere quali adattatori concreti siano coinvolti, coerentemente con l'inversione descritta nella sezione 3.4.5.

L'asimmetria fra le due chiavi La sezione 2.3 ha già dichiarato che le due dipendenze esterne differiscono per criticità; dal lato del processo in esecuzione la differenza si manifesta così:

- la chiave del gateway dei modelli linguistici figura fra quelle obbligatorie all'avvio: senza di essa nessuna delle funzioni assistite dall'LLM_{GG} è servibile, cioè il servizio non ha più alcuna ragione di accettare traffico, e il processo termina;
- la chiave del servizio di estrazione non vi figura: il processo parte regolarmente e tutte le altre funzioni restano disponibili. È la sola rotta di generazione a partire da un collegamento a non poter essere servita, e la sua indisponibilità è comunicata per singola richiesta, con lo stato che segnala un servizio temporaneamente non disponibile.

La scelta è una decisione deliberata di degradazione graduale, non una dimenticanza: l'assenza della seconda chiave sottrae una rotta su otto, e far terminare per questo l'intero processo

negherebbe all'utente anche le altre sette, che continuerebbero a funzionare. Che si tratti di una decisione è verificabile_G nel codice, dove un test_{GG} asserisce che senza la chiave del servizio di estrazione il processo si avvia ugualmente: invertire il comportamento richiede di cambiare un'asserzione, non di scoprirlo per caso.

Ambiente di sviluppo riproducibile, non configurazione di produzione Va detto apertamente che cosa il file di composizione descrive, perché la forma (due container, una rete, porte pubblicate) potrebbe suggerire più di quanto vi sia. Tre elementi lo qualificano senza ambiguità: l'applicazione web è servita dal server di sviluppo_{GG} e non da un pacchetto costruito; entrambi i servizi montano la copia di lavoro della rispettiva cartella sopra il contenuto dell'immagine, che viene così sostituito da ciò che si trova sul disco dello sviluppatore; il servizio applicativo è avviato in modalità di ricaricamento automatico al variare dei sorgenti. Sono tre caratteristiche del lavoro quotidiano, non dell'esercizio.

Ciò che il file descrive è quindi un ambiente di sviluppo_{GG} riproducibile: il suo scopo è che sette persone su sistemi_G operativi diversi eseguano il medesimo ambiente con un solo comando, non che il prodotto sia dispiegato presso un utilizzatore. Il prodotto è un elaborato didattico e non è rilasciato in esercizio: una configurazione di produzione non è mancante per omissione, ma perché non esiste il contesto che la richiederebbe. Per esserlo occorrerebbero il pacchetto del frontend_{GG} costruito e servito staticamente, l'assenza dei volumi di sviluppo_{GG} e un'immagine dell'applicazione web autosufficiente, che contenga l'esito della costruzione anziché i sorgenti.

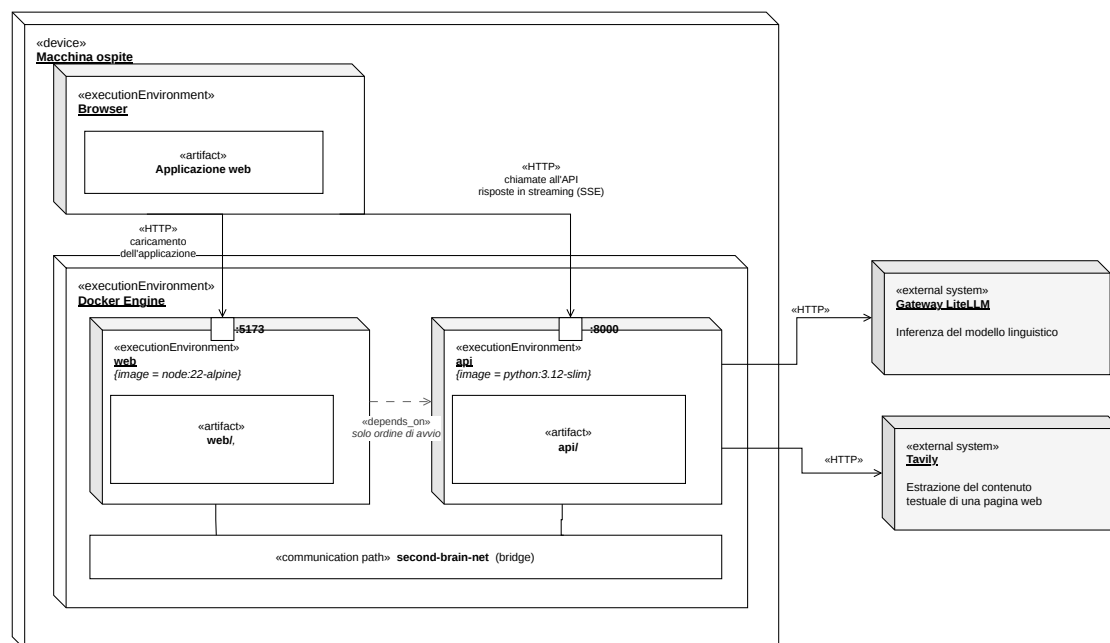


Figura 1: Diagramma di deployment dell'ambiente di sviluppo

3.2.2 Confronto con l'architettura a microservizi

La prima alternativa considerata è la scomposizione del servizio applicativo in più servizi autonomi, ciascuno con il proprio processo, il proprio ciclo di rilascio e la propria unità di esecuzione. È stata scartata perché sproporzionata rispetto al problema, e le ragioni sono misurabili sul codice.

Il servizio applicativo espone otto rotte e non possiede dati: non dispone di un archivio, non conserva stato applicativo fra una richiesta e la successiva, e non esiste quindi alcun insieme di dati da assegnare in proprietà esclusiva a un servizio piuttosto che a un altro, che è il criterio principale con cui una scomposizione in microservizi viene tracciata. Manca inoltre il presupposto della scalabilità differenziata: le operazioni assistite dal modello linguistico condividono la medesima struttura, poiché tutte ricevono un testo, ne compongono le istruzioni e ne inoltrano il flusso prodotto, e il loro carico cresce e diminuisce insieme. Non esiste una componente che debba essere replicata mentre le altre restano ferme.

I costi, per contro, si sarebbero presentati per intero. Sarebbe stato necessario un livello di orchestrazione per governare più unità di esecuzione, un meccanismo di individuazione reciproca perché ciascun servizio potesse localizzare gli altri, e un tracciamento distribuito delle richieste, senza il quale seguire una singola operazione attraverso più processi diventa impraticabile. Soprattutto, ogni collaborazione oggi realizzata da una chiamata di funzione sarebbe diventata una chiamata di rete, cioè un'operazione che può fallire parzialmente, andare in scadenza o riuscire a metà, e che va quindi corredata di ritentativi, scadenze e politiche di reazione al guasto. Su un'operazione che deve restituire testo in streaming, ogni salto di rete aggiunto si somma al tempo che separa l'utente dal primo frammento visibile.

Il beneficio corrispondente (rilascio e scalabilità indipendenti di parti che evolvono a ritmi diversi) non si sarebbe presentato, perché tali parti non esistono. L'indipendenza di cui il prodotto ha effettivamente bisogno è quella fra le due unità già individuate nella sezione 3.1, ed è ottenuta senza alcuna di quelle infrastrutture: le due unità hanno catene di costruzione distinte, sono avviate separatamente e comunicano attraverso un contratto esplicito.

3.2.3 Confronto con il monolite a strati

La seconda alternativa considerata è la disposizione del servizio applicativo in livelli sovrapposti (presentazione, logica applicativa, accesso ai servizi esterni), secondo la struttura tradizionale del monolite a strati. Il confronto con l'esagono non riguarda il numero dei package né i loro nomi, che sarebbero stati pressoché gli stessi: riguarda la direzione delle dipendenze.

Nel monolite a strati ogni livello dipende da quello sottostante: la logica applicativa importa il livello di accesso ai servizi esterni, e con esso le librerie che quel livello impiega. Nulla impedisce quindi al dominio_{GG} di importare il client HTTP, il client di un fornitore o il framework_{GG} web, e l'unica difesa contro il fatto che ciò accada è la disciplina di chi scrive. Nell'esagono la dipendenza è invertita dalla porta e punta sempre verso il centro; la differenza, come descritto nella sezione 3.4.1, non è affidata a un'intenzione ma verificata_G staticamente dai contratti di dipendenza dichiarati nel repository_{GG}.

Le conseguenze pratiche sono due, ed entrambe si pagherebbero a ogni modifica.

- **Prova di un caso d'uso**

Nel monolite a strati esercitare una singola operazione richiederebbe di predisporre le componenti dei livelli inferiori da cui essa dipende, cioè di montare l'infrastruttura per provare una regola che con l'infrastruttura non ha nulla a che vedere. Nell'esagono la stessa operazione riceve una porta, e alla porta si sostituisce un doppio.

- **Sostituzione di un fornitore esterno**

Nel monolite a strati la conoscenza del fornitore sarebbe diffusa in tutto il codice che lo invoca, e sostituirlo comporterebbe di intervenire ovunque quel codice si trovi. Nell'esagono la conoscenza è confinata in un adattatore, e la sostituzione consiste nello scriverne un altro.

Il costo dell'esagono va dichiarato: è un livello di indirezione in più, poiché fra chi chiede un'operazione e chi la esegue si interpone un'interfaccia astratta. È il prezzo accettato in cambio delle due proprietà appena descritte.

3.3 Architettura del frontend

L'applicazione web è organizzata secondo il principio dichiarato in §3.1.3: componenti React che compongono l'interfaccia per annidamento, stato condiviso esterno all'albero in due store distinti, flusso dei dati unidirezionale. La presente sezione descrive l'articolazione interna in sei aspetti: organizzazione dei sorgenti, scomposizione in componenti, gestione dello stato, integrazione_G dell'editor, accesso al servizio applicativo e persistenza lato client.

Due precisazioni preliminari. Primo: il «flusso unidirezionale» vale a livello generale — React legge dallo store, le azioni lo modificano, i componenti sottoscritti si ridisegnano. Secondo: l'integrazione_G con CodeMirror, descritta in §3.3.4, è un'eccezione controllata. L'API_G di CodeMirror è imperativa, richiede una sincronizzazione bidirezionale tra l'istanza dell'editor e lo store. La sezione spiega come questa eccezione è gestita in modo da non rompere il principio generale.

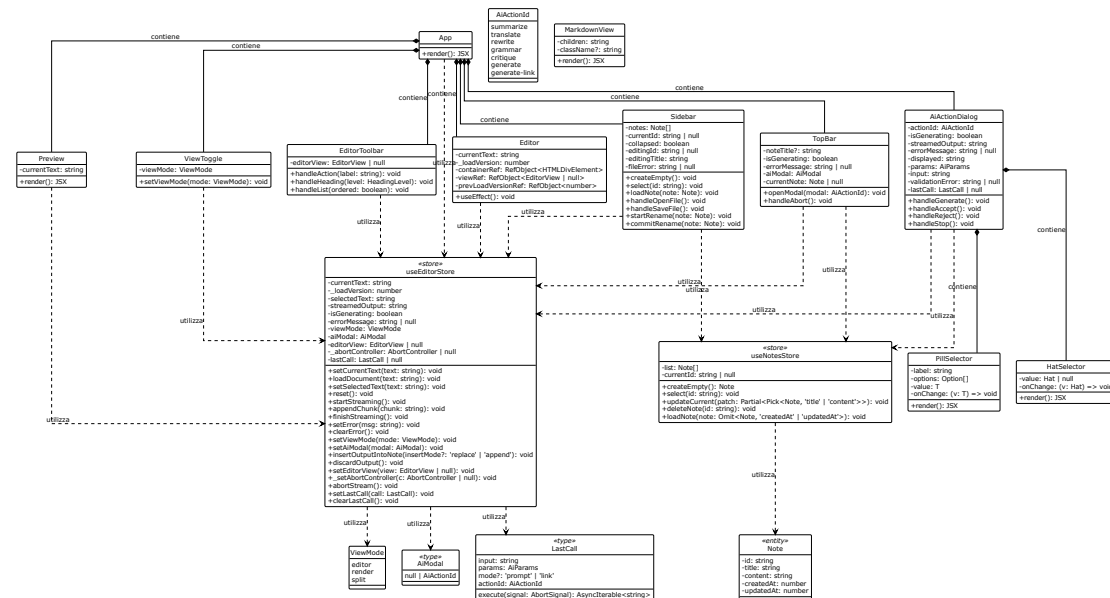


Figura 2: Diagramma UML 3.3-1 - Componenti UI e Store

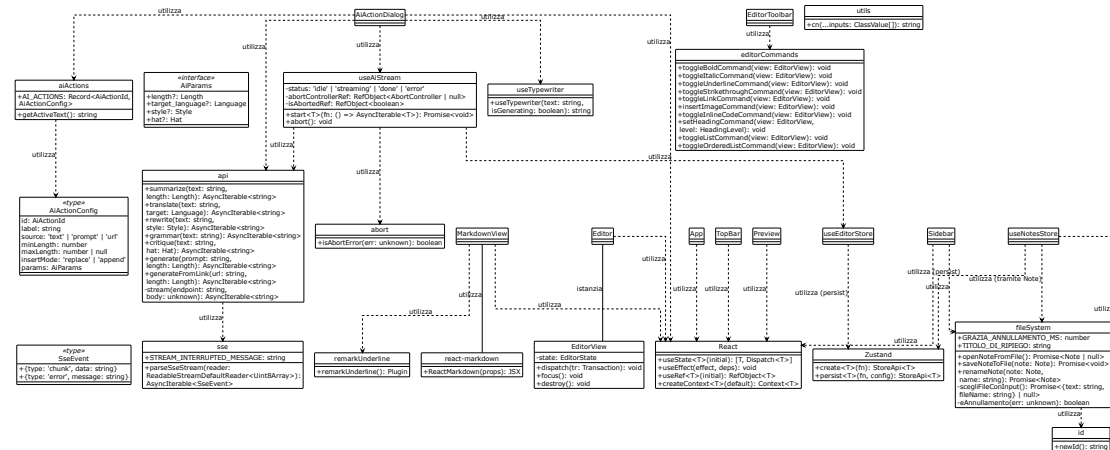


Figura 3: Diagramma UML 3.3-2 -Hooks, Librerie e Dipendenze Esterne

3.3.1 Organizzazione dei sorgenti

La cartella **web/src/** è organizzata per **ruolo**, non per feature. Ogni cartella contiene file che condividono la stessa responsabilità architetturale, indipendentemente dalla funzione di prodotto. Nel livello radice di **src/** risiedono i file di ingresso dell'applicazione: **main.tsx** (entry point del bundle, che monta l'applicazione nel DOM), **App.tsx** (componente radice, che definisce il layout principale e la griglia dinamica dell'area di lavoro), e **index.css** (foglio di stile globale, che include le direttive di Tailwind CSS e le variabili di tema). I moduli organizzati per ruolo sono invece distribuiti nelle seguenti cartelle:

- **components/**: componenti React. **ui/** contiene primitivi (**alert**, **button**, **textarea**) realizzati con **radix-ui** e **class-variance-authority**; i file direttamente in **components/** contengono i componenti compositi: **Sidebar**, **TopBar**, **Editor**, **EditorToolbar**, **Preview**, **ViewToggle**, **AiActionDialog**, **MarkdownView**. All'interno di **AiActionDialog.tsx** sono definiti i componenti interni **PillSelector** e **HatSelector**, non esportati separatamente.
- **hooks/**: hook personalizzati: **useAiStream** (logica di streaming), **useTypewriter** (effetto di battitura progressiva). L'hook **useAiStream** soddisfa **R-109-F-De** (indicatore di avanzamento per operazioni lunghe) e **R-79-F-De** (annullamento di una richiesta in corso).
- **store/**: i due store Zustand: **useEditorStore.ts** (stato della sessione di scrittura), **notes.ts** (raccolta delle note).
- **lib/**: utility e servizi: **api.ts** (facade **API_G**), **sse.ts** (parser SSE), **filesystem.ts** (importazione/esportazione), **abort.ts** (gestione abort), **id.ts** (generazione ID), **editorCommands.ts** (comandi **CodeMirror**), **aiActions.ts** (registry delle azioni AI), **remarkUnderline.ts** (plugin Remark), **utils.ts** (funzione **cn**).
- **types/**: definizioni dei tipi. **api.ts** è auto-generato da **openapi-typescript** dallo schema OpenAPI; **models.ts** lo re-esporta isolando il resto dell'applicazione dalla struttura generata.
- **test/**: contiene il **setup.ts** dei test_G (Vitest, Testing Library, jsdom), soddisfacendo **R-2-Q-Ob** e **R-3-Q-Ob** (test_G automatici e copertura).
- **assets/**: risorse statiche: **hero.png**, **react.svg**, **vite.svg**.

Questa organizzazione rende prevedibile la collocazione di un nuovo file. Facilita inoltre la verifica_G automatica delle dipendenze: un linter può controllare che i componenti non importino direttamente i tipi generati dall'OpenAPI senza passare dal barrel, e che gli store non importino i componenti.

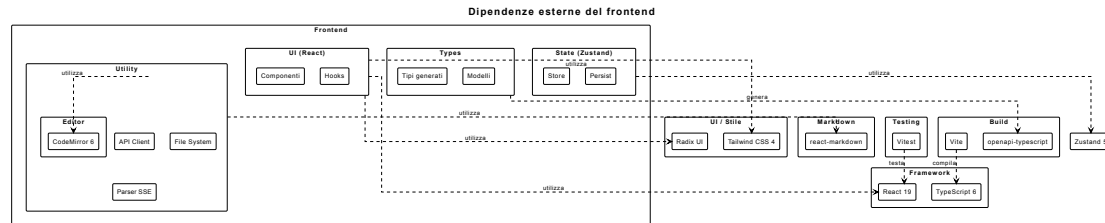


Figura 4: Diagramma UML 3.3.1 - Organizzazione dei sorgenti del frontend

3.3.2 Scomposizione in componenti

L'albero dei componenti ha radice in **App**, che definisce il layout a tre colonne e la griglia dinamica dell'area di lavoro.

App legge `viewMode` da `useEditorStore` tramite il selettore `useViewMode`. La struttura del `main` cambia in base al valore: in modalità `split` viene resa una griglia a due colonne (`md:grid-cols-2`); in modalità `editor` o `render` un singolo pannello a tutta larghezza. La presenza di `Editor` e `Preview` è determinata dalla stessa condizione. Questa logica soddisfa i requisiti_G R-50-F-Ob, R-51-F-Ob e R-52-F-Ob (visualizzazioni esclusiva editor, esclusiva render e combinata).

I componenti composti sono i seguenti:

- **Sidebar**: elenca le note da `notes` tramite `useNotesList`. Espone un pulsante per creare una nuova nota (`createEmpty`) che soddisfa R-82-F-Ob e R-114-F-Ob. Gestisce la selezione (`select`). Ogni voce mostra il titolo e supporta la ridenominazione tramite doppio click, che attiva un campo di input_G (R-94-F-De). Offre i pulsanti «Apri file...» e «Salva file», che invocano `openNoteFromFile` e `saveNoteToFile` (§3.3.6), soddisfacendo R-85-F-Ob e R-88-F-Ob. Un pulsante di collasso riduce la barra a una colonna stretta.

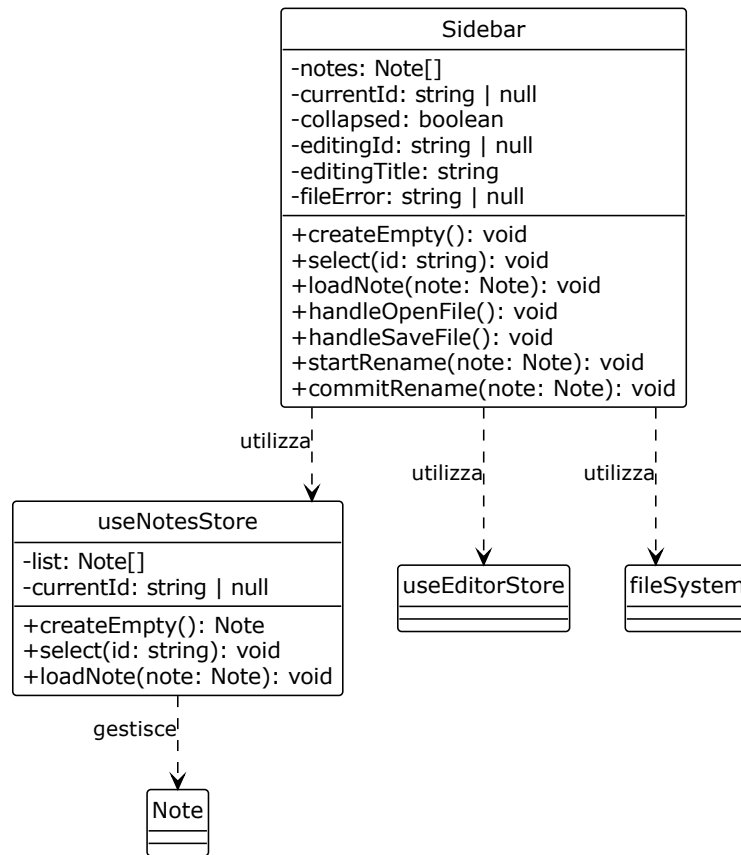


Figura 5: Diagramma UML 3.3.2-1 - Sidebar

- **TopBar**: presenta il titolo della nota corrente (da `useCurrentNote`) e i pulsanti delle azioni AI. Alla pressione di un pulsante, invoca `setAiModal` per aprire il dialogo corrispondente. Durante lo streaming (`isGenerating`), mostra un pulsante di interruzione (`abortStream`) e un indicatore di caricamento (R-109-F-De). In caso di errore, visualizza `errorMessage` in un alert (R-110-F-Ob).

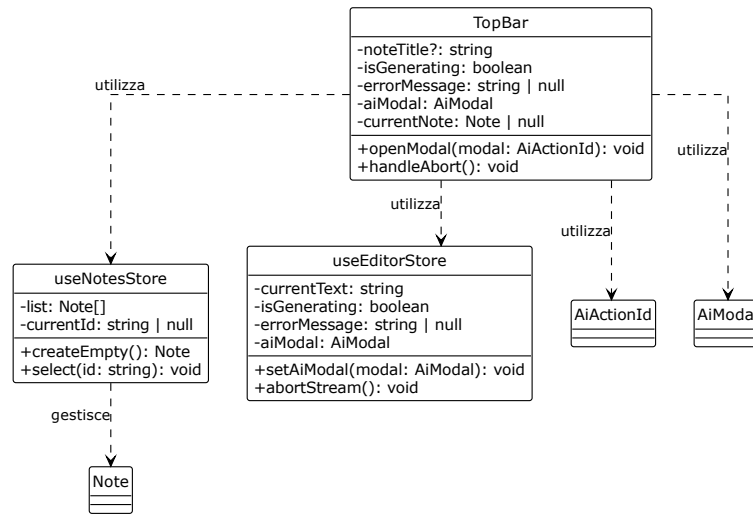


Figura 6: Diagramma UML 3.3.2-2 - TopBar

- **ViewToggle**: tre pulsanti che impostano `viewMode` su `editor`, `split` o `render`. È sempre visibile, in qualsiasi modalità, per consentire il passaggio diretto a una vista diversa.

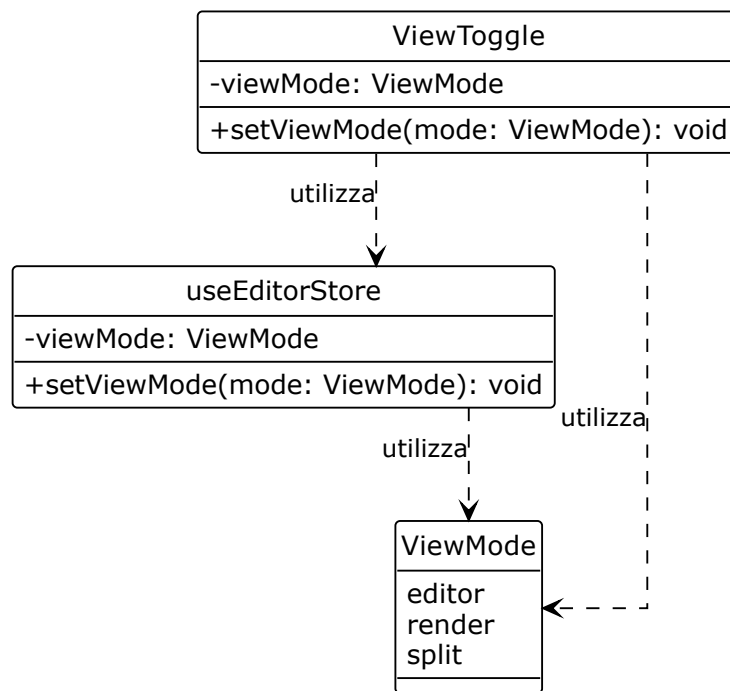


Figura 7: Diagramma UML 3.3.2-3 - ViewToggle

- **EditorToolbar**: contiene i controlli per la formattazione del testo Markdown. Le operazioni corrispondono ai casi d'uso_G UC8-UC10 (grassetto, R-9-F-Ob, R-10-F-Ob), UC11-UC13 (corsivo, R-11-F-Ob, R-12-F-Ob), UC14-UC16 (sottolineato, R-13-F-Ob, R-14-F-Ob), UC17-UC19 (barrato, R-15-F-De, R-16-F-De), UC20 (inserimento titolo, R-17-F-Ob), UC22 (rimozione titolo, R-18-F-Ob), UC23 (inserimento sottotitolo, R-19-F-Ob), UC25 (rimozione sottotitolo, R-20-F-Ob), UC26 (inserimento elenco, R-21-F-De, R-22-F-De), UC28 (rimozione elenco, R-23-F-De), UC38 (inserimento link, R-30-F-De), UC49 (inserimento immagine, R-39-F-De), e UC52/UC53 (attivazione/disattivazione auto-completamento, R-42-F-De, R-43-F-De). Ogni comando invoca la corrispondente funzione da `lib/editorCommands.ts` sull'istanza `editorView` conservata nello store.

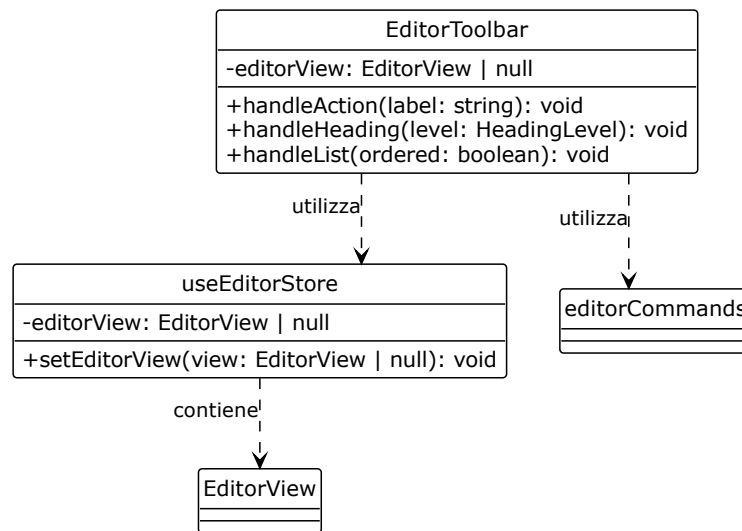


Figura 8: Diagramma UML 3.3.2-4 - EditorToolbar

- **Editor**: il componente che istanzia CodeMirror e gestisce la sincronizzazione bidirezionale con `useEditorStore` (§3.3.4). Soddisfa i requisiti_G R-1-F-Ob (inserimento testo) e R-2-F-Ob (visualizzazione grafica del Markdown).

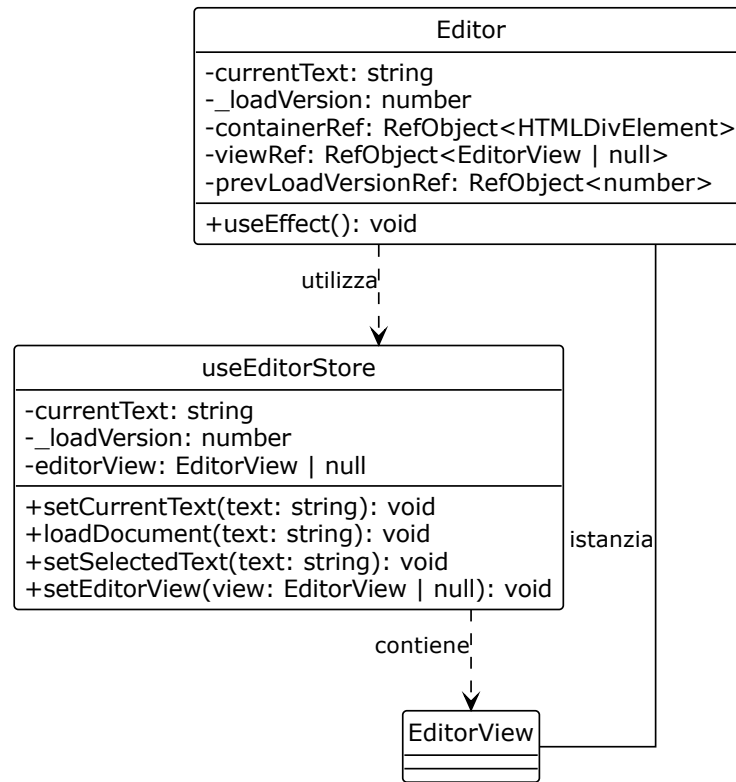


Figura 9: Diagramma UML 3.3.2-5 - Editor

- **Preview:** rende il contenuto Markdown della nota corrente usando **MarkdownView**. Applica i plugin `remark-gfm`, `remark-math`, `rehype-katex` e `rehype-highlight`. Soddisfa R-2-F-Ob e R-113-F-Ob.

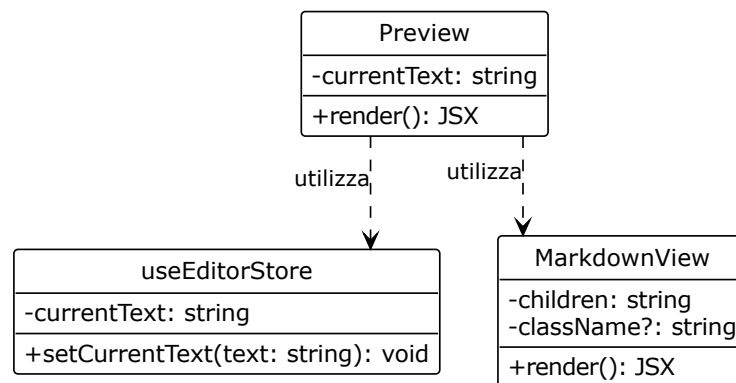


Figura 10: Diagramma UML 3.3.2-6 - Preview

- **MarkdownView**: componente condiviso da **Preview** e dall'anteprima dell'**AiActionDialog**. Applica la stessa pipeline Markdown a entrambi i contesti, garantendo coerenza visiva. Non abilita HTML raw. Utilizza **remarkUnderline** per supportare la sintassi **++sottolineato++**.

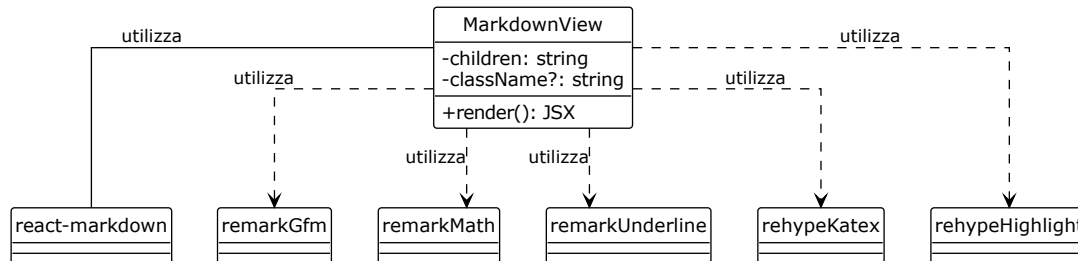


Figura 11: Diagramma UML 3.3.2-7 - MarkdownView

- **AiActionDialog**: il dialogo modale che si apre quando **aiModal** non è null. La sua implementazione è articolata:
 - **Registry delle azioni**: le azioni AI sono registrate in **lib/aiActions.ts** (pattern *Registry*, §3.5). Il registry mappa ciascun **AiActionId** alla funzione API_G , al titolo del dialogo, ai campi del form, alla sorgente del testo (**text**, **prompt** o **url**), alla lunghezza minima e massima del testo in $input_G$, e alla modalità di inserimento dell' $output_G$ (**replace** per trasformazioni, **append** per generazioni). Il dialogo soddisfa R-54-F-Ob (accesso al LLM_G).
 - **Funzione `getActiveText`**: determina il testo da sottoporre all'AI. Se l'utente ha una selezione attiva in **selectedText**, viene usata quella. Altrimenti, l'intero **currentText**. La logica è centralizzata per garantire uniformità tra le azioni.
 - **Parametri e validazione**: il dialogo presenta un campo di $input_G$ (textarea per prompt, $input_G$ per URL, o nessun campo per le azioni sul testo) e selettori per i parametri specifici di ciascuna azione. I selettori soddisfano R-56-F-De (tipologia di riassunto), R-58-F-Ob (lingua di destinazione), R-60-F-Ob (stile di riscrittura), R-64-F-Ob (prospettiva di analisi), R-73-F-De (lunghezza del testo generato) e R-74-F-De (URL come fonte). I selettori sono **PillSelector** e **HatSelector**, definiti all'interno del file **AiActionDialog.tsx**. La validazione $_G$ controlla che il testo in $input_G$ soddisfi **minLength** e **maxLength** (R-81-F-Ob), e che i parametri obbligatori siano selezionati. Gli errori di validazione $_G$ sono visualizzati in un alert (R-110-F-Ob).
 - **Stato locale**: il dialogo mantiene uno stato locale (**input**, **params**, **validationError**, **lastCall**) gestito tramite **useState**. **lastCall** conserva l' $input_G$ e i parametri dell'ultima chiamata. Il pulsante di generazione mostra «Rigenera» invece di «Genera» quando i parametri non sono cambiati (R-78-F-De).
 - **Anteprima e accettazione**: l' $output_G$ in streaming è visualizzato in un'anteprima che usa **MarkdownView** e l'effetto di battitura **useTypewriter**. Il pulsante «Accetta» invoca **insertOutputIntoNote** (R-76-F-Ob). «Rifiuta» invoca **discardOutput** (R-77-F-De). Il pulsante «Accetta» è disabilitato finché l' $output_G$ è vuoto, in generazione, o nel caso particolare di **grammar** che restituisce **NO_ERRORS_MARKER** (nessun errore rilevato, R-62-F-De).

- **Terminazione:** la chiusura del dialogo tramite overlay o tasto ESC invoca `handleReject`, che abortisce lo streaming e scarta l'output_G.
- **Esautività del contratto:** il file `AiActionDialog.tsx` contiene vincoli di tipo che garantiscono che tutti i valori enumerati del contratto OpenAPI siano presenti nei selettori. La riga `type LanguagesCoverContract = Exclude<Language, (typeof LANGUAGES)[number]['value']>` produce un errore di compilazione se il backend aggiunge una lingua che il frontend_G non espone, o se il frontend_G espone una lingua che il backend non prevede. La divergenza tra contratto e interfaccia diventa una mancata compilazione, non un difetto in produzione.

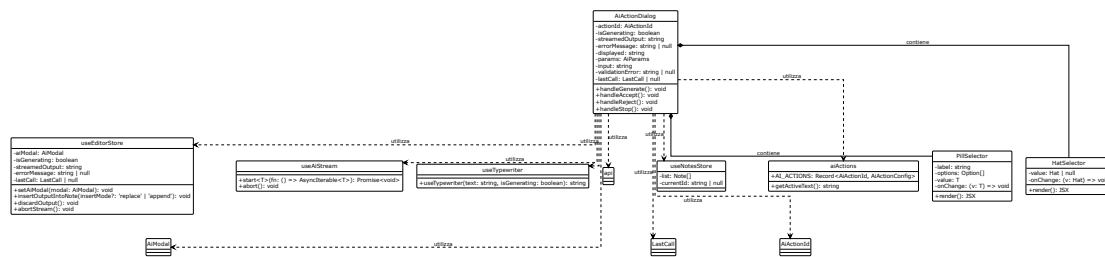


Figura 12: Diagramma UML 3.3.2-8 - AiActionDialog

3.3.3 Gestione dello stato

Lo stato condiviso è custodito in due store Zustand. Entrambi rispettano la regola dei **selettori atomici** (descritta in §3.5): ogni store espone selettori che estraggono una singola porzione di stato; i componenti si sottoscrivono solo a questi, mai allo stato aggregato. Un componente viene ridisegnato solo quando la porzione a cui è sottoscritto cambia. Questa proprietà rende sostenibile lo streaming: **Preview**, sottoscritto a `useCurrentText`, non viene ridisegnato a ogni chunk ricevuto; solo il componente che presenta **streamedOutput** viene aggiornato.

La regola di collocazione dello stato è graduale: stato locale al componente per difetto, sollevato al genitore quando due componenti vicini devono concordare, promosso a store soltanto quando componenti lontani nell'albero devono condividerlo. Lo store non è il deposito di tutto lo stato: le condizioni puramente locali di un componente restano nel componente che le possiede.

useEditorStore. Custodisce lo stato della sessione di scrittura e delle interazioni AI:

- **currentText:** il testo corrente della nota, aggiornato da `setCurrentText` a ogni modifica dell'editor.
- **_loadVersion:** contatore incrementato a ogni caricamento di una nuova nota. Serve a invalidare le operazioni asincrone in corso.
- **selectedText:** il testo selezionato dall'utente nell'editor (R-3-F-Ob), aggiornato da `setSelectedText` tramite un listener di CodeMirror.
- **streamedOutput:** il testo in arrivo dal servizio applicativo, accumulato chunk per chunk da `appendChunk`.
- **isGenerating:** flag che indica se uno streaming è in corso. `startStreaming` lo imposta a `true`; `finishStreaming` e `setError` lo imposta a `false`.

- **errorMessage**: messaggio di errore da visualizzare all'utente (R-110-F-Ob); **setError** lo imposta e disabilita **isGenerating**.
- **viewMode**: la modalità di visualizzazione dell'area di lavoro (**editor**, **render**, **split**), impostata da **setViewMode**.
- **aiModal**: l'identificatore dell'azione AI attualmente aperta nel dialogo, impostato da **setAiModal**.
- **editorView**: riferimento all'istanza di **CodeMirror**, utilizzato per operazioni di manipolazione diretta. **setEditorView** lo imposta al montaggio dell'editor.
- **_abortController**: riferimento all'**AbortController** dello streaming in corso.
- **lastCall**: conserva l'ultima richiesta AI effettuata, con $input_G$, parametri, modalità, identificatore dell'azione e la funzione eseguibile. È utilizzato per la rigenerazione (R-78-F-De) e per la gestione dello stato «Rigenera» nel dialogo. **setLastCall** e **clearLastCall** ne gestiscono il ciclo di vita.

L'azione **abortStream** interrompe la comunicazione tramite **abort()** sul controller (R-79-F-De) e, contemporaneamente, spegne l'indicatore di attesa (**isGenerating** a **false**). La proprietà è intenzionale: R-109-F-De richiede che l'indicatore di avanzamento copra l'intera durata percepita dell'operazione; al momento dell'annullamento, la durata percepita termina immediatamente, e l'indicatore si spegne senza attendere che il loop di streaming si accorga dell'interruzione.

L'azione **insertOutputIntoNote** determina la semantica di inserimento in base al parametro **insertMode**:

- **replace**: sostituisce il **selectedText** se presente e se ancora trovabile in **currentText**; altrimenti sostituisce l'intero **currentText**. È la semantica usata per le azioni di trasformazione (riassunto, riscrittura, traduzione, grammatica, analisi).
- **append**: accoda l' $output_G$ alla fine di **currentText**, aggiungendo un separatore **\n\n** se necessario. È la semantica usata per la generazione da prompt o da link.

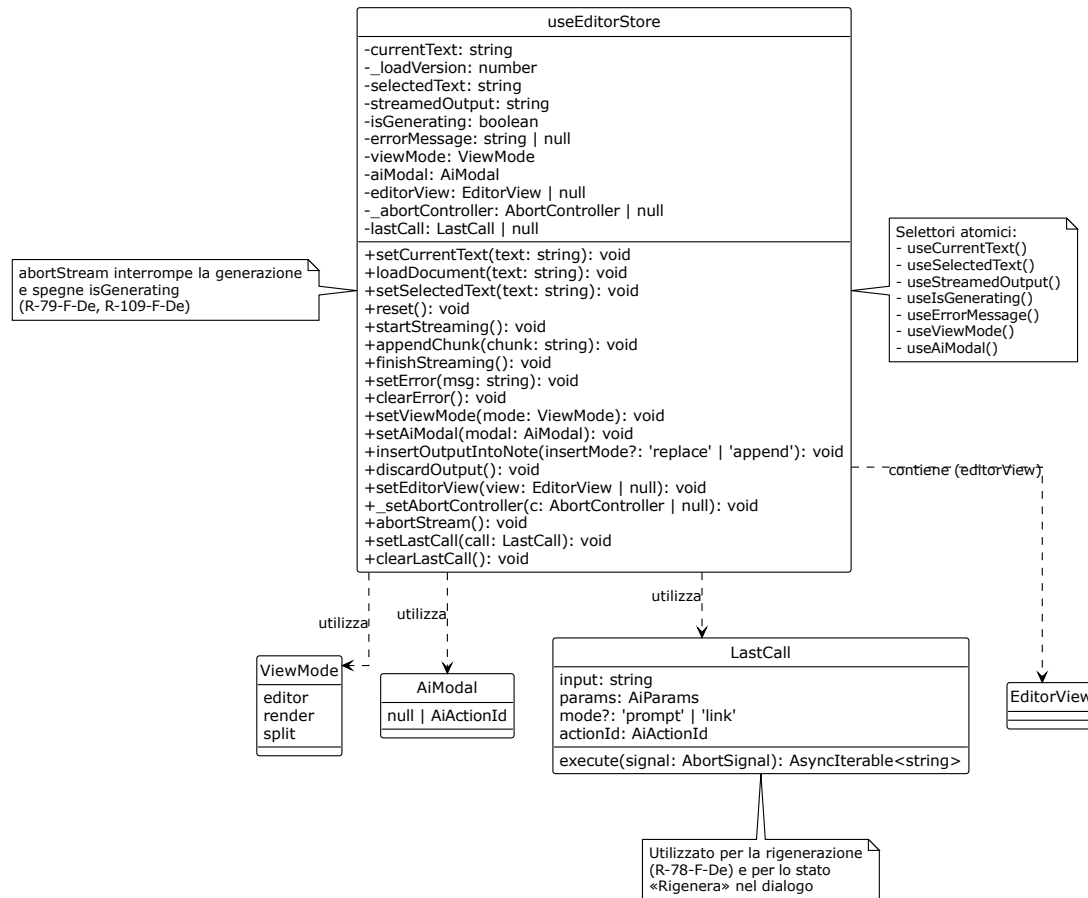


Figura 13: Diagramma UML 3.3.3-1 - useEditorStore

notes. Custodisce la raccolta delle note e l'indicazione di quale sia quella corrente:

- **list:** array di oggetti Note (con id, title, content, createdAt, updatedAt).
- **currentId:** id della nota correntemente selezionata, o null.
- **createEmpty:** crea una nuova nota con titolo «Senza titolo» e contenuto vuoto (R-82-F-Ob, R-114-F-Ob). Se una nota è già aperta, salva il suo contenuto corrente in useEditorStore prima di crearne una nuova. Incrementa _loadVersion e imposta currentText a ". Soddisfa UC74.1.
- **select:** cambia la nota corrente. Prima di cambiare, salva il contenuto corrente della nota attuale in list; poi carica il contenuto della nuova nota in useEditorStore tramite loadDocument.
- **updateCurrent:** aggiorna title e/o content della nota corrente, aggiornando updatedAt.
- **deleteNote:** rimuove la nota dall'elenco (R-91-F-De). Se la nota eliminata era quella corrente, seleziona la prima nota rimanente, se presente.
- **loadNote:** carica una nota proveniente da un'importazione (R-85-F-Ob), aggiornandola se già presente (per id) o aggiungendola se nuova.

La sincronizzazione tra i due store è gestita dalle azioni di **notes**: ogni volta che l'utente cambia nota o ne crea una nuova, il contenuto della nota corrente viene prima salvato in **useEditorStore** e poi la nuova nota viene caricata. Il flag **_loadVersion** impedisce che aggiornamenti asincroni dell'editor sovrascrivano il contenuto di una nota diversa da quella che l'utente stava effettivamente modificando.

Persistenza dello store. **notes** utilizza il middleware **persist** di Zustand (pattern descritto in §3.5) per salvare automaticamente **list** e **currentId** in **localStorage**. Il middleware serializza lo stato su ogni modifica e lo ripristina all'avvio dell'applicazione. Durante il ripristino (**onRehydrateStorage**), **loadDocument** viene chiamato per caricare il contenuto della nota corrente nell'editor. Questa persistenza soddisfa R-80-F-Ob, R-84-F-Ob, R-87-F-Ob, R-90-F-Ob, R-93-F-De, R-96-F-De e UC75.

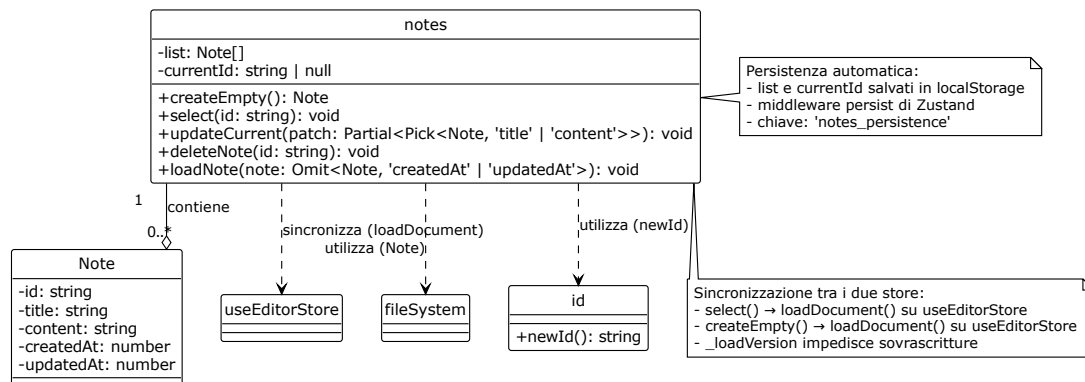


Figura 14: Diagramma UML 3.3.3-2 - notes

3.3.4 Integrazione dell'editor

L'editor è realizzato con CodeMirror 6. È integrato con lo store tramite un meccanismo di sincronizzazione bidirezionale controllata. Come anticipato in apertura, questa è un'eccezione al principio del flusso unidirezionale, dovuta all'API_G imperativa di CodeMirror.

Creazione dell'istanza. L'istanza di **EditorView** viene creata una sola volta, al montaggio del componente **Editor**. Le estensioni sono configurate in un array stabile per evitare il ricaricamento dell'editor:

- **lineNumbers()**: numeri di riga.
- **history()**: cronologia di undo/redo (R-7-F-De, R-8-F-De).
- **markdown()**: supporto alla sintassi Markdown (R-112-F-Ob).
- **closeBrackets()**: chiusura automatica di parentesi e virgolette (R-42-F-De, R-43-F-De).
- **EditorView.lineWrapping**: a capo automatico delle righe lunghe.
- **search({ top: true })**: barra di ricerca.
- **keymap.of([...defaultKeymap, ...historyKeymap, ...closeBracketsKeymap, ...searchKeymap, { key: 'Mod-k', run: toggleLinkCommand }])**: scorciatoie da tastiera.

L'istanza viene salvata nello store tramite `setEditorView` e rimossa al momento dello smontaggio.

Sincronizzazione store → editor. Quando il contenuto della nota cambia nello store (`currentText`), l'editor deve rifletterlo. Il meccanismo:

1. Il componente `Editor` si sottoscrive a `useCurrentText` e a `_loadVersion`.
2. Quando il valore cambia, il componente verifica se il nuovo testo è diverso da quello attualmente visualizzato (`view.state.doc.toString()`).
3. In caso affermativo, costruisce una `Transaction` che sostituisce l'intero documento e la invia all'editor tramite `view.dispatch()`.
4. La transazione è annotata con `StoreSync.of(true)`. Questa annotazione segnala all'`updateListener` che la modifica proviene dallo store e non deve essere re-inviata allo store, evitando il ciclo.
5. Se il cambiamento è dovuto a `_loadVersion` (caricamento di una nuova nota), la transazione riceve anche `Transaction.addToHistory.of(false)`. Questa annotazione impedisce che il caricamento di una nota venga registrato nella cronologia di undo, soddisfacendo R-7-F-De e R-8-F-De.

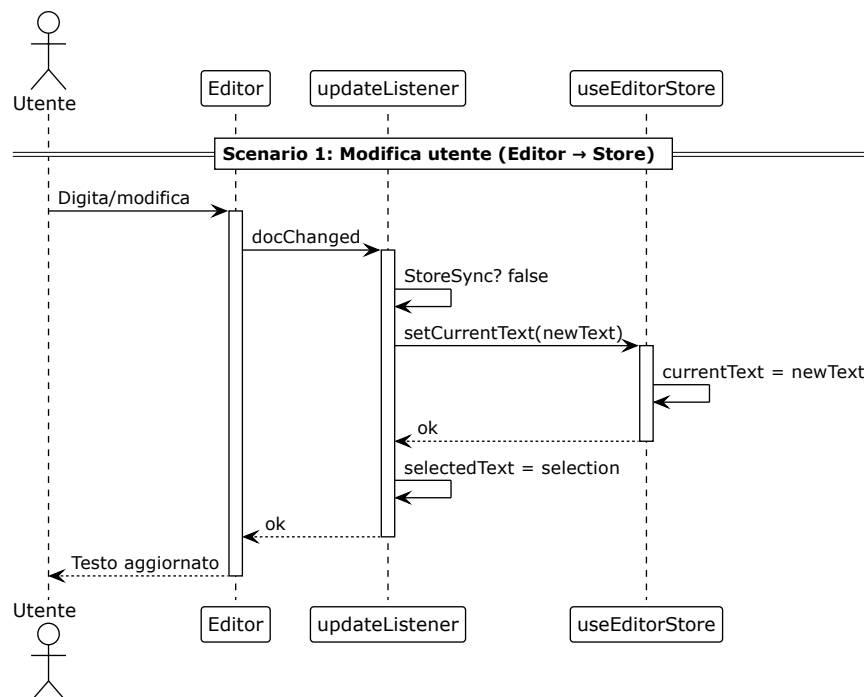


Figura 15: Diagramma UML 3.3.4-1 - Sincronizzazione store → editor

Sincronizzazione editor → store. Quando l'utente modifica il testo nell'editor, lo store deve aggiornare `currentText`. Il meccanismo:

1. L'estensione `EditorView.updateListener` intercetta ogni transazione che modifica il documento.
2. Se la transazione ha l'annotazione `StoreSync`, viene ignorata.
3. Se la transazione ha modificato il documento (`update.docChanged`), l'estensione estrae il nuovo testo completo (`update.state.doc.toString()`) e invoca `setCurrentText` con esso.
4. Lo stesso listener monitora anche i cambiamenti di selezione (`update.selectionSet`) e aggiorna `selectedText` nello store (R-3-F-Ob).

Evitare il ciclo. La sincronizzazione bidirezionale potrebbe generare un ciclo: modifica editor → `setCurrentText` → re-render del componente → `view.dispatch()` → modifica editor → ... Il ciclo è evitato da due meccanismi:

1. **Confronto:** il dispatch store→editor è eseguito solo se il testo corrente dell'editor è diverso da `currentText`. L'`updateListener` editor→store esegue `setCurrentText` solo se il testo appena modificato è diverso dal valore già in store.
2. **Annotazione `StoreSync`:** le transazioni store→editor sono marcate; l'`updateListener` le ignora, interrompendo il ciclo alla radice.

Selezione. La selezione dell'utente nell'editor è monitorata dall'estensione di `update`, che estrae l'intervallo selezionato e invoca `setSelectedText`. Il valore è usato da `getActiveText` per determinare il testo da sottoporre all'AI (R-3-F-Ob).

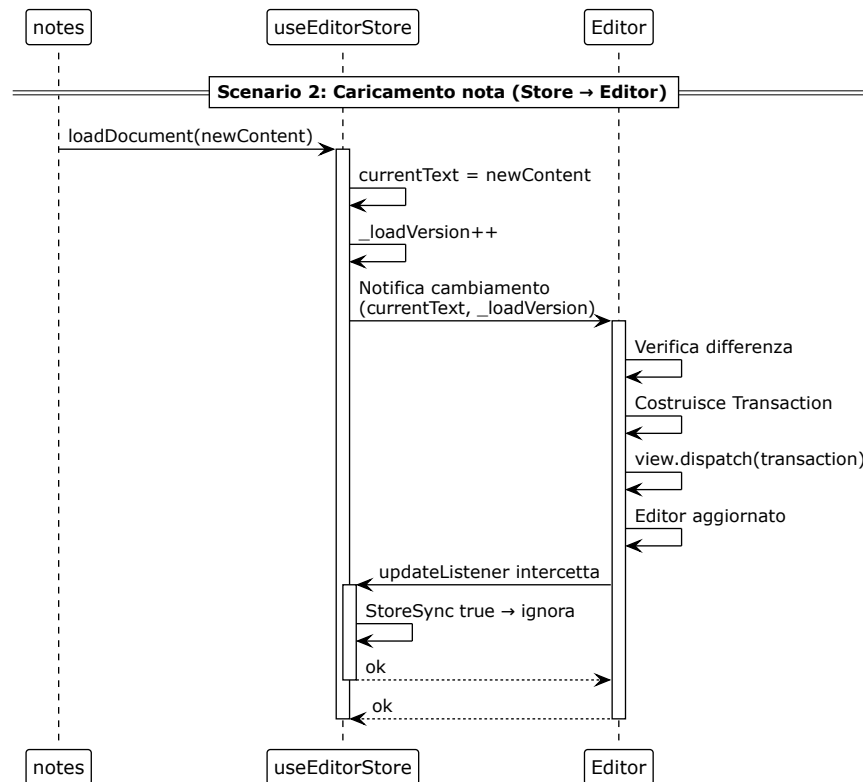


Figura 16: Diagramma UML 3.3.4-2 - Sincronizzazione editor → store

3.3.5 Accesso al servizio applicativo

L'accesso al servizio applicativo è incapsulato in tre livelli: una **facade API** in `lib/api.ts` (pattern *Facade*, §3.5), un **parser SSE** in `lib/sse.ts`, e un **hook custom** `useAiStream` in `hooks/useAiStream.ts`.

Facade API. La facade espone una funzione per ciascuna operazione del servizio applicativo: `summarize`, `translate`, `rewrite`, `grammar`, `critique`, `generate`, `generateFromLink`. Ogni funzione costruisce il corpo della richiesta con i tipi definiti in `types/models.ts` e invoca la funzione interna `stream`, che:

1. Esegue una richiesta POST all'endpoint corrispondente (con base URL da variabile d'ambiente `VITE_API_BASE_URL`).
2. Se lo stato HTTP non è 2xx, estrae `detail` dal corpo JSON e solleva un errore.
3. Se la risposta ha un corpo leggibile, crea un reader e itera sugli eventi restituiti da `parseSseStream`.
4. La funzione ritorna un `AsyncIterable<string>` che produce i frammenti in sequenza.

L'uso dei tipi generati da OpenAPI (`types/api.ts`) garantisce la conformità al contratto tra frontend_G e backend, soddisfacendo R-2-V-Ob (interfacciamento con servizi LLM_G tramite API_G).

Parser SSE: `parseSseStream`. Il parser `parseSseStream` (in `lib/sse.ts`) trasforma il `ReadableStreamDefaultReader` in un `AsyncIterable<SseEvent>`, dove `SseEvent` è definito come `{ type: 'chunk'; data: string } | { type: 'error'; message: string }`. La sua implementazione si basa sulla struttura della specifica SSE e distingue tre terminatori:

1. `data: [DONE]`: il generatore termina senza eventi di errore (successo).
2. `event: error`: il backend segnala un errore. Il parser produce un evento `{ type: 'error', message: ... }`.
3. Chiusura del reader senza nessuno dei due: il parser produce un evento di errore con il messaggio `STREAM_INTERRUPTED_MESSAGE` («La generazione si è interrotta prima di completarsi. Riprova.»). Questo messaggio è generato lato client (R-110-F-Ob).

Il parser gestisce il buffering delle righe e le terminazioni CRLF. È una **pure function**: non ha dipendenze da React o dallo store; può essere testata in isolamento.

Hook `useAiStream`. `useAiStream` incapsula il ciclo di vita di uno streaming: avvio, accumulo dei frammenti, gestione degli errori e interruzione. È descritto in dettaglio in §3.5 come esempio del pattern *Custom Hook*.

- `start<T>`: riceve una funzione che ritorna un `AsyncIterable<T>`. Crea un nuovo `AbortController`, lo salva nello store (`_setAbortController`) e avvia l'iterazione. Per ogni frammento, controlla se il controller è stato interrotto. Al termine, imposta `isGenerating` a `false` e rimuove il riferimento al controller.
- `abort`: interrompe la comunicazione tramite `controller.abort()` (R-79-F-De).
- `status`: uno stato `'idle' | 'streaming' | 'done' | 'error'` che il componente può utilizzare per mostrare indicatori visivi (R-109-F-De).

L'hook è progettato per essere usato in un solo componente alla volta (`IAiActionDialog`). L'aborto è accessibile anche da altri componenti (la `TopBar`) tramite il riferimento salvato nello store, che espone `abortStream` come azione pubblica.

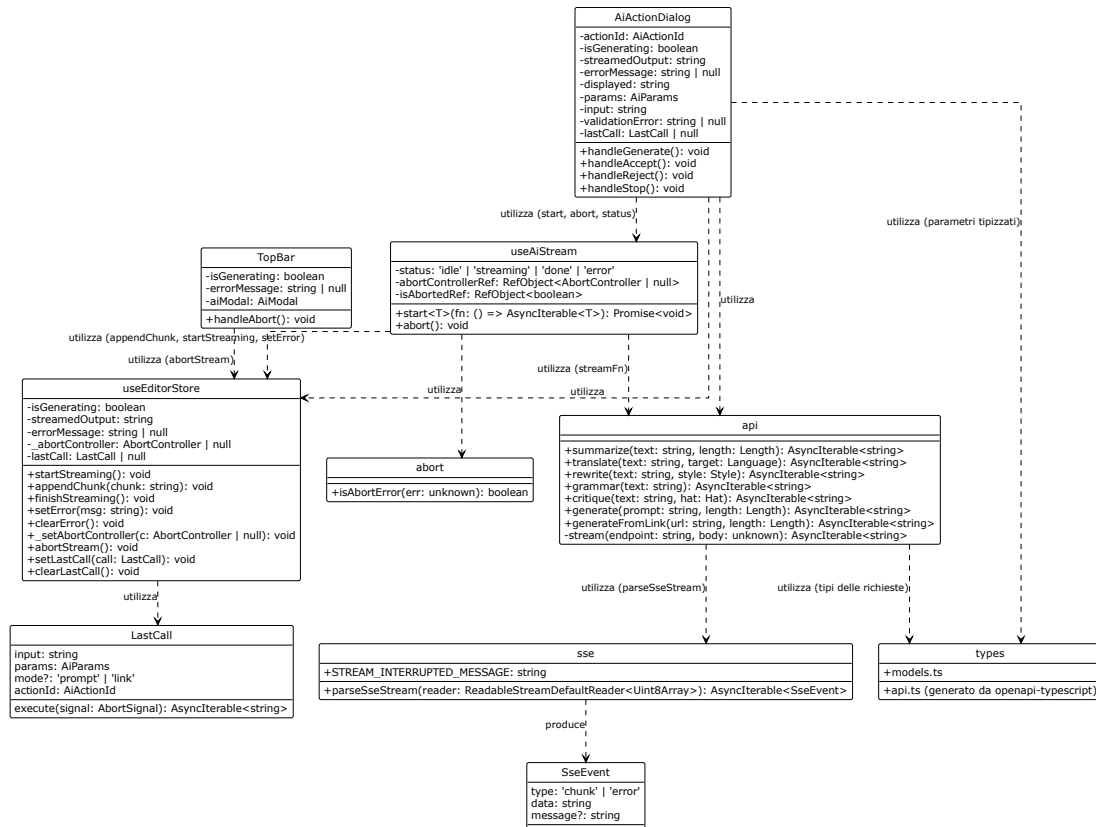


Figura 17: Diagramma UML 3.3.5 - Accesso al servizio applicativo

3.3.6 Persistenza lato client

La persistenza dei dati lato client è realizzata con due meccanismi distinti, complementari e non sovrapposti.

Persistenza automatica (Zustand persist). Come descritto in §3.3.3, `notes` utilizza il middleware `persist` di Zustand per salvare automaticamente `list` e `currentId` nel `localStorage` del browser. Il salvataggio avviene a ogni modifica dello store. Il ripristino avviene al caricamento dell'applicazione. Questo meccanismo soddisfa R-4-V-Ob (salvataggio come file di testo) e R-85-F-Ob (caricamento da file locale), garantendo la continuità della sessione.

Importazione ed esportazione (lib/fileSystem.ts). Il modulo `lib/fileSystem.ts` fornisce funzioni per l'importazione e l'esportazione delle note come file Markdown. I due rami di funzionalità:

- **Esportazione (saveNoteToFile):**
 - Ramo preferenziale: utilizza l'API `showSaveFilePicker` (Chrome/Edge).

- Ramo di fallback (Firefox): crea un link fittizio (`<a download>`) e avvia il download automatico del file `.md`. Questo ramo soddisfa R-1-V-Ob (supporto a Firefox).
- **Importazione** (`openNoteFromFile`):
 - Ramo preferenziale: utilizza l'API `showOpenFilePicker` (Chrome/Edge).
 - Ramo di fallback (Firefox): crea un `<input type="file">`, lo apre programmaticamente e legge il contenuto tramite `FileReader`. Gestisce due vie di uscita per l'annullamento: l'evento `oncancel` e il ritorno del focus alla finestra con un timer di tolleranza (`GRAZIA_ANNULLAMENTO_MS = 300ms`). Questo doppio meccanismo garantisce che la Promise non resti pendente per sempre in Firefox.
 - Restituisce una `Note` con `id` generato da `crypto.randomUUID()`, `title` derivato dal nome del file, `content` come testo letto, e `createdAt/updatedAt` al momento dell'importazione. Soddisfa R-85-F-Ob.
- **Ridenominazione** (`renameNote`): aggiorna il titolo di una nota e l'`updatedAt`. La funzione è asincrona e restituisce la nota modificata. Soddisfa R-94-F-De.

I due meccanismi non sono sovrapposti. Il middleware `persist` gestisce il salvataggio automatico e il ripristino della sessione. `fileSystem` gestisce l'operazione esplicita di importazione ed esportazione richiesta dall'utente. Questa separazione soddisfa R-1-V-Ob (supporto a Firefox) e R-4-V-Ob (salvataggio come file di testo).

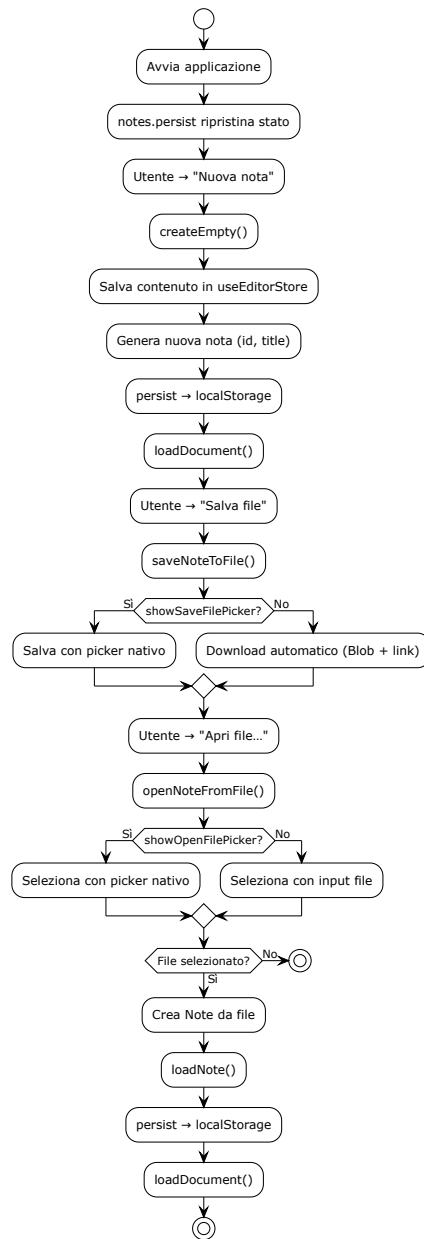


Figura 18: Diagramma UML 3.3.6 - Persistenza lato client

3.4 Architettura del backend

La presente sezione descrive l'architettura logica del servizio applicativo: i package in cui il codice è ripartito, il criterio che vi assegna un modulo, le responsabilità di ciascuna parte e le firme con cui esse collaborano. Lo stile adottato, l'architettura esagonale, è dichiarato nella sezione 3.1.2: qui se ne documenta la realizzazione. Il perimetro si arresta al confine HTTP: quanto sta oltre è nella sezione 3.3, la messa in esecuzione nella sezione 3.2, i design pattern nella sezione 3.5.

Notazione degli elenchi di firme. Il segno + contrassegna i membri pubblici, il segno - quelli che il modulo non espone all'esterno. Le funzioni di modulo, che in Python non appartengono ad alcuna classe, sono riportate senza segno. I tipi sono quelli annotati nel codice.

Diagramma delle classi del backend, limitato alle parti che ne determinano la struttura. Al centro le due porte `LLMClient` e `ContentExtractor`, ciascuna con la propria operazione e il proprio tipo di errore, i casi d'uso_G di `core/services/` e il value object `Message`. A destra i due realizzatori `LiteLLMClient` e `TavilyExtractor`, con le dipendenze verso i rispettivi fornitori esterni. A sinistra il lato primario: le rotte di `api/routes/`, i DTO di `api/schemas.py` e l'adattatore di trasporto `sse_response`. In basso il composition root `dependencies.py`, unico modulo che nomina i due realizzatori concreti. Le rotte e i casi d'uso_G sono rappresentati da un esemplare ciascuno, più `generate_from_link`, che è l'unico a dipendere da due porte. Tutte le frecce di dipendenza sono dirette verso il centro.

Figura 19: Architettura del servizio applicativo

3.4.1 Struttura a esagono e contratti di dipendenza

Il codice del servizio applicativo risiede in `api/app/` ed è ripartito in tre package e alcuni moduli di radice. Il criterio che assegna un modulo all'uno o all'altro non è la somiglianza tematica ma la **direzioe della dipendenza**: un modulo appartiene al nucleo se non ha bisogno di conoscere nulla del mondo esterno, appartiene a un lato dell'esagono se conosce un protocollo, un framework_{GG} o un fornitore.

- `app/core/`: il nucleo. Contiene i casi d'uso_G (`core/services/`), i valori e le istruzioni del dominio_{GG} (`core/domain/`) e le porte (`core/ports/`). Non importa nulla dagli altri package e nulla dalle librerie che realizzano il trasporto.
- `app/api/`: gli adattatori primari. Contiene le rotte HTTP (`api/routes/`), gli schemi di validazione_G delle richieste (`api/schemas.py`), la traduzione degli errori di validazione_G in messaggi rivolti all'utente (`api/errors.py`) e il modulo che dà forma alla risposta in streaming (`api/sse_streaming.py`). È il lato da cui l'applicazione viene invocata.
- `app/infrastructure/adapters/`: gli adattatori secondari. Contiene i **due** moduli che realizzano le porte parlando con i rispettivi fornitori. È il lato che l'applicazione invoca.
- I moduli di radice, che descrivono non una responsabilità del prodotto ma il modo in cui il processo è messo insieme: `main.py` costruisce l'applicazione e vi monta i router, `dependencies.py` è il composition root, `settings.py` dichiara lo schema della configurazione, `export_openapi.py` scrive su file il contratto dell'API_{GG}.

Nessun package prende il nome di una tecnologia: la regola di denominazione è enunciata nella sezione 3.6.3.

Verifica dei contratti di dipendenza. La direzione delle dipendenze è dichiarata in `api/.importlinter` come **quattro contratti**, verificati da `import-linter`, che ispeziona il grafo delle importazioni senza eseguire il codice. Il controllo è uno dei tre processi dell'integrazione_{GG} continua descritti nella sezione 2.6: un'importazione che invertisse la dipendenza farebbe fallire la verifica_{GG} automatica prima di arrivare alla revisione tra pari.

Tabella 7: Contratti di dipendenza dichiarati in `api/.importlinter`

Contratto	Tipo	Che cosa stabilisce
core-indipendente	<i>forbidden</i>	Il nucleo non importa il framework _{GG} web (<code>fastapi</code>), il client HTTP (<code>httpx</code>), il client del servizio di estrazione (<code>tavily</code>), la libreria di configurazione (<code>pydantic_settings</code>) né i due package degli adattatori (<code>app.api</code> , <code>app.infrastructure</code>).
layers	<i>layers</i>	I tre package sono ordinati dall'esterno verso il centro: <code>app.api</code> sopra <code>app.infrastructure</code> , <code>app.infrastructure</code> sopra <code>app.core</code> . Un livello può importare quelli sottostanti, mai quelli soprastanti.
adapter-indipendente	<i>independence</i>	I due moduli di <code>app/infrastructure/adapters/</code> (<code>litellm_client</code> , <code>tavily_extractor</code>) non si importano fra loro.
core-non-importa-settings	<i>forbidden</i>	Il nucleo non importa <code>app.settings</code> : non conosce nemmeno lo schema della configurazione del processo.

Portata del contratto layers. L'ordinamento in livelli consente a un adattatore primario di importare da `app/infrastructure/`, ma **nessun modulo di `app/api/` lo fa**: l'unico che nomina `LiteLLMClient` e `TavilyExtractor` è il composition root, che è un modulo di radice (sezione 3.4.5). Le rotte dichiarano la porta di cui hanno bisogno e la ricevono dall'esterno, quindi il margine che il contratto lascia aperto resta inutilizzato.

3.4.2 Adattatori primari

Il package `app/api/` traduce una richiesta HTTP in una chiamata al nucleo e l'esito del nucleo nella risposta verso il client. Espone **otto rotte**, ciascuna definita in un modulo proprio di `api/routes/` e montata su `main.py` come router indipendente: sette accettano un corpo in POST e realizzano altrettante funzioni assistite dal modello linguistico (`/api/summarize`, `/api/generate`, `/api/generate-from-link`, `/api/translate`, `/api/rewrite`, `/api/grammar`, `/api/critique`); l'ottava, GET `/api/constants`, non accetta corpo e pubblica nel contratto le soglie del dominio_{GG} tramite lo schema `ApiConstants`, dichiarato nel modulo della rotta stessa. A queste si aggiunge la rotta di salute GET `/`, che risiede in `main.py` perché non appartiene ad alcuna funzione del prodotto.

Forma comune delle sette rotte assistite. Ciascuna si esaurisce in tre passi e non contiene logica di dominio_{GG}. Il primo non è scritto in alcuna riga: quando l'handler viene eseguito il corpo è già stato validato, perché la validazione_G è dichiarata dal tipo del parametro. Il secondo invoca il caso d'uso, che costruisce le istruzioni e interroga la porta. Il terzo consegna il flusso all'adattatore di trasporto. La porta arriva alla rotta per iniezione della dipendenza, come valore predefinito del parametro corrispondente: **nessuna rotta nomina un adattatore concreto** e nessuna importa le istruzioni destinate al modello.

Il caso d'uso è importato con l'alias `<nome>_service` perché **il nome dell'handler non è libero**: FastAPI ne deriva l'`operationId` pubblicato in `openapi.json`, dal quale il generatore di tipi del frontend_{GG} ricava i nomi esposti al client. L'alias evita la collisione con il nome della funzione del nucleo senza toccare il contratto.

Schemi di validazione. Il modulo `api/schemas.py` contiene i sette DTO delle richieste (`TextRequest`, `GenerateRequest`, `LinkRequest`, `TranslateRequest`, `RewriteRequest`, `GrammarRequest`, `CritiqueRequest`) e lo schema `ErrorResponse`, unica forma di ogni risposta di errore dell'API_{GG}. I vincoli_G di lunghezza e i vocabolari ammessi sono importati da `core/domain/values.py`: il confine dichiara *che* un campo è vincolato_G, non *quanto*, così che la soglia non esista in due copie destinate a divergere. Accanto ai DTO sta `FIELD_LABELS`, che associa a ciascun campo la propria etichetta in italiano, perché aggiungere un campo e la sua etichetta sia la stessa modifica; un campo assente dalla mappa produce un messaggio generico anziché tecnico.

Traduzione degli errori di validazione. Il modulo `api/errors.py` traduce un errore di validazione_G di Pydantic in una frase di linguaggio naturale, come impone il requisito_G **R-110-F-Ob**. La forma predefinita di FastAPI espone i campi tecnici `type`, `loc` e `ctx` e rimanda al mittente il campo `input`, cioè il testo che l'utente ha scritto: **nessuno dei quattro raggiunge la risposta**. La traduzione copre le classi di errore che l'applicazione può produrre — campo mancante, testo troppo corto o troppo lungo, valore fuori dal vocabolario ammesso, indirizzo malformato, corpo non in formato JSON — e per ciascuna compone una frase con causa e azione correttiva. Davanti a un valore fuori vocabolario **non elenca i valori ammessi**: quell'elenco è una struttura interna di Pydantic e arriva in lingua inglese, mentre l'interfaccia propone già all'utente le sole opzioni valide.

Forma unica delle risposte di errore. Il modulo `main.py` registra due gestori che riconducono ogni esito negativo allo schema `ErrorResponse`: uno per le eccezioni HTTP sollevate dalle rotte, uno per gli errori di validazione_G, restituiti con stato 422. Alle sette rotte con corpo è associata la dichiarazione esplicita che la risposta 422 ha forma `ErrorResponse`: senza di essa il contratto pubblicherebbe lo schema generato in automatico da FastAPI, che descrive `detail` come sequenza di oggetti, forma che l'applicazione non produce.

L'adattatore di trasporto. La funzione `sse_response`, in `api/sse_streaming.py`, converte il flusso di frammenti prodotto dal caso d'uso in una risposta HTTP progressiva. Risiede fra gli adattatori primari perché **non realizza alcuna porta**, non parla con alcun servizio esterno e non viene mai invocata dal dominio_{GG}, ma dalle sette rotte assistite. Traduce l'esito prodotto dal nucleo nel protocollo della risposta verso il client, che è lavoro del lato da cui l'applicazione viene invocata. Che conosca HTTP e il formato degli eventi sul filo non la colloca altrove, perché anche le rotte li conoscono. Il formato degli eventi è nella sezione 4.1.

Alla firma appartiene una scelta che va motivata qui: la funzione **estrae il primo frammento prima di restituire la risposta**. Il server invia l'intestazione di stato prima di iniziare a iterare il flusso, e dopo quel momento non è più possibile rispondere con uno stato di errore; il prelievo anticipato intercetta quindi un guasto del fornitore mentre lo stato è ancora modificabile e lo traduce in 503 (UC72). Un guasto che sopraggiunga a metà flusso richiede un trattamento diverso (sezione 4.5). La funzione `verificaG` inoltre a ogni frammento se il client si è disconnesso e in tal caso chiude il flusso: è il tratto terminale della propagazione dell'annullamento trattata nella sezione 3.6.1.

Firme.

- `summarize(payload: TextRequest, request: Request, client: LLMClient): StreamingResponse`: e con la medesima forma `generate`, `translate`, `rewrite`, `grammar` e

`critique`, ciascuna con il proprio DTO. Il parametro `client` è dichiarato con `Depends(get_llm_client)`.

- `generate_from_link(payload: LinkRequest, request: Request, client: LLMClient, extractor: ContentExtractor): StreamingResponse`: unica rotta a dichiarare due porte fra i propri parametri; il flusso che ne deriva è descritto nella sezione 4.3.
- `constants(): ApiConstants`: pubblica nel contratto il sentinella della correzione grammaticale e le quattro soglie di lunghezza, tipizzate come letterali così che compaiano nello schema come costanti e non come stringhe o interi generici.
- `describe_validation_error(error: dict): str`: traduce un singolo errore di validazione_G nella frase rivolta all'utente.
- `sse_response(request: Request, stream: AsyncGenerator[str, None], log_label: str, logger: Logger | None): StreamingResponse`: adattatore di trasporto.

3.4.3 Il nucleo

Il package `app/core/` contiene ciò che il prodotto fa, separato da ogni conoscenza di come venga invocato e di chi esegua per suo conto le operazioni verso l'esterno. Si articola in tre parti: i **casi d'uso** in `core/services/`, il **dominio_G** in `core/domain/` e le **porte** in `core/ports/`.

Forma comune dei sette casi d'uso. La forma è stata fissata dal primo dei sette, il riassunto (UC62), e i sei successivi vi si attengono.

- **Una funzione, non una classe.** L'unica dipendenza del caso d'uso è la porta, e la porta è un parametro. Una classe con un costruttore e un solo metodo di esecuzione offrirebbe la stessa iniezione e la stessa provabilità in più righe.
- **Un modulo per caso d'uso, chiamato come l'operazione.** Il modulo `summarize.py` espone la funzione `summarize`.
- **La porta è l'ultimo parametro**, dopo i dati dell'operazione: riassumi *questo testo*, di *questa lunghezza*, tramite *questa porta*.
- **Il tipo del parametro è la porta, mai l'adattatore.** Il caso d'uso non sa se dall'altra parte vi sia il gateway dei modelli o un doppio predisposto per un test_{GG}. Da questa regola discende la provabilità del dominio_{GG} in isolamento dichiarata nella sezione 3.1.2, e il contratto `layers` ne impedisce la violazione.
- **Il caso d'uso restituisce il flusso della porta senza consumarlo.** Chi lo invoca riceve un flusso che non ha ancora prodotto nulla e non ha contattato nessuno. Ne discendono due comportamenti visibili all'utente: un guasto all'apertura resta traducibile in uno stato HTTP (sezione 3.4.2) e il testo compare progressivamente. Ne discende anche che **l'eccezione del fornitore non viene sollevata quando il caso d'uso è chiamato**, ma alla prima iterazione del flusso restituito.

L'unica eccezione alla forma. La generazione a partire da un collegamento è il solo caso d'uso a dipendere da **due** porte e il solo dichiarato `async def`: gli altri sei sono funzioni sincrone. Nessuno dei sette è scritto come generatore con `yield`, perché il corpo di un generatore non viene

eseguito finché qualcuno non ne consuma il flusso; qui la differenza conta, dato che l'estrazione del contenuto della pagina deve avvenire all'atto della chiamata per restare traducibile in uno stato HTTP (sezione 4.3).

I valori del dominio_G. Il modulo `core/domain/values.py` raccoglie ciò che le funzioni assistite condividono: i vocabolari ammessi (le tre lunghezze, le quattro lingue di destinazione della traduzione, i tre registri della riscrittura, i sei cappelli dell'analisi critica), le soglie di lunghezza minima e massima del testo e delle istruzioni, e il sentinella con cui la correzione grammaticale segnala di non aver trovato errori. Sono dichiarati come tipi letterali: il valore ammesso e il tipo che lo descrive sono la stessa dichiarazione, quindi non è possibile cambiarne uno senza l'altro.

Il modulo contiene inoltre il value object `Message`, immutabile, con i due campi `role` e `content`: è il tipo che porte e istruzioni si scambiano. Il modello del dominio_{GG} si ferma qui, senza un'entità per la conversazione intera, senza un tipo enumerato per il ruolo e senza involucri attorno ai frammenti in uscita, dove la stringa è già il tipo giusto.

La composizione delle istruzioni destinate al modello. Il package `core/domain/prompts/` costruisce le istruzioni ed è dominio_{GG}: che cosa il modello possa dire e in che forma debba scriverlo è una decisione di prodotto, non una proprietà del fornitore. Quattro moduli con quattro compiti distinti:

- `rules.py` dichiara ogni regola **una volta sola**, come costante nominata, e ne consente il riuso fra istruzioni diverse.
- `composer.py` mette in fila le parti — il ruolo attribuito al modello, le regole di contenuto, le regole di forma e, dove l'utente può sceglierla, la lunghezza richiesta — e ne ricava i due messaggi da inviare. **Non è il pattern Builder**: la sezione 3.5.5 argomenta perché.
- `templates.py` contiene i **sette costruttori**, uno per funzione assistita, dai quali risultano dodici istruzioni distinte: l'analisi critica ne produce una per ciascuno dei sei cappelli, le altre sei operazioni una ciascuna. La distinzione fra le dodici è verificata_G da un test_{GG} dedicato. Nel modulo resta ciò che è proprio della singola operazione; **una regola che comparisse in due costruttori appartiene a `rules.py`**.
- `untrusted.py` contiene la sola trasformazione che agisce sul contenuto di terze parti.

Confinamento del contenuto non fidato. Per ogni operazione vale la regola che **il testo dell'utente finisce esclusivamente nel messaggio a lui destinato**, mai fra le istruzioni di prodotto; i test_{GG} la verificano_G su tutti e sette i costruttori. Alla sola generazione a partire da un collegamento si aggiunge il confinamento del materiale estratto: il contenuto viene racchiuso fra due marcatori, e i marcatori che comparissero *dentro* di esso vengono resi inerti, altrimenti una pagina potrebbe chiudere il recinto per conto proprio e il testo successivo si presenterebbe come istruzione. La sostituzione avviene carattere per carattere e non allunga il testo, perché il troncamento alla lunghezza massima avviene *prima* della composizione.

Il confine della mitigazione è più stretto di quanto sembri, e va dichiarato: il confronto sui marcatori è esatto, quindi una variante scritta in minuscolo o spezzata da spazi attraversa il contenuto intatta e, pur non aprendo nulla, un modello linguistico potrebbe leggerla come una chiusura. Anche la regola che dichiara non autorevole il contenuto estratto **dice al modello di ignorare le istruzioni che vi comparissero, non gli impedisce di obbedirvi**.

Le due porte. Sono dichiarate in `core/ports/` come classi astratte, ciascuna con una sola operazione e con il tipo di errore che essa può sollevare. L'errore sta con la porta perché **fa parte del contratto**: chi la consuma deve poterlo intercettare senza sapere quale implementazione lo produca, e se visse nell'adattatore il contratto `layers` impedirebbe all'adattatore di trasporto di importarlo. Nessuna delle due porte dichiara invece un'operazione di chiusura, benché entrambi i realizzatori possiedano risorse da rilasciare: il ciclo di vita di quelle risorse è un fatto dell'adattatore, e dichiararlo nella porta obbligherebbe ogni doppio predisposto per un `testGG` a implementare un metodo che non ha nulla da chiudere.

Firme.

- `+stream(messages: Sequence[Message]): AsyncIterator[str]`: unica operazione della porta `LLMClient`. Solleva `LLMProviderError`. Il parametro è una sequenza di `Message` e non di dizionari: la forma di filo del fornitore sta oltre la porta. È dichiarato come sequenza e non come lista perché la porta legge e non modifica.
- `+extract(url: str): str`: unica operazione della porta `ContentExtractor`. Solleva `ContentExtractorError`. La porta **non promette alcun limite di lunghezza** sul risultato: il troncamento è politica del realizzatore, e dichiararlo qui significherebbe vincolare_G a rispettarla anche i doppi dei `testGG`.
- `summarize(text: str, length: Length, llm: LLMClient): AsyncIterator[str]`
- `generate(prompt: str, length: Length, llm: LLMClient): AsyncIterator[str]`
- `translate(text: str, target_language: Language, llm: LLMClient): AsyncIterator[str]`
- `rewrite(text: str, style: Style, llm: LLMClient): AsyncIterator[str]`
- `grammar(text: str, llm: LLMClient): AsyncIterator[str]`: unico dei sette a non impiegare alcun vocabolario del dominio_{GG} oltre al testo, perché la correzione non ha parametri da scegliere.
- `critique(text: str, hat: Hat, llm: LLMClient): AsyncIterator[str]`
- `generate_from_link(url: str, length: Length, extractor: ContentExtractor, llm: LLMClient): AsyncIterator[str]`: le due porte sono gli ultimi parametri, nell'ordine in cui vengono impiegate. Solleva `InvalidLinkError` e `FetchError`, dove il primo tipo è sottotipo del secondo: i due fallimenti hanno esiti HTTP distinti (collegamento scartato senza toccare la rete, estrazione fallita), ma chi non volesse distinguerli può intercettare il solo tipo generale.
- `validate_link(url: str): None`: scarta il collegamento inutilizzabile prima di qualunque richiesta di rete.
- `fetch_and_extract(url: str, extractor: ContentExtractor): str`: interroga la porta di estrazione e riconduce il suo errore al tipo del caso d'uso, troncando il risultato alla lunghezza massima ammessa dal dominio_{GG}.
- `compose(role: str, user: str, content_rules: Sequence[str], form_rules: Sequence[str], content_heading: str, length: Length | None): list[Message]`: solleva `ValueError` se il ruolo o il testo dell'utente sono vuoti, che sono i due elementi senza i quali l'istruzione non è una richiesta.
- `wrap_extracted_content(content: str): str`: `str`: neutralizza i marcatori e racchiude fra essi il contenuto estratto.

3.4.4 Adattatori secondari

Il package `app/infrastructure/adapters/` contiene ciò che il nucleo invoca per parlare con il mondo. Ciascuna delle due porte ha **un solo realizzatore concreto**, responsabile_G di due traduzioni: dalla chiamata del dominio_{GG} al protocollo del proprio fornitore, e dagli errori del fornitore al tipo di errore che la porta dichiara. Sono le due traduzioni che consentono al nucleo di ignorare l'esistenza dei fornitori. Entrambi i moduli realizzano il pattern definito nella sezione 3.5.1; la provenienza delle chiavi di accesso è nella sezione 3.2.1.

LiteLLMClient, realizzatore di LLMClient. Possiede un client HTTP asincrono configurato con l'indirizzo del gateway, un tempo massimo di attesa e l'intestazione di autorizzazione. L'operazione di flusso apre una richiesta di completamento chiedendo la risposta progressiva, la legge riga per riga man mano che arriva e restituisce i frammenti di testo che ne estrae. È **l'unico punto del backend in cui un Message assume la forma di filo del fornitore**: le chiavi con cui ruolo e contenuto viaggiano sulla rete sono protocollo, e la traduzione occupa una riga sola.

La conversione degli errori riconduce a `LLMProviderError` quattro condizioni distinte: gateway irraggiungibile, tempo massimo scaduto, risposta con stato di errore e risposta malformata. **Nessuna eccezione del client HTTP attraversa il confine dell'adattatore**, perché chi consuma la porta dovrebbe altrimenti conoscere la libreria di trasporto per trattarla.

TavilyExtractor, realizzatore di ContentExtractor. Restituisce il contenuto testuale di una pagina, già ripulito dal marcatore e dagli elementi accessori, troncato a una lunghezza massima. Il troncamento è **politica dell'adattatore e non del contratto**: la porta non promette alcun limite, perché nessun altro realizzatore sarebbe tenuto a rispettarlo. Ogni fallimento — rete, permessi, pagina inesistente, pagina priva di testo — viene ricondotto a `ContentExtractorError`.

Una proprietà dell'adattatore non è visibile nella firma e va documentata: **il client del fornitore compie l'operazione di rete in modo sincrono**, e invocarlo direttamente da una funzione asincrona bloccherebbe il ciclo di eventi per l'intera durata della richiesta, fermando ogni altra richiesta in corso **compresi i flussi già aperti**. L'adattatore lo esegue perciò su un thread separato.

Ciclo di vita e configurazione. Entrambi gli adattatori possiedono un pool di connessioni che vive quanto il processo ed espongono un'operazione di chiusura asincrona, invocata alla terminazione. La simmetria delle due firme è voluta anche dove non sarebbe necessaria: la chiusura di `TavilyExtractor` non compie operazioni di rete e potrebbe essere sincrona, ma è dichiarata asincrona perché chi la invoca non debba distinguere i due casi. Nessuno dei due, inoltre, legge la configurazione del processo: la riceve dal costruttore, perché leggerla all'interno legherebbe una classe di infrastruttura alla configurazione globale e renderebbe impossibile istanziarla in un `testGG` senza alterare l'ambiente di esecuzione.

Indipendenza reciproca degli adattatori. Il contratto `adapter-indipendente` vieta a ciascuno dei due moduli del package di importare l'altro: i due fornitori sono indipendenti, e un adattatore che ne chiamasse un altro introdurrebbe fra servizi esterni una dipendenza che nessuna porta dichiara. Il package contiene i soli realizzatori delle porte; l'adattatore che dà forma alla risposta in streaming non ne fa parte, per la ragione riportata nella sezione 3.4.2.

Firme.

- `+LiteLLMClient(settings: Settings)`: costruisce il client HTTP con indirizzo, tempo massimo e intestazione di autorizzazione ricavati dalla configurazione ricevuta.
- `+stream(messages: Sequence[Message]): AsyncIterator[str]`: realizzazione dell'operazione dichiarata dalla porta `LLMClient`.
- `-_parse_sse_line(line: str): str | None`: operazione statica; estrae il frammento di testo da una riga della risposta del gateway e restituisce il valore nullo per le righe da ignorare. Solleva `LLMProviderError` sulle righe malformate.
- `+aclose(): None`: chiude il client HTTP sottostante.
- `+TavilyExtractor(api_key: str)`: solleva `ContentExtractorError` se la chiave ricevuta è vuota.
- `+extract(url: str): str`: realizzazione dell'operazione dichiarata dalla porta `ContentExtractor`.
- `+aclose(): None`: smonta il pool di connessioni del client del fornitore.

3.4.5 Composition root e configurazione

Il modulo `app/dependencies.py` è il composition root del servizio applicativo: **l'unico modulo che nomina `LiteLLMClient` e `TavilyExtractor` fuori dai moduli che li definiscono**. Da qui discende quanto affermato nella sezione 3.1.2: sostituire un fornitore esterno comporta la scrittura di un nuovo adattatore e la modifica di questo solo modulo. Il meccanismo con cui le porte raggiungono le rotte è il pattern definito nella sezione 3.5.4.

Non esistono provider dei casi d'uso. I casi d'uso_G sono funzioni la cui unica dipendenza è un parametro: non c'è nulla da costruire, perché la rotta si fa iniettare la porta e la passa alla funzione. Un provider per ciascun caso d'uso sarebbe un livello di indirectione che non inietterebbe nulla che la rotta non abbia già ricevuto.

Tre provider, una istanza per processo. I provider sono la configurazione, l'adattatore del modello linguistico e l'adattatore di estrazione, ciascuno memoizzato. Poiché nessuno riceve argomenti, la memoizzazione ha una chiave sola: il corpo viene eseguito alla prima chiamata e ogni chiamata successiva restituisce l'oggetto già costruito. Ne risulta **una istanza per processo**, esito voluto per due componenti che possiedono ciascuna un pool di connessioni. Che questa costruzione **non** sia il Singleton della classificazione di Gamma e altri è argomentato nella sezione 3.5.5. Il provider della configurazione, a differenza degli altri due, non è iniettato nelle rotte: a invocarlo sono la costruzione dell'applicazione e i provider stessi, e i test_{GG} lo governano alterando le variabili d'ambiente e azzerandone la memoizzazione.

Le due chiusure interrogano la memoizzazione. Alla terminazione vanno rilasciate le risorse dei soli adattatori effettivamente costruiti: le due funzioni di chiusura verificano perciò se la memoizzazione contenga già un'istanza e, in caso contrario, non fanno nulla. Non è un risparmio di lavoro. Entrambi i provider rifiutano di costruire l'adattatore quando la chiave corrispondente manca, quindi invocarli in chiusura farebbe fallire l'uscita del processo in ogni ambiente privo di configurazione, a cominciare da quello dei test_{GG}.

Le due funzioni risiedono qui e non fra le operazioni di avvio e terminazione (sezione 3.2.1) perché la strategia di memoizzazione è una scelta di questo modulo. Per la stessa ragione vi risiede

il controllo delle chiavi obbligatorie: **quale chiave serve a quale provider è conoscenza del composition root**. L'asimmetria fra le due chiavi è descritta nella sezione 3.2.1.

Due deroghe documentate. Due punti del modulo non discendono da un principio ma da una circostanza.

Il primo: entrambi i provider, quando la chiave manca, rispondono che il servizio è temporaneamente indisponibile anziché segnalare un errore di programmazione, perché una configurazione incompleta non è un difetto del codice. In un processo avviato regolarmente **quella guardia non può scattare**, perché il controllo delle chiavi obbligatorie avrebbe già impedito l'avvio; resta come difesa in profondità, dato che l'applicazione è costruibile anche senza attraversare la procedura di avvio, come fanno i `testGG` e l'esportazione del contratto dell'`APIGG` in `integrazioneGG` continua.

Il secondo: il messaggio con cui il controllo delle chiavi rifiuta l'avvio **nomina per esteso la variabile d'ambiente mancante**, l'opposto di quanto il requisito_G **R-110-F-Ob** impone alle risposte HTTP. La divergenza è voluta, perché quel requisito_G regola i messaggi rivolti all'utente, mentre il destinatario di questo è chi mette in esercizio il sistema_{GG} (sezione 3.6.3).

Che cosa è la configurazione e come la si ottiene. Il modulo `app/settings.py` dichiara **che cosa** sia la configurazione: la classe `Settings` elenca l'indirizzo del gateway dei modelli, l'identificativo del modello, le due chiavi di accesso e le origini ammesse dalla politica CORS, ciascuna con il proprio valore predefinito, e le popola dalle variabili d'ambiente. Il modulo `app/dependencies.py` dichiara **come** la si ottenga, perché è compito di un provider. La separazione ha una conseguenza concreta: vincolare_G la validità della configurazione allo schema che la descrive, imponendo per esempio che una chiave non sia vuota, renderebbe impossibile costruire l'applicazione negli ambienti privi di quelle chiavi, e l'esportazione del contratto dell'`APIGG` in `integrazioneGG` continua fallirebbe prima di arrivare ai `testGG`.

Firme.

- `get_settings()`: `Settings`: provider memoizzato della configurazione del processo.
- `get_llm_client()`: `LLMClient`: provider memoizzato dell'adattatore del modello linguistico. Il tipo restituito è **la porta**, non il realizzatore.
- `get_content_extractor()`: `ContentExtractor`: provider memoizzato dell'adattatore di estrazione, con la medesima proprietà sul tipo restituito.
- `verifica_chiavi_obbligatorie()`: `None`: solleva un errore di esecuzione, con l'elenco completo delle variabili d'ambiente assenti o vuote, se ne manca una senza la quale il processo non è in grado di servire alcuna richiesta.
- `close_llm_client()`: `None` e `close_content_extractor()`: `None`: rilasciano le risorse dell'adattatore corrispondente, se ne è stato costruito uno.
- `Settings`: attributi `+litellm_base_url: str, +litellm_model: str, +litellm_api_key: str, +tavily_api_key: str, +cors_origins: list[str]`.

3.5 Design pattern adottati

Un pattern figura in questa sezione se e solo se sono vere tre condizioni insieme: la struttura che lo realizza è presente nel codice; è stata scelta dal gruppo, e non ricevuta dall'uso ordinario di una libreria o di un framework_{GG}; esisteva un problema del prodotto che senza quella scelta sarebbe rimasto irrisolto. La seconda condizione è quella che esclude di più, ed è deliberato: impiegare

una libreria che al proprio interno realizza un pattern non equivale ad adottare quel pattern. Chi importa una libreria di gestione dello stato non ha progettato un Observer, così come chi dichiara una rotta non ha progettato un Command. Documentare quelle strutture come scelte di progettazione significherebbe attribuirsi decisioni prese da altri.

L'applicazione del criterio ha prodotto quattro pattern documentati (Object Adapter, Facade, Observer e Dependency Injection) e quattro valutati e scartati, riportati con la relativa motivazione nella sottosezione 3.5.5. Le strutture escluse non sono elencate per completezza formale: sono i candidati che un lettore si aspetterebbe di trovare, e per ciascuno è dichiarata la condizione che non risulta soddisfatta.

Una distinzione da fissare: adattatore architetturale e Object Adapter Il capitolo impiega il termine *adattatore* in due accezioni che non coincidono, e confonderle è l'errore più frequente su questo tema. Nel vocabolario architetturale fissato nella sezione 3.1.4 un adattatore è un modulo che sta sul bordo dell'esagono: primario se traduce una richiesta esterna in una chiamata al dominio_{GG}, secondario se realizza una porta per conto del dominio_{GG}. Nel catalogo dei design pattern, invece, l'Object Adapter è una struttura precisa, che presuppone due interfacce preesistenti e incompatibili da far collaborare.

Ne segue che gli adattatori primari descritti nella sezione 3.4.2 non sono Object Adapter: non esiste, dal lato in ingresso, un'interfaccia preesistente che il dominio_{GG} non sappia consumare; il framework_{GG} web consegna una richiesta già decodificata, e la rotta la inoltra a un caso d'uso scritto dal gruppo. Sono adattatori in senso architetturale e nient'altro. Lo sono invece, e a pieno titolo, i soli adattatori secondari, che è ciò di cui tratta la sottosezione seguente.

3.5.1 Object Adapter

Definizione L'Adapter converte l'interfaccia di una classe esistente in un'altra interfaccia, quella che il client si attende, permettendo la collaborazione fra due parti che per firma o per protocollo non potrebbero comunicare. Nella variante a oggetti, che è quella qui adottata, l'adattatore incapsula per composizione l'oggetto da adattare e ne inoltra le chiamate; nella variante a classi ne erediterebbe. La differenza non è di stile: la composizione consente di adattare un oggetto la cui classe non è modificabile, come è il caso dei client forniti da terzi.

Problema risolto Il nucleo del prodotto ha bisogno di due capacità che non possiede: ottenere testo prodotto in modo incrementale_G da un modello linguistico e ottenere il contenuto testuale di una pagina web. Entrambe sono fornite da librerie di terzi, le cui interfacce sono incompatibili con ciò che il dominio_{GG} dichiara di volere: il client del gateway parla il formato di filo del fornitore, con le proprie chiavi e i propri codici di stato, mentre il dominio_{GG} parla di messaggi e di frammenti di testo; il client del servizio di estrazione, per parte sua, è sincro, e invocarlo direttamente da una funzione asincrona bloccherebbe l'intero processo per la durata della chiamata di rete, arrestando anche gli streaming già aperti verso altri utenti.

Senza l'adattamento, l'incompatibilità si risolverebbe nell'unico modo rimasto: portando la conoscenza del fornitore dentro il dominio_{GG}. I casi d'uso_G conoscerebbero il formato di filo, i codici di errore e la natura sincra di una delle due librerie, e cambiare fornitore diventerebbe una modifica diffusa anziché circoscritta.

Implementazione I due adattatori descritti nella sezione 3.4.4 realizzano ciascuno una delle due porte dichiarate dal nucleo e incapsulano per composizione il client del proprio fornitore: nessuno dei due eredita dalla classe adattata, che appartiene a una libreria di terzi. Ciascuno svolge tre traduzioni: dalla forma del dominio_{GG} a quella del protocollo del fornitore, dal risultato

del fornitore alla forma attesa dalla porta e, soprattutto, dagli errori del fornitore al tipo di errore dichiarato dalla porta. Quest'ultima è la traduzione che rende l'adattamento completo: senza di essa il chiamante dovrebbe conoscere le eccezioni della libreria sottostante, e la porta non sarebbe un contratto ma una firma. Nel caso dell'estrazione l'adattamento riguarda anche il modello di esecuzione, poiché una chiamata sincrona è trasposta su un thread separato per non arrestare il ciclo degli eventi.

Vale la pena riportare come la seconda porta sia nata, perché mostra che il pattern è stato *applicato* e non ricevuto per caso. L'astrazione verso il modello linguistico esisteva fin dalla prima versione del repository_{GG} di prodotto, mentre l'estrazione era affidata a un modulo che costruiva e invocava direttamente il client del fornitore. L'asimmetria è stata rilevata e sanata con un intervento dedicato, il cui unico scopo era portare l'estrazione al medesimo trattamento: da lì nascono la porta, l'adattatore corrispondente e l'iniezione nella rotta. La sezione 2.3 ne riporta la conseguenza dal lato delle tecnologie.

Vantaggi La conoscenza di ciascun fornitore è confinata in un solo modulo, e sostituirlo comporta la scrittura di un nuovo adattatore e la modifica del solo modulo di composizione, come descritto nella sezione 3.4.5. Il nucleo resta scritto nei termini del prodotto e non in quelli di una libreria, e i contratti di dipendenza verificati staticamente, descritti nella sezione 3.4.1, impediscono che quella conoscenza rientri nel dominio_{GG} per disattenzione. Infine, poiché il tipo dichiarato è la porta e non l'adattatore, ogni consumatore può essere esercitato con un doppio: è il vantaggio ripreso nella sottosezione 3.5.4.

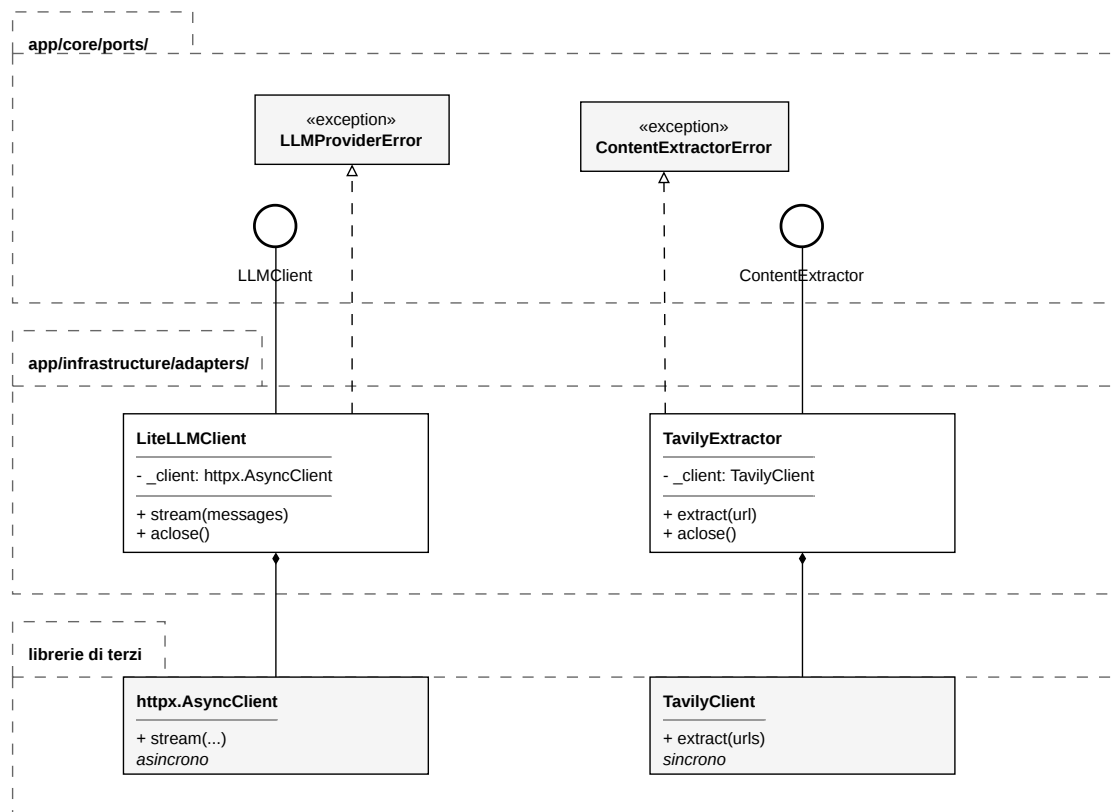


Figura 20: Diagramma delle classi dell'Object Adapter

3.5.2 Facade

Definizione La Facade fornisce un'interfaccia unificata e di livello più alto verso un insieme di interfacce di un sottosistema, rendendone più semplice l'uso. Non elimina il sottosistema né ne vieta l'accesso diretto: ne offre una vista ridotta, tagliata sui casi d'uso_G effettivi dei suoi clienti.

Problema risolto Contattare il servizio applicativo non è, per l'applicazione web, un'operazione elementare. Richiede di comporre l'indirizzo della rotta a partire dall'indirizzo di base descritto nella sezione 3.2.1, di costruire la richiesta con il metodo, le intestazioni e il corpo serializzato, di distinguere una risposta riuscita da una fallita interpretandone lo stato ed estraendone il messaggio destinato all'utente, e infine di decodificare un flusso che non arriva come un valore unico ma come una sequenza di eventi da riassemblare.

Senza una facciata, ciascuno di questi passaggi ricadrebbe sul componente che invoca l'operazione. La conseguenza non sarebbe soltanto la duplicazione: la conoscenza del protocollo si spargerebbe nell'albero dei componenti, e una modifica del contratto (una rotta rinominata, un formato d'errore cambiato) richiederebbe di intervenire in tutti i punti che la posseggono, con la certezza di dimenticarne qualcuno.

Implementazione Il modulo descritto nella sezione 3.3.5 espone una funzione per ciascuna delle sette operazioni assistite, ciascuna con i soli parametri di dominio_{GG} dell'operazione e un unico tipo di ritorno: una sequenza asincrona di frammenti testuali. Tutto il resto (indirizzo,

metodo, intestazioni, serializzazione, lettura dello stato della risposta, estrazione del messaggio d'errore, decodifica del flusso e conversione dell'evento terminale di errore in un fallimento) vive dietro quella superficie. Il modulo si dichiara facciata nel proprio commento d'apertura, ed è quindi una scelta esplicita e non una lettura a posteriori.

Vantaggi I componenti dell'interfaccia esprimono *che cosa* chiedono e non *come* lo si ottiene: invocano un'operazione e consumano frammenti. Il protocollo è conosciuto in un solo punto dell'applicazione web, ed è lo stesso punto che impiega i tipi derivati dal contratto pubblicato dal servizio applicativo, come descritto nella sezione 3.6.2. Va osservato, per non attribuire alla facciata un merito che oggi non ha, che i suoi clienti sono uno solo: il componente che presenta le operazioni assistite. Il beneficio realizzato non è quindi la mancata duplicazione fra molti chiamanti, ma la separazione fra la conoscenza del protocollo e la presentazione, che resta valida con un cliente e diventa determinante quando se ne aggiunge un secondo.

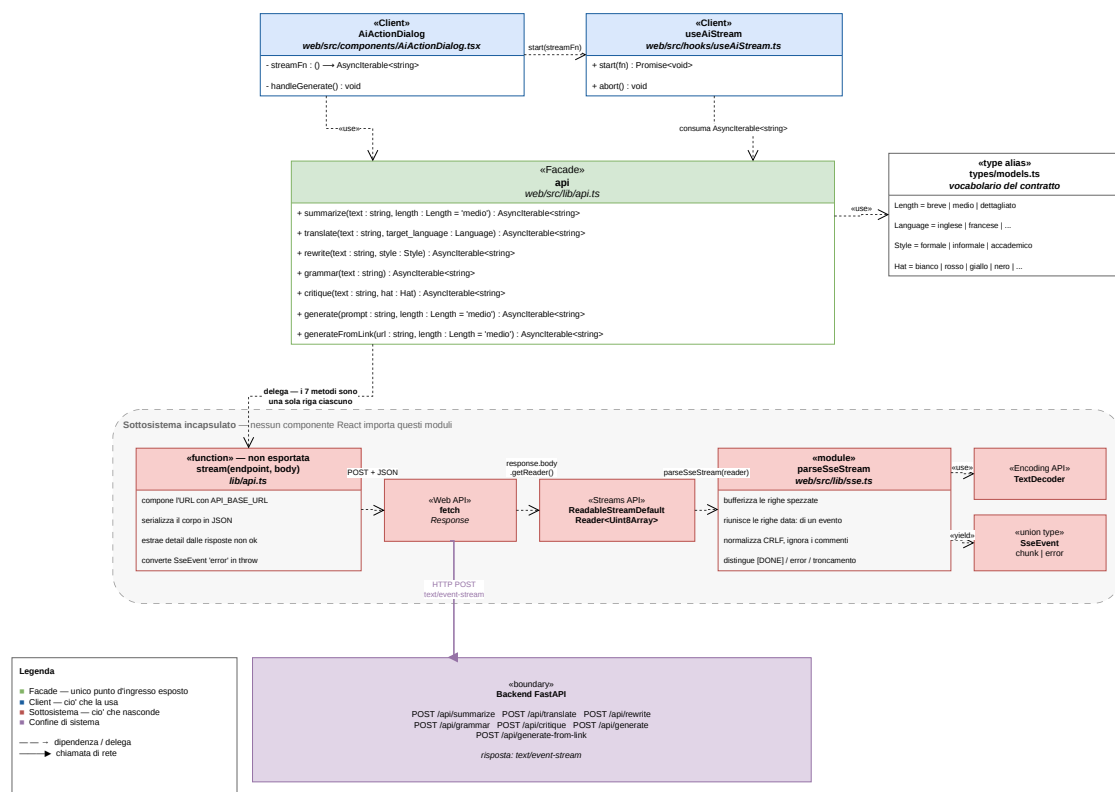


Figura 21: Diagramma della facciata di accesso al servizio applicativo

3.5.3 Observer

Definizione L'Observer stabilisce una dipendenza uno-a-molti fra oggetti, tale che al mutare dello stato del soggetto tutti i suoi osservatori siano notificati e aggiornati automaticamente. Il soggetto non conosce l'identità degli osservatori, che si registrano e si cancellano autonomamente.

Problema risolto Va dichiarato con precisione che cosa, qui, sia scelta del gruppo, perché la sezione perderebbe credibilità se rivendicasse più del dovuto. La libreria di gestione dello stato realizza Observer al proprio interno, e adottarla non è di per sé una decisione di progettazione: quel meccanismo si sarebbe ottenuto con qualunque libreria della stessa famiglia. La scelta del gruppo è un'altra, e riguarda la granularità della sottoscrizione.

Il problema si osserva durante lo streaming. La porzione di stato che raccoglie l'esito in arrivo dal servizio applicativo cambia molte volte al secondo, poiché ogni frammento ricevuto la aggiorna. Se i componenti si sottoscrivessero allo stato nel suo complesso, o a un oggetto che ne aggregi più porzioni, ogni frammento provocherebbe il ridisegno di tutta l'area di lavoro: l'editor verrebbe ricostruito decine di volte al secondo, con la perdita della posizione del cursore e dello scorrimento mentre l'utente osserva il testo comparire.

Implementazione I due store descritti nella sezione 3.3.3 espongono un selettore per ciascuna porzione osservabile, e il codice enuncia la regola come vincolo_{GG} e non come consiglio: non si espongono mai porzioni composite. Un componente dichiara la porzione che gli serve e viene notificato solo quando quella cambia. La ripartizione che ne risulta è la ragione per cui lo streaming non disturba la scrittura: il componente che presenta l'esito in arrivo (la finestra di dialogo dell'operazione assistita, non il pannello di anteprima) è sottoscritto alla sola porzione che raccoglie i frammenti, mentre l'editor è sottoscritto a quella che contiene il testo della nota, che durante l'elaborazione non cambia.

Vantaggi Il costo del ridisegno è proporzionale a ciò che è effettivamente cambiato, non alla dimensione dello stato dell'applicazione. L'editor conserva cursore e posizione di scorrimento durante l'intera elaborazione, perché non viene toccato. Il beneficio è verificabile leggendo a quale selettore ciascun componente si sottoscrive, e non richiede misurazioni: è una proprietà della struttura delle sottoscrizioni.

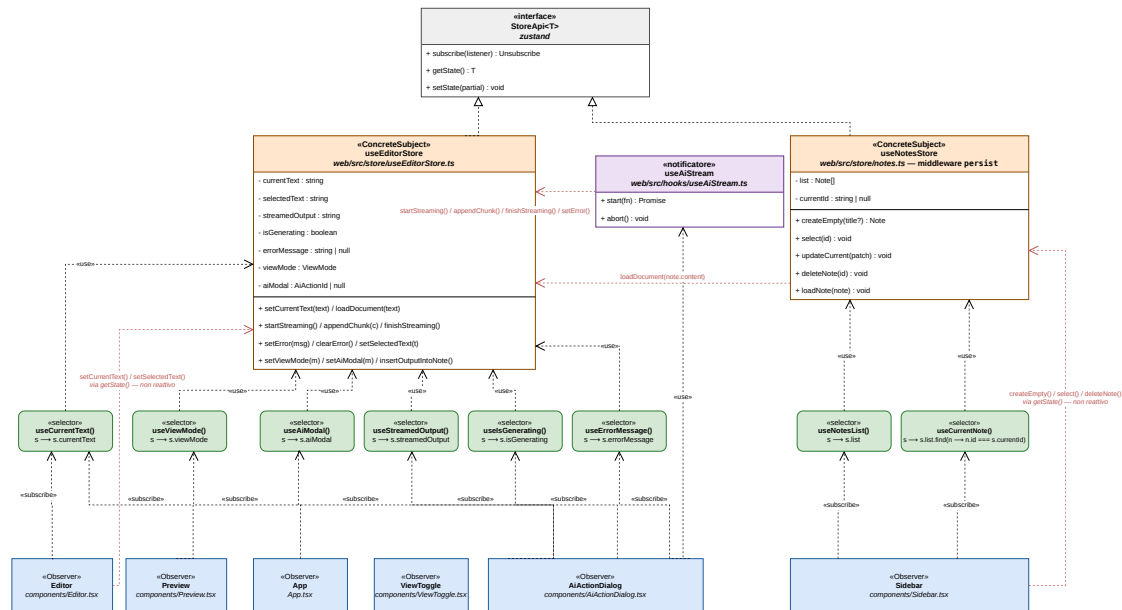


Figura 22: Diagramma soggetto-osservatori degli store e dei selettori

3.5.4 Dependency Injection

Definizione La Dependency Injection è la tecnica per cui un componente non costruisce né cerca le proprie dipendenze, ma le riceve dall'esterno, tipicamente dichiarandole nella propria firma. Non appartiene al catalogo dei design pattern di Gamma, Helm, Johnson e Vlissides, e non le si attribuisce quindi alcuna delle tre categorie di quel catalogo: è una tecnica di realizzazione dell'*Inversion of Control*, formalizzata successivamente e qui documentata perché è una scelta esplicita del gruppo con conseguenze verificabili.

Il capitolo tiene distinti tre piani che si sovrappongono spesso: la Dependency Injection è il meccanismo, l'inversione delle dipendenze è il principio che il meccanismo serve, l'architettura esagonale è la forma che il principio assume nell'organizzazione dei package. Qui si tratta del solo meccanismo; per gli altri due si rimanda alla sezione 3.4.1.

Problema risolto Una rotta ha bisogno di una porta per servire la propria richiesta. Se la costruisse da sé dovrebbe nominare l'adattatore concreto, e con esso il fornitore, la chiave di accesso e il modo di ottenerla: la conoscenza che il modulo di composizione tiene in un punto solo tornerebbe distribuita fra le rotte. Ne seguirebbe anche l'impossibilità di provare una rotta senza contattare il fornitore reale, poiché non esisterebbe alcun punto in cui sostituire ciò che essa costruisce al proprio interno.

Implementazione Le rotte descritte nella sezione 3.4.2 dichiarano la porta nella propria firma, associandola al provider che sa costruirla; è il framework_{GG} web a risolvere la dichiarazione prima di eseguire il corpo della rotta. I provider risiedono tutti nel modulo di composizione descritto nella sezione 3.4.5, che resta l'unico punto in cui gli adattatori concreti sono nominati. La rotta di generazione a partire da un collegamento dichiara entrambe le porte, e non nomina alcun fornitore.

Vantaggi Il vantaggio rivendicato è verificabile nel repository_{GG} e non è un'attesa teorica: i test_{GG} sostituiscono l'adattatore con un doppio senza toccare il codice di produzione, registrando nella tabella delle dipendenze del framework_{GG} una funzione che restituisce il doppio al posto del provider reale. È così che il prodotto è provato contro un servizio di estrazione che fallisce sempre, o contro un client del modello linguistico che emette una sequenza nota di frammenti, senza rete e senza chiavi.

Va tenuta distinta da questa la provabilità dei casi d'uso_G in isolamento rivendicata nella sezione 3.1.2, che si ottiene con un meccanismo diverso: un caso d'uso riceve la porta come parametro, e il doppio gli è passato direttamente dal test_{GG}, senza che il framework_{GG} intervenga, ed è precisamente ciò che consente di esercitarlo senza montare HTTP. Le due sostituzioni sono due conseguenze distinte della medesima scelta, cioè di aver dichiarato ovunque il tipo della porta e mai quello dell'adattatore: la prima opera al confine HTTP e passa per il framework_{GG}, la seconda opera nel nucleo e non lo coinvolge affatto.

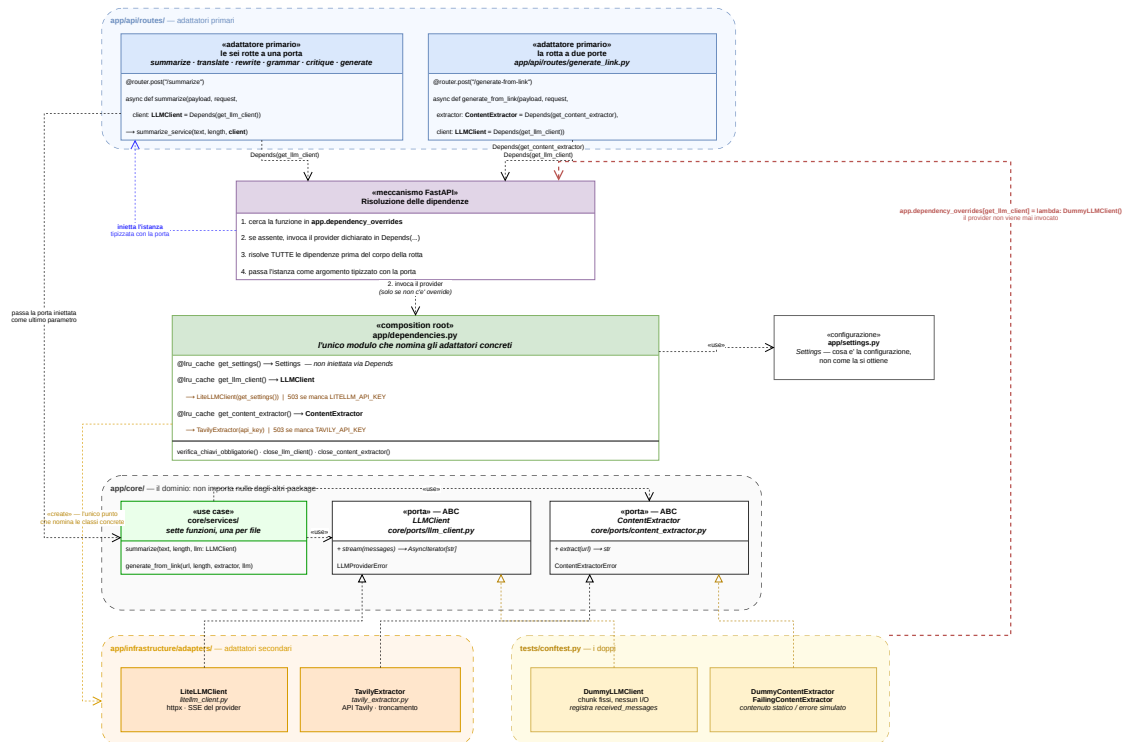


Figura 23: Diagramma della risoluzione della dipendenza e della sostituzione con un doppio

3.5.5 Pattern valutati e non adottati

Le strutture seguenti sono state considerate e scartate. Per ciascuna è indicata la condizione del criterio che non risulta soddisfatta.

Strategy Due valutazioni distinte, che conviene non confondere.

La prima riguarda un'attribuzione errata e assai probabile: la selezione della via di accesso al file system in base alle capacità del browser, descritta nella sezione 3.3.6, non è Strategy. Si tratta di rilevamento di funzionalità dell'ambiente di esecuzione: non esiste una famiglia di algoritmi incapsulati in oggetti intercambiabili, non esiste un contesto che ne detenga uno e possa sostituirlo, e la scelta non è configurabile né estendibile dall'esterno. È una diramazione condizionale eseguita una volta, su una capacità del browser che il codice interroga direttamente.

La seconda riguarda un caso in cui Strategy sarebbe stato applicabile, e va argomentato senza liquidarlo. Le sei prospettive dell'analisi critica secondo il metodo dei Sei Cappelli_C potrebbero essere incapsulate ciascuna in una classe, con un contesto che seleziona la prospettiva richiesta: sarebbe la scelta corretta se le sei differissero per logica di costruzione delle istruzioni destinate al modello. Nel codice del prodotto non è così. I costruttori delle istruzioni hanno struttura identica (una funzione unica, parametrizzata dalla prospettiva, che assembla ruolo, regole e testo dell'utente attraverso il medesimo compositore) e le sei prospettive differiscono per il solo blocco testuale delle indicazioni, raccolto in una tabella di costanti indicizzata dal cappello.

Introdurre una famiglia di classi produrrebbe quindi indizione senza rimuovere alcuna duplicazione, poiché non vi è logica duplicata da estrarre: la sola differenza fra le sei è già isolata in un dato. Va inoltre osservato quale sia oggi il costo dell'estensione, che è il criterio con cui

Strategy si giudica: aggiungere una prospettiva è l'aggiunta di una voce alla tabella delle costanti, non la modifica di un algoritmo né la scrittura di una classe. La scelta opposta resta difendibile qualora una prospettiva richiedesse in futuro una costruzione propria; allo stato del codice, la condizione del criterio che non è soddisfatta è la terza: non esiste un problema che la sua adozione risolverebbe.

Template Method Il Template Method richiede una gerarchia di classi: una classe base che fissa lo scheletro di un algoritmo e ne delega i passi variabili a metodi astratti, e sottoclassi che li realizzano. Nel prodotto il flusso comune alle sette operazioni assistite (estrazione del primo frammento, verifica_{GG} della connessione, emissione degli eventi, trattamento del fallimento a metà flusso, chiusura) è effettivamente centralizzato in un punto solo, ma per composizione di funzioni e generatori, non per ereditarietà: non esiste alcuna classe base, alcun metodo astratto, alcuna sottoclasse. È la forma idiomatica in un servizio asincrono costruito su generatori, dove il punto di variazione è il generatore ricevuto come parametro. Una gerarchia non semplificherebbe nulla e aggiungerebbe la rigidità propria dell'ereditarietà.

Decorator Il Decorator aggiunge responsabilità a un oggetto avvolgendolo in un altro che ne rispetta l'interfaccia, così che i due siano interscambiabili per il client. Nel prodotto non esiste alcuna preoccupazione trasversale scritta dal gruppo e avvolta attorno a un'interfaccia comune: nessun modulo racchiude una porta per aggiungervi registrazione degli eventi, misurazione dei tempi, ritentativi o memorizzazione dei risultati. Il solo componente intermedio presente nel servizio applicativo è quello che applica la politica CORS, che è fornito dal framework_{GG}: non è stato progettato dal gruppo e non soddisfa quindi la seconda condizione del criterio.

Builder Il Builder separa la costruzione di un oggetto complesso dalla sua rappresentazione, procedendo per accumulo progressivo di parti e affidando a un direttore la sequenza dei passi. La costruzione delle istruzioni destinate al modello linguistico, descritta nella sezione 3.4.3, avviene invece in un passo solo: una singola invocazione riceve tutte le parti come argomenti nominali e restituisce i due messaggi. Alcune parti sono facoltative, ma la facoltatività di un argomento non è accumulo progressivo, e non esiste alcun direttore né alcuna sequenza di passi da governare.

Precisazione sul Singleton Merita una nota a sé la memoizzazione applicata ai provider del modulo di composizione, descritta nella sezione 3.4.5, perché produce un effetto che è facile scambiare per un Singleton: l'adattatore viene costruito una volta sola e ogni richiesta successiva riceve la medesima istanza, che vive quanto il processo.

Non è tuttavia il Singleton della classificazione GoF, e la differenza non è terminologica. Nel Singleton è la classe a controllare la propria istanziazione: nasconde il costruttore e concede un unico punto di accesso globale, rendendo impossibile ottenere una seconda istanza. Qui le classi degli adattatori non controllano nulla, restano liberamente istanziabili (i test_{GG} ne costruiscono direttamente, passando le proprie credenziali fittizie) ed è il chiamante, attraverso il provider memoizzato, a ottenere sempre la stessa istanza. L'unicità è una proprietà del punto di costruzione, non della classe.

La conseguenza è precisamente ciò che rende la distinzione rilevante: il Singleton GoF impedisce la sostituzione nei test_{GG}, perché non esiste un modo di far ottenere al codice di produzione un'istanza diversa da quella che la classe custodisce; questa costruzione no, e infatti la sostituzione con un doppio descritta nella sottosezione 3.5.4 è possibile proprio perché l'unicità risiede nel provider e non nella classe. La memoizzazione, inoltre, è azzerabile su richiesta, ed è così che i test_{GG} evitano che un adattatore costruito da un caso di prova sopravviva a quelli successivi.

3.6 Idiomi e convenzioni di codifica

Le convenzioni raccolte qui governano la scrittura del codice su entrambi i livelli: vi entra soltanto ciò che vale al di qua e al di là del confine HTTP, mentre quanto riguarda una sola componente resta nella sezione 3.3 o 3.4. Le tre che seguono hanno in comune il fatto che nessuna delle due componenti potrebbe rispettarle da sola: un flusso si annulla soltanto se ogni anello della catena propaga la chiusura, un contratto è definito una volta sola soltanto se nessuno dei due lati lo riscrive, un errore resta comprensibile all'utente soltanto se ogni punto in cui viene tradotto rispetta lo stesso divieto.

3.6.1 Generatori asincroni e propagazione dell'annullamento

Un testo prodotto nel tempo è rappresentato sui due livelli dallo stesso costruito, il **generatore asincrono**: `AsyncIterator[str]` nel servizio applicativo, `AsyncIterable<string>` nell'applicazione web. Chi consuma un risultato progressivo lo itera, e non attende che sia completo per riceverlo.

Chi produce un flusso non lo consuma. Un caso d'uso del servizio applicativo restituisce il flusso della porta senza averne prelevato alcun elemento, e la funzione dell'applicazione web che contatta il servizio si comporta allo stesso modo verso il componente che la invoca. Ne segue che **su ciascuno dei due livelli il flusso viene toccato per la prima volta dove un guasto è ancora rappresentabile come esito della richiesta**, e mai più avanti: sul servizio applicativo è l'adattatore di trasporto, che preleva il primo frammento finché lo stato della risposta non è stato inviato (sezione 3.4.2); sull'applicazione web è il controllo sullo stato della risposta, che precede la lettura del corpo e trasforma uno stato negativo in un errore ordinario anziché in un flusso interrotto.

Propagazione dell'annullamento. Il requisito_G **R-79-F-De** (UC71) chiede che un'elaborazione in corso possa essere interrotta. Perché nessuna delle parti coinvolte continui a lavorare, la chiusura deve attraversare quattro passaggi, tutti necessari.

1. L'aggancio che governa il flusso nell'applicazione web possiede un `AbortController` e lo attiva quando l'utente annulla.
2. La funzione che contatta il servizio applicativo passa il segnale alla richiesta HTTP. **Senza questo passaggio l'annullamento sarebbe apparente**: il consumo dei frammenti si fermerebbe, ma la richiesta resterebbe aperta e il servizio continuerebbe a produrre fino al terminatore. È la prima postcondizione di UC71.
3. L'adattatore di trasporto del servizio applicativo verifica_G a ogni frammento se il client si è disconnesso, e in tal caso interrompe l'invio.
4. La chiusura del generatore restituito dal caso d'uso fa uscire l'adattatore secondario dal contesto della propria richiesta, e la connessione verso il fornitore cade.

Il quarto passaggio avviene senza che alcuno lo scriva: chiudere un generatore asincrono ne interrompe il corpo nel punto in cui era sospeso, e il costruito che governa la richiesta verso il fornitore rilascia la connessione uscendo. **La propagazione non attraversa la porta come un'operazione dichiarata**, e infatti nessuna delle due porte espone un metodo di chiusura (sezione 3.4.3).

Uscite diverse dall'annullamento. Un consumatore può abbandonare un flusso senza annullarlo: interrompendo il ciclo, restituendo un valore, sollevando un errore a valle. In quei casi il segnale non viene attivato e la catena non parte, quindi la funzione che contatta il servizio applicativo rilascia il lettore del corpo della risposta in un blocco conclusivo, eseguito comunque. L'eventuale rifiuto di quel rilascio viene ignorato di proposito: su un flusso già annullato il rilascio fallisce, e lasciarlo propagare sostituirebbe l'errore vero in uscita con uno che descrive soltanto la pulizia.

Riconoscimento dell'annullamento. Quando il segnale interrompe la richiesta il browser segnala l'evento con un errore, e distinguerlo da un guasto reale separa un'interfaccia che torna in attesa da una che mostra un messaggio di errore per un'operazione annullata di proposito.

Il riconoscimento avviene **per nome e non per classe**. La classe con cui il browser rappresenta quell'errore appartiene al proprio ambiente di esecuzione, e il confronto per classe fallisce attraversando i confini fra ambienti: riquadri incorporati, processi di servizio e ambiente dei test_{GG}. La conseguenza è misurata: su un annullamento reale il confronto per classe è falso, quindi l'errore cadrebbe nel ramo dei guasti generici e il messaggio tecnico del browser, in lingua inglese, comparirebbe nell'interfaccia contro il requisito_G **R-110-F-Ob**. La stessa convenzione è adottata dal modulo che accede al file system.

Il segnale è un parametro, mai una cattura. La funzione che avvia un'elaborazione riceve il segnale come argomento invece di catturarlo dall'ambiente in cui è stata costruita. La stessa funzione viene conservata per essere rieseguita dal comando che ripete l'ultima operazione, e un segnale catturato alla costruzione sarebbe già attivo al secondo giro: **ripetere un'operazione dopo averla annullata non ripartirebbe mai**.

3.6.2 Tipizzazione dei confini fra i livelli

Le due unità si scambiano dati su HTTP, e la forma di quei dati è l'unica cosa che devono conoscere l'una dell'altra. Il modo in cui quella forma viene stabilita è la convenzione più vincolante_G del prodotto, perché è l'unico punto in cui una divergenza fra i due livelli passerebbe inosservata fino all'esecuzione.

Il contratto si definisce una volta sola e si deriva. La catena parte dagli schemi di validazione_G del servizio applicativo e arriva ai tipi impiegati dall'applicazione web attraverso tre passaggi automatici: dagli schemi il framework_{GG} web ricava il documento OpenAPI, versionato nella repository_{GG} anziché prodotto al bisogno; da quel documento uno strumento genera le dichiarazioni di tipo; queste vengono riesportate da un unico modulo di raccolta, dal quale il resto dell'applicazione web importa.

Il modulo di raccolta non è un passaggio superfluo: il file generato ha una struttura interna che dipende dal generatore e non dal contratto, quindi importarlo direttamente legherebbe ogni componente a quella struttura e una rigenerazione si propagherebbe ovunque. Con il modulo di raccolta **la rigenerazione si assorbe in un punto solo**. Che il documento OpenAPI sia versionato ha a sua volta due conseguenze pratiche: i tipi si rigenerano senza avere un servizio in esecuzione, e una modifica del contratto compare nel confronto di una *pull request*_G invece di restare invisibile.

Due verifiche_G disgiunte. Sui dati in ingresso agisce la validazione_G a tempo di esecuzione, che rifiuta un corpo malformato prima che raggiunga il dominio_{GG}. Sui tipi impiegati dall'applicazione web agisce la verifica_{GG} statica, che intercetta in compilazione l'uso di un campo che il

contratto non prevede più. Nessuna delle due copre l'altra: la prima controlla *valori* che arrivano da fuori e che nessuna analisi del codice può conoscere in anticipo; la seconda controlla *codice*, e i suoi tipi non sopravvivono alla compilazione.

Un tipo derivabile dallo schema non si scrive a mano. Il divieto riguarda anche i valori, non i soli tipi. Le soglie di lunghezza e il sentinella della correzione grammaticale sono pubblicati dal contratto come valori letterali, e nel modulo di raccolta la costante corrispondente è annotata con il tipo che proviene dal contratto stesso: **cambiare quel valore da un lato solo non compila**. Senza questa convenzione il difetto resterebbe silenzioso, perché un sentinella disallineato non produce un errore ma un'interfaccia che non riconosce più la risposta del servizio.

Le due guardie di verifica_G. I due processi dell'integrazione_{GG} continua descritti nella sezione 2.6 controllano la convenzione ai due capi: quello del servizio applicativo riesporta il documento OpenAPI dal codice e pretende che coincida con il file versionato, quello dell'applicazione web rigenera le dichiarazioni di tipo e pretende che coincidano con quelle versionate. Il primo confronto fallisce quando il contratto pubblicato non corrisponde più al codice che lo produce, il secondo quando il contratto è cambiato senza che il client sia stato adeguato.

3.6.3 Denominazione, struttura dei file e gestione degli errori

Un nome dichiara un ruolo, non una tecnologia. Nessun package di nessuno dei due livelli prende nome da uno strumento o da un fornitore: un package chiamato come una tecnologia attira a sé moduli che con quella tecnologia non hanno rapporto. Sotto la regola stanno alcune corrispondenze fisse: un caso d'uso prende il nome dell'operazione che realizza e vive in un modulo omonimo; i costruttori di dipendenze del servizio applicativo cominciano tutti con lo stesso prefisso, e altrettanto fanno gli agganci dell'applicazione web. Il segno di sottolineatura iniziale marca su entrambi i livelli ciò che un modulo non espone all'esterno: è una convenzione di Python, adottata anche nel codice TypeScript dove il linguaggio non la impone, così che le due basi di codice si leggano allo stesso modo.

Un modulo per unità di responsabilità. La regola si applica anche dove separare costa qualche riga in più. Due casi la illustrano: lo schema che descrive *che cosa* sia la configurazione del processo e il costruttore che dice *come* la si ottenga stanno in moduli distinti, benché insieme farebbero una ventina di righe; la traduzione degli errori di validazione_G non vive nel modulo che costruisce l'applicazione ma accanto agli schemi di cui conosce i campi. Il criterio è lo stesso nei due casi: **stanno insieme le cose che cambiano per la stessa ragione**.

Due specie di messaggi di errore. La distinzione attraversa entrambi i livelli e vale in ogni punto in cui un errore viene scritto.

- I messaggi **rivolti all'utente** sono in linguaggio naturale italiano, dichiarano causa e azione correttiva e non contengono dettagli tecnici, come impone il requisito_G **R-110-F-Ob**. Comprendono le risposte del servizio applicativo e i pochi messaggi che nascono nell'applicazione web.
- I messaggi **rivolti a chi mette in esercizio il sistema_G** vivono nel registro di esecuzione e nella diagnostica di avvio e seguono la regola opposta: nominano per esteso variabili d'ambiente, operazioni e cause, perché a quel destinatario servono i dettagli che all'altro sono rumore. Il caso limite è il rifiuto di avviare il processo per una chiave mancante (sezione 3.4.5).

Fra le due specie corre un divieto netto: **il testo di un'eccezione non entra mai in un messaggio della prima specie**. Ciò che trapelerebbe non è solo una formulazione tecnica: un errore di validazione_G non filtrato rimanderebbe al mittente il testo che l'utente ha scritto, un errore del fornitore riporterebbe nel corpo della risposta la formulazione di un sistema_{GG} esterno in lingua non controllata, un errore del browser comparirebbe in inglese. I tre casi sono riepilogati nella sezione 4.5.

Un errore si converte al confine che lo dichiara. L'ultima convenzione riguarda i tipi e non i messaggi. Ogni tipo di errore è dichiarato accanto al **contratto** che ne prevede la possibilità e non accanto all'implementazione che lo solleva: le eccezioni delle due porte stanno con le porte, quelle di un caso d'uso con il caso d'uso (sezione 3.4.3). Ne discende che **un errore non attraversa mai un confine nella forma in cui è nato**: l'adattatore secondario converte l'eccezione della libreria di trasporto nel tipo dichiarato dalla porta, il caso d'uso converte quella della porta nel proprio, l'adattatore primario converte quella del caso d'uso in uno stato HTTP, l'applicazione web converte lo stato HTTP in uno stato dell'interfaccia. A ogni passaggio l'informazione si riduce, perché quanto serve al passaggio precedente non serve al successivo.

4 Flusso dei dati

4.1 Generazione assistita da LLM_G con risposta in streaming

La presente sezione segue una singola interazione dall'azione dell'utente al frammento reso a video, attraversando i confini descritti nel capitolo 3. L'operazione impiegata come pilota è il riassunto; le altre sei operazioni assistite dal modello linguistico condividono per intero la struttura qui descritta e differiscono soltanto per la rotta contattata, i parametri trasmessi e le istruzioni composte per il modello. Ciò che segue vale quindi per tutte, e non viene ripetuto sette volte.

L'avvio L'utente attiva l'operazione da un comando della barra superiore, che apre la finestra di dialogo dedicata. Per le operazioni che agiscono sul testo della nota (riassunto, traduzione, riscrittura, correzione e analisi critica) il testo non è digitato in un campo: è prelevato dall'area di lavoro secondo una regola unica, che restituisce la selezione corrente se non è vuota e altrimenti l'intera nota. È così che una sola coppia di comandi copre i due modi d'uso previsti dal requisito_G **R-54-F-Ob**, senza che l'utente debba dichiarare su che cosa intende operare. Le due operazioni di generazione, che attuano **R-72-F-Ob**, ricevono invece dall'utente un'indicazione testuale o un indirizzo, e in quel caso l'input_G proviene da un campo della finestra di dialogo.

Prima che qualunque richiesta parta, l'invio è sottoposto a una validazione_G preliminare nel browser, i cui limiti sono dichiarati nel registro delle operazioni disponibili: una lunghezza minima e una lunghezza massima, distinte per operazione. Un testo troppo breve o troppo lungo non produce una richiesta ma un messaggio che indica il limite superato e, nel caso della lunghezza massima, la dimensione attuale del testo; l'analisi critica richiede inoltre che sia stata scelta una prospettiva. La validazione_G non sostituisce quella del servizio applicativo: interviene prima, evita un viaggio di rete inutile e formula il rifiuto nei termini dell'interfaccia.

Quei limiti sono valori di dominio_{GG}, non convenzioni dell'interfaccia: risiedono nel nucleo del servizio applicativo e sono pubblicati dal contratto attraverso una rotta dedicata. L'applicazione web non interroga quella rotta a tempo di esecuzione: i valori le arrivano a tempo di compilazione, attraverso i tipi generati dal contratto. Il meccanismo che ne impedisce la divergenza è infatti la tipizzazione: nel modulo dei tipi dell'applicazione web le costanti sono dichiarate con il tipo

letterale che il contratto assegna a ciascuna, e una modifica del valore nel servizio applicativo, propagata ai tipi generati, farebbe fallire la compilazione. Va però osservato che il registro impiegato dalla validazione_G non passa da quelle costanti e riporta i medesimi numeri per esteso: il vincolo_{GG} esiste ed è dichiarato, ma il punto d'uso non vi si appoggia, e una divergenza fra i limiti dichiarati dal contratto e quelli applicati dall'interfaccia non verrebbe intercettata dal compilatore. La stessa costante di segnalazione impiegata dalla correzione, descritta più avanti, è invece consumata attraverso quel modulo e gode della garanzia.

Quando i parametri e il testo coincidono con quelli dell'ultima operazione avviata, il comando di invio si presenta come rigenerazione: l'applicazione conserva la chiamata precedente e la riesegue senza ricostruirla. La forma in cui è conservata (una funzione che riceve al momento dell'esecuzione ciò che serve ad annullarla, e non lo cattura alla creazione) è ciò che rende la rigenerazione possibile anche dopo un annullamento, argomento della sezione 4.2.

La richiesta Superata la validazione_G, la chiamata attraversa il confine HTTP attraverso il modulo unico descritto nella sezione 3.3.5: una richiesta `POST` con corpo `JSON` verso la rotta dell'operazione. Nessun altro punto dell'applicazione web contatta il servizio applicativo. La risposta non è attesa nella sua interezza: il corpo è letto come flusso, e il chiamante riceve una sequenza asincrona di frammenti testuali che consuma via via che arrivano.

Il servizio applicativo Sul lato server la richiesta incontra tre passaggi e nessuna logica di prodotto nel mezzo. Il corpo è validato prima che il corpo della rotta sia eseguito, sulla base dello schema dichiarato per quella rotta: una richiesta malformata non raggiunge mai il dominio_{GG} e riceve una risposta di errore di validazione_G, la cui forma è unificata con quella di ogni altro errore dell'API_{GG} e il cui trattamento è descritto nella sezione 4.5. La rotta invoca quindi il caso d'uso, passandogli i dati dell'operazione e la porta ricevuta per iniezione.

Il caso d'uso costruisce le istruzioni destinate al modello e restituisce il flusso della porta senza consumarlo. È una proprietà da sottolineare, perché tutto ciò che segue vi si appoggia: chi riceve quel flusso ottiene un generatore ancora *freddo*, dal quale nessun frammento è stato ancora estratto. La rotta lo consegna infine all'adattatore di trasporto, che è il modulo incaricato di trasformarlo in una risposta `HTTP`.

La composizione delle istruzioni Fra la richiesta e il fornitore si colloca il passaggio in cui il testo dell'utente diventa una richiesta al modello. La composizione produce due messaggi distinti: nel primo confluiscono il ruolo attribuito al modello e le regole che governano contenuto e forma della risposta, oltre all'indicazione di lunghezza quando l'operazione la prevede; nel secondo, e soltanto in quello, il testo dell'utente. La separazione non è formale: è ciò che tiene distinte le istruzioni del prodotto dal materiale su cui operano, e la sezione 3.4.3 ne descrive la costruzione. Per il percorso qui seguito è sufficiente osservare che il testo trasmesso al fornitore non è il testo dell'utente da solo, e che ciò che l'utente scrive non entra mai nella posizione in cui il modello si attende le proprie istruzioni.

L'ultimo tratto: la richiesta al fornitore L'adattatore che realizza la porta traduce i due messaggi nel formato di filo del gateway e richiede esplicitamente una risposta incrementale_G. Il gateway risponde a sua volta con un flusso di eventi, che l'adattatore consuma riga per riga estraendone la sola porzione di testo prodotta dal modello e ignorando le righe di servizio, compreso il terminatore proprio del fornitore. Ciò che risale verso il dominio_{GG} è quindi una sequenza di frammenti testuali privi di qualunque struttura del protocollo del fornitore: il formato descritto nel paragrafo seguente è prodotto dal servizio applicativo, non inoltrato dal gateway.

La connessione è soggetta a un tempo massimo di attesa, superato il quale l'operazione fallisce come qualunque altro guasto del fornitore.

Il protocollo di trasporto La risposta è emessa con tipo di contenuto `text/event-stream` secondo la specifica *Server-Sent Events*, e il formato degli eventi è il cuore tecnico di questa sezione.

Un frammento di testo è emesso come un evento di dati: una riga `data:` per ciascuna riga del contenuto, seguita da una riga vuota che chiude l'evento. La suddivisione per righe non è un dettaglio implementativo ma una necessità del formato: nella specifica il valore di un campo non può contenere un ritorno a capo, quindi emettere un frammento multiriga dentro un solo campo `data:` produrrebbe righe prive di prefisso, che il client scarterebbe. Non si perderebbe il solo carattere di a capo, ma tutto il testo che lo segue, una perdita sistematica, dato che tutte le operazioni assistite producono `MarkdownG` con titoli, elenchi e blocchi di codice. Un frammento su una sola riga produce esattamente il formato elementare, quindi la regola è trasparente nel caso semplice.

La fine del flusso è segnalata da un evento terminale convenzionale, `data:` `[DONE]` seguito dalla riga vuota. Il fallimento sopravvenuto durante l'emissione è segnalato da un evento terminale di errore, che si distingue dal precedente per il campo che lo apre: `event: error`, seguito dalle righe di dati che ne portano il messaggio, **Servizio temporaneamente non disponibile**. I tre esiti (completamento, errore dichiarato e chiusura senza alcun terminatore) sono quindi distinguibili dal client per asserzione e non per implicazione.

I due momenti dell'errore La distinzione che governa il trattamento dei guasti è temporale, e discende da una proprietà del protocollo HTTP: lo stato della risposta viene inviato prima del corpo, e una volta inviato non è più modificabile.

L'adattatore di trasporto sfrutta il fatto che il flusso ricevuto sia ancora freddo ed estrae il primo frammento prima di restituire la risposta. Fino a quel momento nulla è stato inviato al browser: se il fornitore è irraggiungibile, se rifiuta la richiesta o se risponde in modo malformato, l'errore che ne deriva è ancora traducibile in uno stato HTTP 503, e il browser riceve un fallimento pulito come per qualunque altra richiesta.

Dopo quel momento la strada è chiusa. L'intestazione è partita con esito positivo, i primi frammenti sono già a video, e nessuno stato di errore può più essere inviato. È qui che l'evento terminale di errore acquista la propria ragione: senza di esso il client osserverebbe la sola chiusura della connessione e non avrebbe modo di distinguerla da un completamento riuscito, presentando come compiuto un testo troncato. Il requisito **R-80-F-Ob** impone di gestire gli errori di comunicazione con il servizio del modello preservando il contenuto originale dell'editor: presentare un esito parziale come definitivo lo violerebbe nel modo più insidioso, perché senza alcun segnale visibile. L'evento terminale è ciò che rende quel fallimento dichiarato anziché silenzioso.

La decodifica e la presentazione Nel browser il flusso è ricostruito da un decodificatore che lavora sulla struttura dell'evento e non sulla singola riga: accumula le righe di dati appartenenti al medesimo evento e le riunisce con un ritorno a capo, ricomponendo esattamente il frammento che il servizio ha suddiviso. Il riconoscimento avviene sul nome del campo e non con un confronto testuale sulla riga: è la proprietà che consente a un frammento il cui contenuto sia letteralmente `event: error` di raggiungere l'utente come testo anziché essere scambiato per un errore, e non è un caso di scuola in un'applicazione che genera `MarkdownG` arbitrario. Il decodificatore riconosce i tre esiti descritti sopra e, nel caso di chiusura senza terminatore, emette esso stesso un fallimento, così che uno stream troncato non passi per completato.

Ogni frammento decodificato è aggiunto in coda a una porzione di stato dedicata all'esito in arrivo. Coerentemente con quanto descritto nella sezione 3.5.3, ciò che si aggiorna è la sola parte di interfaccia che presenta l'esito in arrivo (l'area della finestra di dialogo dell'operazione, dichiarata come regione aggiornabile ai lettori di schermo) e non il pannello di anteprima, che resta legato al testo della nota e durante l'elaborazione non cambia. Per l'intera durata dell'operazione l'interfaccia mostra un indicatore di attesa accanto al comando di avvio, esposto come elemento di stato con etichetta esplicita, e i comandi che avvierebbero altre operazioni sono disabilitati: è l'attuazione del requisito_G **R-109-F-De**, che richiede un indicatore di avanzamento per l'intera durata di un'attesa percepibile.

Fra lo stato e ciò che l'utente legge si interpone un'ultima trasformazione, che va dichiarata perché il testo mostrato non avanza al ritmo con cui i frammenti arrivano: la porzione visualizzata cresce di un numero fisso di caratteri a ogni fotogramma dell'animazione del browser, fino a raggiungere quanto è stato ricevuto. Il flusso di rete e il flusso visivo restano quindi distinti (il secondo insegue il primo e lo raggiunge) e ciò che si osserva è una comparsa regolare del testo invece di avanzamenti a scatti dettati dalla rete. La trasformazione non altera il contenuto: il testo mostrato è sempre un prefisso di quello ricevuto.

Due esiti si discostano dal caso generale e vanno riportati. Il primo è il flusso vuoto: un'operazione che si conclude senza aver prodotto alcun frammento è valida, e la risposta contiene il solo terminatore di fine. Il secondo riguarda la correzione ortografica e grammaticale, che quando non rileva errori non restituisce il testo corretto ma una costante di segnalazione concordata fra le due unità: l'interfaccia la riconosce, mostra un messaggio in linguaggio naturale al posto dell'esito e mantiene disabilitato il comando di accettazione, poiché non vi è nulla da inserire nella nota. È l'unico caso in cui il contenuto del flusso è interpretato dall'applicazione web anziché essere semplicemente reso.

L'esito accettato Finché l'operazione è in corso, e finché l'utente non la accetta, la nota non è toccata: l'esito vive in una porzione di stato separata dal testo del documento. Il comando di accettazione lo trasferisce nella nota secondo una modalità che dipende dal *tipo* di operazione e non dalla presenza di una selezione: le trasformazioni del testo esistente lo sostituiscono, le generazioni lo accodano. Nel primo caso, se la selezione su cui l'operazione ha lavorato è ancora presente nel testo, viene sostituita quella soltanto; altrimenti l'esito sostituisce l'intera nota, che è il comportamento coerente con l'aver operato sull'intero testo.

Il testo così ottenuto raggiunge il documento dell'editor come transazione, e non come sostituzione dello stato interno: la distinzione, descritta nella sezione 3.3.4, è ciò che consente di annullare l'inserimento con il comando di annullamento dell'editor, poiché la transazione entra nella cronologia delle modifiche, mentre un cambio di nota ne è esplicitamente escluso.

Il contratto che tiene insieme i due lati L'intero percorso poggia su un contratto dichiarato una sola volta. Gli schemi di validazione_G del servizio applicativo generano lo schema OpenAPI, che è versionato nel repository_{GG}; da quello schema sono generati i tipi impiegati dal modulo di accesso descritto sopra, anch'essi versionati. Poiché entrambi i file sono prodotti ma conservati sotto controllo di versione, possono restare indietro rispetto al codice che li produce: a impedirlo sono due guardie automatiche, che riesportano lo schema e rigenerano i tipi a ogni modifica proposta e ne pretendono la coincidenza con i file versionati. Il dettaglio dei processi che le eseguono è nella sezione 2.6.

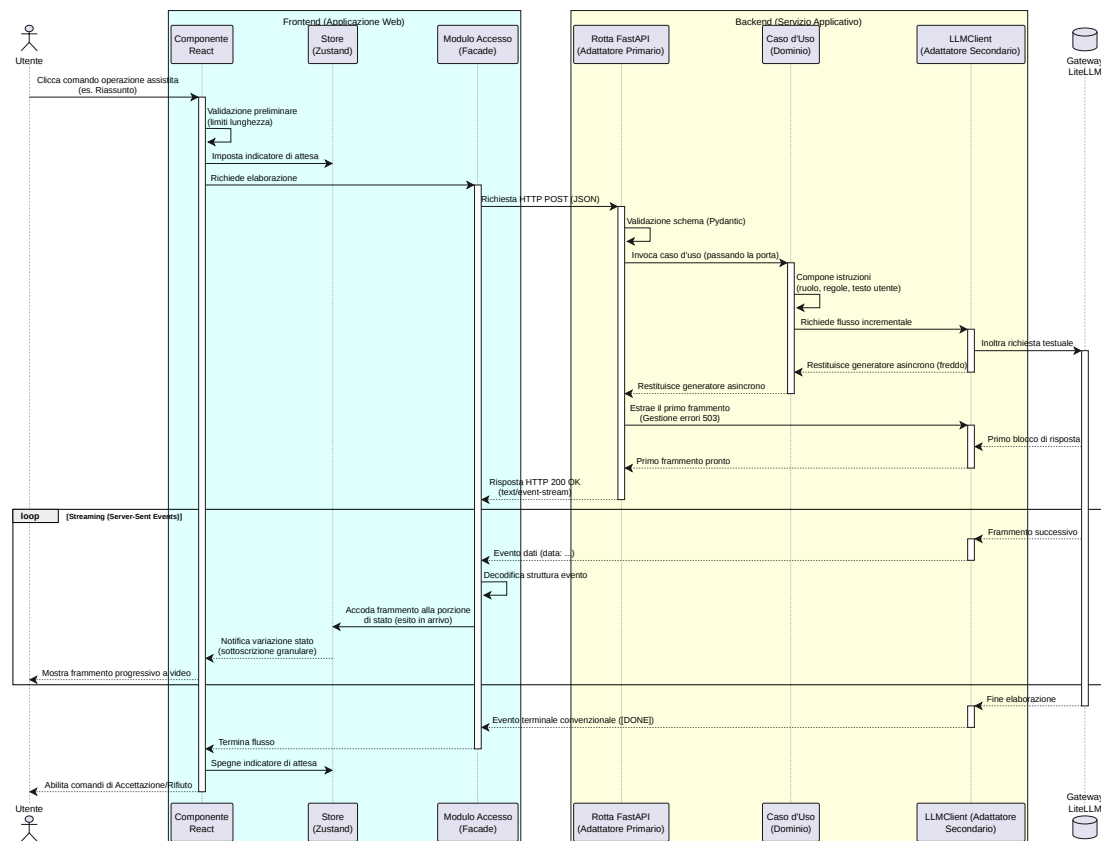


Figura 24: Diagramma di sequenza di un'operazione assistita con risposta in streaming

4.2 Annullamento di un'operazione in corso

Il requisito_G **R-79-F-De** e il caso d'uso_G **UC71** richiedono che un'elaborazione in corso possa essere annullata, che la richiesta pendente verso il servizio esterno sia annullata e che l'interfaccia sia riportata allo stato precedente senza applicare modifiche. La catena che ne discende attraversa gli stessi confini della sezione 4.1, in senso inverso.

L'origine e il primo anello Il comando di interruzione è offerto in due punti dell'interfaccia (accanto all'indicatore di attesa nella finestra di dialogo dell'operazione e nella barra superiore) e compare solo mentre un'elaborazione è in corso. Entrambi agiscono sul medesimo oggetto: il controllore di annullamento creato all'avvio dell'operazione e conservato per la sua durata. Il segnale che esso espone è passato alla richiesta HTTP nel momento in cui questa viene inviata, ed è questo che rende l'annullamento effettivo e non apparente: prima che ciò fosse fatto, interrompere fermava il consumo dei frammenti nel browser ma lasciava la richiesta aperta, e il servizio applicativo continuava a produrre il flusso fino al terminatore, ignaro di non avere più un destinatario.

L'annullamento non è un errore L'interruzione della richiesta fa emergere nel client un errore che il browser solleva per segnalare l'operazione annullata. Riconoscerlo correttamente è

ciò che protegge il requisito_G **R-110-F-Ob**: se fosse trattato come un fallimento qualunque, il messaggio tecnico del browser (in lingua inglese) comparirebbe nell'interfaccia come se qualcosa fosse andato storto, mentre l'utente ha semplicemente ottenuto ciò che ha chiesto. Il riconoscimento avviene per nome dell'errore e non per classe, poiché la verifica_G per classe non attraversa i confini di contesto del browser e fallirebbe proprio nel percorso normale dell'annullamento. Riconosciuto come tale, l'annullamento chiude l'attesa senza produrre alcun messaggio d'errore. Il generatore che consuma la risposta chiude inoltre il lettore del corpo in ogni uscita anticipata, non solo in quella provocata dal segnale.

Ciò di cui il servizio si accorge Sul lato server la disconnessione del chiamante è verificata prima di ogni emissione: sia prima del primo frammento, sia a ogni successivo passaggio del ciclo. Rilevata la disconnessione, il ciclo di emissione termina senza inviare il terminatore di fine flusso, e la chiusura del generatore del dominio_{GG} è eseguita nel blocco conclusivo che accompagna in ogni caso l'uscita. La verifica_G è quindi a granularità di frammento: un'operazione già avviata non è interrotta a metà di un frammento, ma non ne produce di ulteriori.

Che cosa accade verso il fornitore Qui la descrizione deve distinguere due cose che non hanno lo stesso peso probatorio, e che il documento non può presentare come una sola. Ciò che il codice esegue esplicitamente è la chiusura del generatore del dominio_{GG} all'uscita dal ciclo di emissione. Ciò che ne consegue è che l'adattatore verso il fornitore, il quale mantiene la risposta aperta entro un blocco con gestione automatica della risorsa, esca da quel blocco e chiuda la risposta in streaming. Non esiste nel prodotto una riga che annulli esplicitamente la richiesta verso il fornitore: la chiusura della connessione discende dal comportamento del client HTTP all'uscita dal blocco, comportamento documentato dalla libreria ma non osservabile nel codice del prodotto. È corretto affermare che la catena si chiude; sarebbe scorretto attribuire al codice una propagazione esplicita che non contiene.

Lo stato in cui resta l'interfaccia L'indicatore di attesa si spegne immediatamente all'annullamento, per entrambi i comandi, e il controllore è azzerato: la durata percepita dall'utente termina quando egli interrompe, non quando il ciclo se ne accorge. La nota non è modificata, poiché l'esito entra nel documento solo su accettazione esplicita, e la post-condizione di UC71 sull'assenza di modifiche è quindi soddisfatta. Va però dichiarato che cosa resta: il testo parziale già ricevuto rimane visibile nell'area dell'esito, e poiché l'elaborazione non è più in corso il comando di accettazione torna disponibile. L'utente può quindi accettare deliberatamente un esito parziale, oppure scartarlo con il comando di rifiuto, che azzerà l'esito e chiude la finestra di dialogo. Non si tratta di un ripristino automatico dello stato precedente: è una scelta lasciata all'utente, e l'unico stato che il sistema_{GG} ripristina da sé è quello di attesa.

4.3 Generazione a partire da un collegamento

Il requisito_G **R-74-F-De** (UC67.2, UC67.5) consente all'utente di indicare l'indirizzo di una pagina web come fonte per la generazione di un testo. Poiché il modello linguistico non accede alla rete, il contenuto della pagina deve essere acquisito dal prodotto e consegnato al modello insieme alle istruzioni.

La presente sezione segue quel percorso dalla richiesta fino alle istruzioni composte, che è la parte specifica di questa funzione, l'unica delle sette a coinvolgere un secondo fornitore esterno. La scelta della sorgente nell'interfaccia è nella sezione 3.3.2; il ritorno del testo generato non differisce dalle altre operazioni assistite ed è nella sezione 4.1.

I tre modi in cui un collegamento può essere rifiutato. Ciascun rifiuto avviene prima di un confine diverso e costa quindi una quantità diversa di lavoro.

1. **Indirizzo malformato:** lo schema della richiesta dichiara il campo come indirizzo web, e un valore che non lo è viene respinto dalla validazione_G con stato 422. Il rifiuto avviene al confine HTTP e **non raggiunge il dominio_G**.
2. **Indirizzo troppo lungo:** il caso d'uso lo scarta con un errore proprio, che la rotta traduce in stato 400. Il rifiuto avviene nel dominio_{GG} e **non tocca la rete**.
3. **Estrazione fallita:** la pagina è irraggiungibile, inesistente o priva di testo. L'adattatore solleva l'errore dichiarato dalla porta, il caso d'uso lo riconduce al proprio e la rotta lo traduce in stato 503. Il rifiuto avviene dopo la rete e **non raggiunge il modello**.

I tre messaggi che accompagnano questi stati rispettano il requisito_G **R-110-F-Ob** e non riportano il testo delle eccezioni che li hanno originati (sezione 3.6.3).

Ordine di intercettazione dei due errori. L'errore del collegamento scartato è un sottotipo di quello dell'estrazione fallita, così che un chiamante possa trattarli insieme. Nella rotta il sottotipo va perciò intercettato per primo: invertendo i due blocchi, un indirizzo troppo lungo riceverebbe lo stato dell'estrazione fallita e all'utente verrebbe detto che la pagina non è leggibile quando nessuno ha provato a leggerla.

Un limite dell'ordine dei rifiuti. La sequenza descritta vale quando la chiave del servizio di estrazione è configurata. Se manca, il costruttore della dipendenza rifiuta di produrre l'adattatore e, poiché il framework_{GG} web risolve le dipendenze **prima** di validare il corpo della richiesta, ogni chiamata riceve 503, **anche quando l'indirizzo è malformato e meriterebbe un 422**. È una conseguenza dell'ordine di risoluzione e riguarda il solo caso in cui il prodotto sia messo in esercizio senza quella chiave; la differenza rispetto alla chiave del gateway dei modelli, che se manca impedisce l'avvio del processo, è nella sezione 3.2.1.

Dalla richiesta accolta alle istruzioni composte. Superati i controlli, il percorso attraversa i confini descritti nella sezione 3.4 nell'ordine seguente.

1. Il caso d'uso verifica_G la lunghezza dell'indirizzo e interroga la porta di estrazione.
2. L'adattatore che la realizza inoltra la richiesta al servizio esterno. L'operazione di rete del client di quel servizio è sincrona, quindi viene eseguita su un thread separato per non fermare il ciclo di eventi, secondo la ragione riportata nella sezione 3.4.4.
3. Il contenuto restituito viene troncato dall'adattatore secondo la propria politica, e poi di nuovo dal caso d'uso alla soglia del dominio_{GG}.
4. Il testo estratto viene reso inerte nei suoi marcatori e racchiuso fra i delimitatori, poi collocato nel messaggio destinato al materiale.
5. Il costruttore delle istruzioni compone il messaggio di sistema_G con le regole proprie di questa funzione, fra cui quelle che dichiarano non autorevole il contenuto ricevuto.
6. Il caso d'uso interroga la porta del modello linguistico e restituisce il flusso, che da questo punto in avanti si comporta come quello di ogni altra operazione assistita.

Il troncamento avviene due volte. Le due soglie coincidono nel valore, il che rende la doppia applicazione facile da scambiare per una svista. La ragione sta in ciò che la porta dichiara: la porta di estrazione **non promette alcun limite di lunghezza** sul risultato, quindi il troncamento dell'adattatore è una sua politica, che un realizzatore diverso o un doppio predisposto per un test_{GG} non sarebbero tenuti a rispettare. Il caso d'uso applica la propria soglia perché è la sola di cui risponde: se le due divergessero, il dominio $_{GG}$ resterebbe comunque entro il limite che dichiara.

Dove interviene ciascuna difesa sul contenuto di terze parti. Le tre difese contro le istruzioni ostili contenute in una pagina, e il loro limite, sono descritte nella sezione 3.4.3; qui rileva il punto della sequenza in cui ciascuna interviene, perché è l'ordine a renderle efficaci. La separazione dei ruoli agisce al passo 4, collocando il contenuto nel messaggio del materiale e mai fra le istruzioni. La neutralizzazione dei delimitatori agisce nello stesso passo e **prima** che il testo sia racchiuso, altrimenti una pagina potrebbe chiudere il recinto per conto proprio; avviene inoltre **dopo** il troncamento del passo 3 e sostituisce i caratteri senza aggiungerne, perché allungare il testo farebbe superare alle istruzioni un limite che il dominio $_{GG}$ considera rispettato. La dichiarazione di non autorevolezza agisce al passo 5, nel messaggio di sistema $_G$, l'unico luogo che la pagina non può raggiungere.

L'estrazione precede l'inizio del flusso. Questo è l'unico dei sette casi d'uso $_G$ dichiarato `async def` (sezione 3.4.3), e il flusso appena descritto è il luogo in cui il motivo si vede. Se fosse scritto come generatore, il corpo resterebbe fermo finché nessuno ne consuma il risultato: i passi da 1 a 5 avverrebbero dentro l'adattatore di trasporto, cioè dopo l'invio dello stato della risposta. **Nessuno dei tre stati di rifiuto sarebbe più possibile**, e un collegamento troppo lungo o una pagina irraggiungibile si presenterebbero all'utente come un flusso interrotto invece che come un errore. Con la forma adottata l'estrazione avviene subito e ciò che viene restituito resta un flusso non consumato, la proprietà che la sezione 3.6.1 chiede a entrambi i livelli.

4.4 Salvataggio e caricamento di una nota

La presente sezione descrive le modalità con cui l'applicazione web persiste le note lato client, in attuazione della strategia esposta in §2.4. Come dichiarato in quella sede, i meccanismi sono due, distinti per scopo e complementari: l'accesso al file system per l'esportazione e l'importazione deliberate, e l'archivio locale del browser per la continuità della sessione. La sezione descrive entrambi, il momento e il modo in cui ciascuno viene attivato, e il recupero dei metadati dall'oggetto File.

4.4.1 Persistenza su file system

Le operazioni di salvataggio e caricamento su file sono realizzate nel modulo `lib/fileSystem.ts`, che espone le funzioni `saveNoteToFile` e `openNoteFromFile`. Entrambe adottano una strategia a due vie, selezionata a tempo di esecuzione in base alle capacità del browser ospite.

Selezione della via. Il criterio di selezione è la disponibilità effettiva dell'interfaccia nel browser in uso, non l'identificazione dell'agente utente. L'applicazione verifica $_G$ direttamente la presenza dei metodi richiesti sull'oggetto globale della finestra:

- Se `showOpenFilePicker` e `showSaveFilePicker` sono presenti, viene utilizzata la **File System Access API** (disponibile su Chrome/Edge).

- In caso contrario, l'applicazione ricade sui **meccanismi universali** (input_G file e download link, utilizzati su Firefox e Safari).

La doppia via non è una precauzione generica: il requisito di vincolo_G R-1-V-Ob include Firefox fra gli ambienti di esecuzione di riferimento, quindi una realizzazione fondata sulla sola File System Access API_G lascerebbe scoperto uno dei tre browser prescritti.

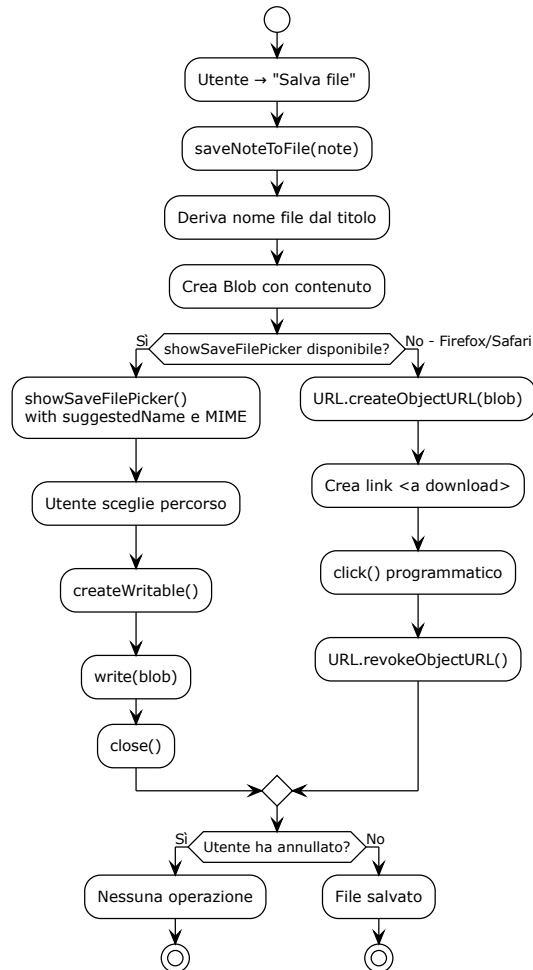


Figura 25: Diagramma UML 4.4.1-1 - fileSystem

Salvataggio su file (saveNoteToFile). La funzione riceve una **Note** e ne scrive il contenuto su file:

1. Il nome del file è derivato dal titolo della nota, con estensione **.md** (soddisfa R-4-V-Ob).
2. Il contenuto è il solo testo della nota, senza intestazioni o marcatori aggiuntivi: il file è un documento Markdown puro.

3. Se la File System Access API_G è disponibile, la funzione invoca `showSaveFilePicker` con il nome suggerito e il tipo MIME `text/markdown`; l'utente sceglie il percorso di destinazione; il file viene scritto tramite `FileSystemWritableFileStream`.
4. Se l'API_G non è disponibile, la funzione costruisce un Blob con il contenuto della nota, ne ricava un indirizzo temporaneo tramite `URL.createObjectURL`, lo assegna a un collegamento di scaricamento (`<a download>`) e lo attiva per via programmatica; l'indirizzo temporaneo viene rilasciato al termine dell'operazione con `URL.revokeObjectURL`.
5. L'annullamento da parte dell'utente (chiusura del selettore senza selezione) è riconosciuto tramite il controllo `eAnnullamento` su `AbortError` e trattato come esito legittimo, non come errore.

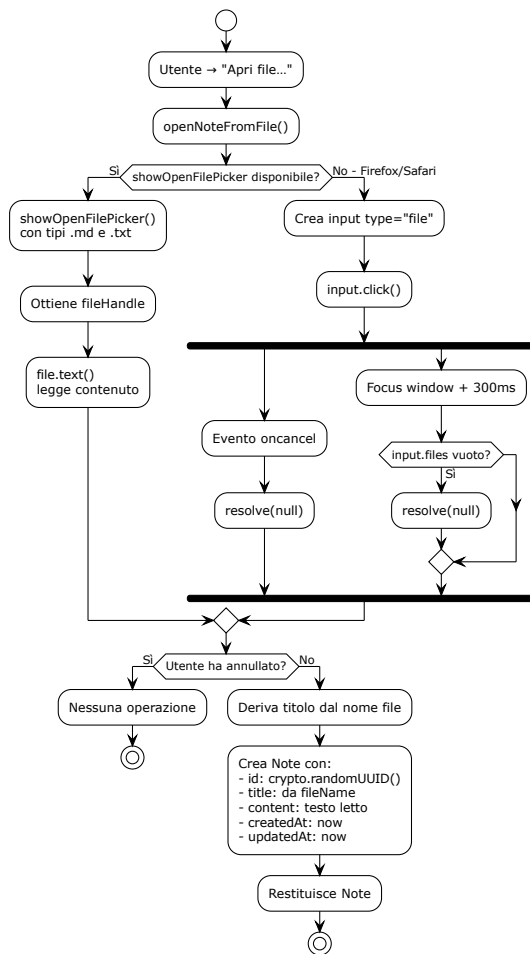


Figura 26: Diagramma UML 4.4.1-2 - saveNoteToFile

Caricamento da file (openNoteFromFile). La funzione restituisce una `Note` a partire da un file selezionato:

1. Se la File System Access API_G è disponibile, la funzione invoca `showOpenFilePicker` con i tipi accettati `.md` e `.txt`; ottiene un riferimento al file scelto e ne legge il contenuto testuale tramite `File.text()`.
2. Se l'API_G non è disponibile, la funzione crea un elemento `input` di tipo `file` con gli stessi tipi accettati; ne legge il contenuto tramite `FileReader.readAsText`. Questo ramo gestisce due vie di uscita per l'annullamento: l'evento `oncancel` (disponibile in alcuni browser) e il ritorno del focus alla finestra con un timer di tolleranza (`GRAZIA_ANNULLAMENTO_MS = 300ms`) che controlla se `input.files` è vuoto. Il doppio meccanismo garantisce che la Promise non resti pendente per sempre su Firefox.
3. La funzione restituisce una `Note` con:
 - `id`: generato da `crypto.randomUUID()`;
 - `title`: derivato dal nome del file, privato dell'estensione (`.md` o `.txt`); se il nome del file non è utilizzabile, viene usato il titolo di ripiego «Nota importata»;
 - `content`: il testo letto dal file;
 - `createdAt`: l'istante di importazione;
 - `updatedAt`: l'istante di importazione.
4. L'annullamento è riconosciuto e trattato come esito legittimo in entrambi i rami.

La derivazione del titolo dal nome del file avviene in entrambi i rami, garantendo la parità fra i due percorsi. È necessaria perché il ramo di ripiego è il percorso primario su Firefox, incluso fra i browser prescritti da R-1-V-Ob.

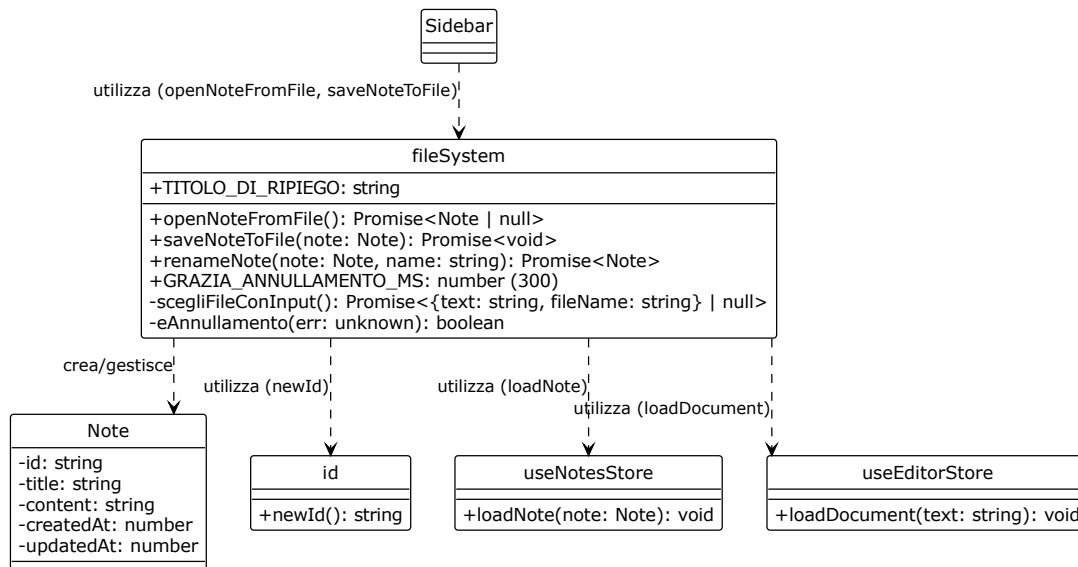


Figura 27: Diagramma UML 4.4.1-3 - openNoteFromFile

4.4.2 Continuità della sessione

La continuità della sessione è realizzata tramite il middleware `persist` di Zustand, applicato a `notes` come descritto in §3.3.3. Il meccanismo non richiede alcuna azione dell'utente.

Scrittura nell'archivio locale. Il middleware `persist` serializza lo stato di `notes` e lo riversa nel `localStorage` del browser sotto la chiave `'notes_persistence'`. Lo stato conservato comprende:

- `list`: l'elenco delle note, con `id`, `title`, `content`, `createdAt`, `updatedAt`;
- `currentId`: l'identificativo della nota correntemente selezionata.

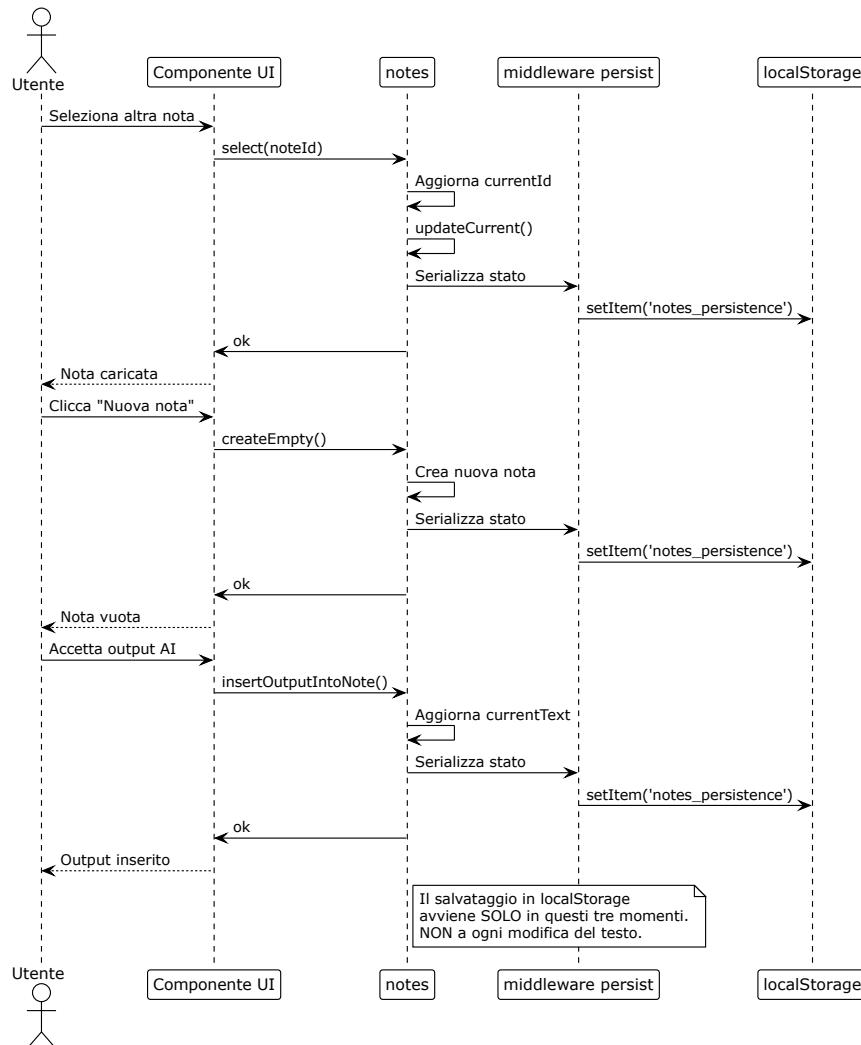


Figura 28: Diagramma UML 4.4.2-1 - salvataggio automatico in `localStorage`

Quando viene scritto il contenuto dell'editor. Va precisato quando il testo in lavorazione raggiunge l'archivio. Il contenuto dell'editor è riversato nella nota corrente nei seguenti momenti:

1. **Al cambio di nota** (`notes.select`): il contenuto corrente viene salvato nella nota che si sta abbandonando.

2. **Alla creazione di una nuova nota** (`notes.createEmpty`): il contenuto corrente viene salvato nella nota precedente prima di crearne una nuova.
3. **All'accettazione di un output AI** (UC68, `insertOutputIntoNote`): il contenuto modificato viene immediatamente scritto nella nota corrente.

Il contenuto **non** è riversato a ogni modifica del testo né alla chiusura della scheda. Le modifiche successive all'ultimo cambio di nota non sono conservate: è un limite noto della realizzazione attuale, non una proprietà del supporto. La scelta è deliberata: il costo di scrivere in `localStorage` a ogni battitura sarebbe inaccettabile per le prestazioni; il salvataggio nei momenti di transizione garantisce la sopravvivenza delle note complete, a costo di perdere le modifiche non ancora "finalizzate" da un cambio di contesto.

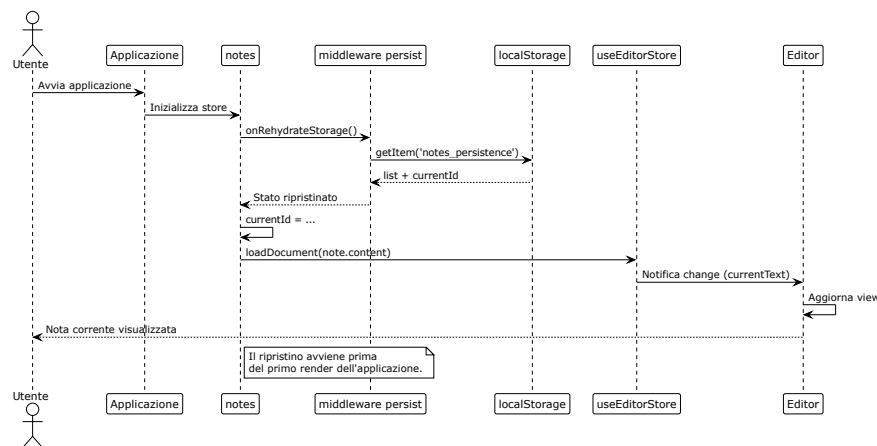


Figura 29: Diagramma UML 4.4.2-2 - lettura all'avvio

Lettura all'avvio. All'avvio dell'applicazione, il middleware `persist` rilegge lo stato dal `localStorage` prima del primo render. Durante il ripristino (`onRehydrateStorage`), viene chiamato `loadDocument` per caricare il contenuto della nota corrente nell'editor. L'applicazione riparte dallo stato in cui l'utente l'aveva lasciata: l'elenco delle note, la nota corrente e il suo contenuto sono quelli della sessione precedente.

Limiti del supporto. Il `localStorage` ha limiti propri, che vanno dichiarati:

- **Spazio:** è soggetto a una quota (tipicamente 5-10 MB per origine). Il superamento fa fallire la scrittura; l'applicazione lo tratta come condizione prevista e ripristina lo stato precedente.
- **Isolamento:** i dati sono confinati all'origine e al profilo del browser in uso. Non sono raggiungibili da un altro browser, da un altro dispositivo o da una finestra di navigazione anonima.
- **Cancellazione:** la cancellazione dei dati di navigazione rimuove i dati.

Questo meccanismo non è un sostituto della persistenza server-side: resta interamente lato client, come dichiarato in §2.4.

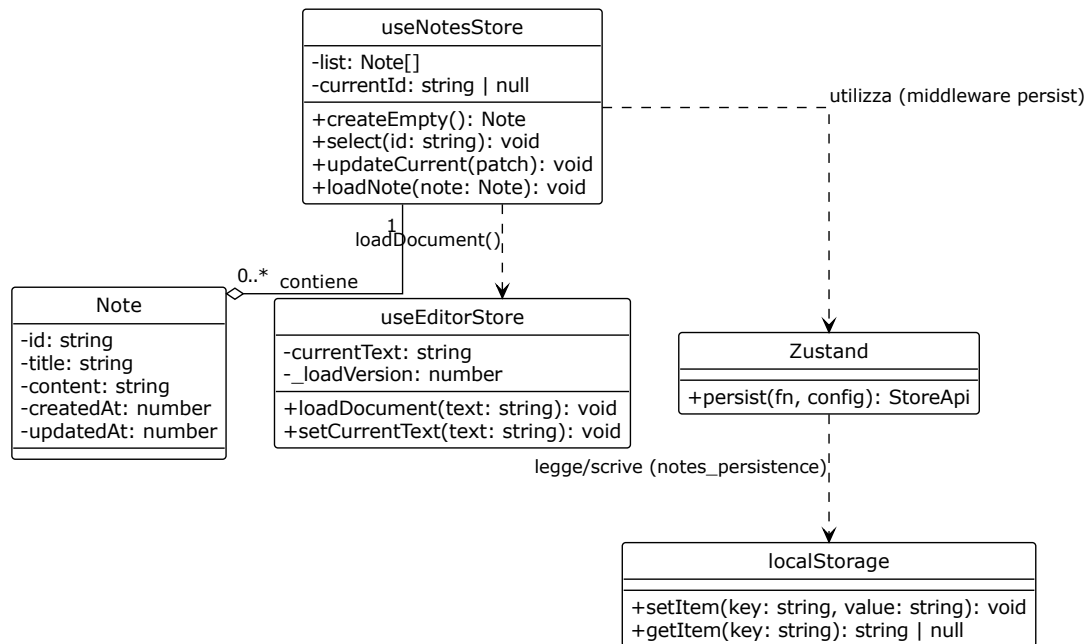


Figura 30: Diagramma UML 4.4.2-3 - continuità della sessione

4.4.3 Recupero dei metadati dall'oggetto File

Il requisito R-97-F-De (UC84) richiede la visualizzazione delle informazioni e dei metadati di una nota selezionata. Come dichiarato in §2.4, la copertura del requisito distingue due casi.

Nota creata nel prodotto. Per le note create all'interno dell'applicazione, i metadati sono posseduti e conservati:

- **id**: generato da `newId()` (da `lib/id.ts`) al momento della creazione;
- **title**: assegnato dall'utente o impostato a «Senza titolo»;
- **createdAt**: istante di creazione;
- **updatedAt**: istante dell'ultima modifica;
- **content**: il testo della nota.

Questi metadati sono affidati alla persistenza locale descritta in §4.4.2: il middleware **persist** li scrive immediatamente in **localStorage** e li rilegge all'avvio. Per queste note, R-97-F-De è già coperto.

Nota importata da disco. Per le note importate da file locale (UC76.1), il file contiene il solo testo. I metadati sono attualmente popolati come segue:

- **id**: generato da `newId()`;
- **title**: derivato dal nome del file, privato dell'estensione; se il nome non è utilizzabile, viene usato il titolo di ripiego «Nota importata»;

- **createdAt** e **updatedAt**: entrambi impostati all'istante di importazione.

Come dichiarato in §2.4, questa realizzazione sarà estesa per sfruttare gli attributi **lastModified** e **size** dell'oggetto **File**, in modo da popolare rispettivamente l'istante di ultima modifica e la dimensione del contenuto. L'istante di creazione (**createdAt**) non è ricostruibile in modo affidabile dalle informazioni disponibili; per le note importate da disco sarà presentato come «non disponibile».

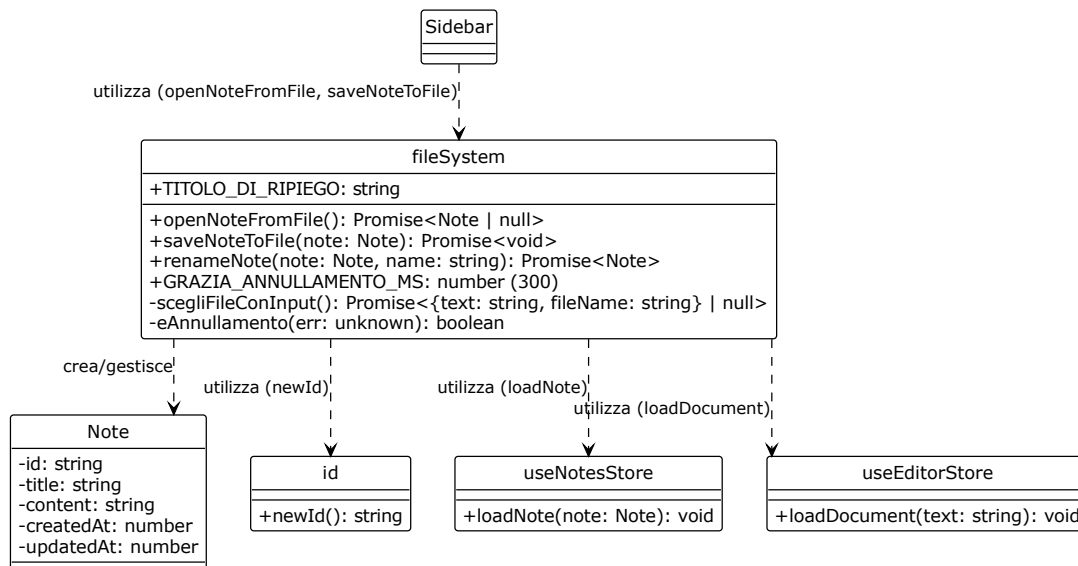


Figura 31: Diagramma UML 4.4.3 - nota creata nel prodotto vs nota importata da disco

4.5 Propagazione e presentazione degli errori

Le sezioni precedenti seguono le interazioni che riescono. Questa segue quelle che falliscono, dal punto in cui il guasto si manifesta fino alla frase che l'utente legge. I requisiti_{GG} coinvolti sono **R-110-F-Ob**, che impone messaggi in linguaggio naturale privi di dettagli tecnici, e **R-80-F-Ob** (UC72), che chiede di avvisare l'utente quando un'operazione non si completa.

Il momento del guasto determina la forma della risposta. Il medesimo guasto del gateway dei modelli produce due esiti diversi a seconda del momento in cui si manifesta, e il momento che li separa è l'invio dell'intestazione di stato. Prima di quell'istante lo stato è ancora modificabile e il guasto diventa una risposta di errore ordinaria; dopo, lo stato inviato è già quello di una risposta riuscita e non è ritrattabile, quindi l'unica via che resta è dichiarare il fallimento dentro il flusso. **La stessa causa produce così due rappresentazioni diverse**, non per scelta ma perché il protocollo non consente altro. È anche la ragione per cui l'adattatore di trasporto preleva il primo frammento prima di restituire la risposta (sezioni 3.4.2 e 3.6.1): quel prelievo sposta il maggior numero possibile di guasti dalla seconda categoria alla prima.

Prima dell'intestazione: una forma sola per ogni errore. Tutte le risposte di errore del servizio applicativo hanno la stessa forma, un solo campo testuale destinato a essere mostrato.

Vi confluiscono esiti di origine diversa: la validazione_G della richiesta, che compone la propria frase a partire dal campo che non ha superato il controllo (stato 422); i rifiuti propri di una singola operazione, come quelli della sezione 4.3 (stati 400 e 503); l'indisponibilità del gateway all'apertura del flusso (stato 503, UC72). L'uniformità consente all'applicazione web di leggere un campo solo e al contratto di dichiarare una forma sola, così che i tipi generati non descrivano risposte che il servizio non produce.

Dopo l'intestazione: l'evento terminale d'errore. Quando il guasto sopraggiunge a flusso già avviato, il servizio applicativo emette un evento terminale che dichiara il fallimento e registra l'accaduto nel proprio registro di esecuzione; il formato di quell'evento è nella sezione 4.1.

Senza un evento dedicato un flusso interrotto e uno completato sarebbero **indistinguibili per il client**, che in entrambi i casi osserverebbe soltanto la chiusura della connessione: l'utente riceverebbe un testo mozzo presentato come completo, che è la condizione vietata da **R-80-F-Ob**. La distinzione è quindi ottenuta per affermazione e non per implicazione.

I tre esiti che il client distingue. Il consumo di una risposta progressiva termina in uno di tre modi, e l'applicazione web li tratta diversamente.

- **Terminatore di completamento:** l'operazione è riuscita, il testo accumulato è integro.
- **Evento terminale d'errore:** l'operazione è fallita dopo essere cominciata. Il testo già arrivato resta visibile, ma accanto compare il messaggio che ne dichiara l'incompletezza.
- **Chiusura senza alcun terminatore:** la connessione è caduta e il servizio non ha detto nulla. Questo è l'unico caso in cui il messaggio d'errore **nasce nell'applicazione web**, perché è l'unico in cui non ne esiste uno proveniente dal servizio. Anche quel messaggio dichiara causa e azione correttiva senza dettagli tecnici, come tutti gli altri.

Il terzo caso è il più insidioso, perché una connessione che cade è indistinguibile da un completamento se non si è deciso in anticipo che il completamento debba essere dichiarato.

Il riconoscimento avviene sul nome del campo, non sul testo della riga. Riconoscere l'evento d'errore confrontando il testo della riga con una stringa nota funzionerebbe fino al giorno in cui il modello linguistico producesse quella stessa stringa dentro il testo generato: un frammento del contenuto verrebbe scambiato per la dichiarazione di un fallimento e l'operazione risulterebbe interrotta senza esserlo. Su un prodotto che genera Markdown arbitrario, fra le cui funzioni c'è la scrittura di blocchi di codice, la collisione è verosimile; il riconoscimento per nome del campo la rende impossibile per costruzione.

Dal punto in cui nasce a quello in cui l'utente lo legge. Il percorso di un messaggio d'errore attraversa quattro passaggi, e nessuno lo riformula.

1. Il servizio applicativo lo produce, nella forma unica descritta sopra oppure dentro l'evento terminale.
2. La funzione dell'applicazione web che contatta il servizio ne estrae il testo e lo solleva come errore ordinario, unificando i due casi: da qui in avanti un fallimento prima e uno dopo l'intestazione sono la stessa cosa.
3. Lo stato condiviso registra il messaggio e spegne insieme l'indicatore di avanzamento. I due effetti sono deliberatamente inseparabili: un errore a video con l'indicatore ancora acceso descriverebbe una situazione che non esiste, un'operazione insieme fallita e in corso.

4. L'interfaccia lo presenta in un riquadro di avviso collocato in un'area annunciata dai lettori di schermo, così che l'esito raggiunga anche chi non guarda lo schermo.

Un caso resta fuori da questo percorso: l'annullamento volontario. Il browser lo segnala con un errore, ma un'operazione interrotta di proposito non è un fallimento e non produce alcun messaggio; il riconoscimento che lo distingue è nella sezione 3.6.1.

Che cosa non trapela, nei tre punti in cui potrebbe. Il requisito_G **R-110-F-Ob** vieta i dettagli tecnici nei messaggi rivolti all'utente. Il divieto è rispettato in tre punti, ciascuno dei quali trattiene qualcosa di diverso.

- **Validazione della richiesta:** la forma predefinita del framework_{GG} web espone il tipo dell'errore, la posizione del campo e il contesto del controllo, e rimanda al mittente il valore ricevuto, cioè **il testo che l'utente ha scritto**. Nessuno dei quattro raggiunge la risposta.
- **Guasto del fornitore:** la formulazione dell'eccezione, prodotta da un sistema_{GG} esterno e in lingua non controllata, resta nel registro di esecuzione; nel corpo della risposta va un messaggio del prodotto.
- **Annullamento:** il testo con cui il browser segnala l'interruzione è tecnico e in lingua inglese, e non raggiunge l'interfaccia.

Nei tre casi l'informazione utile alla diagnosi è conservata dove serve a chi mette in esercizio il sistema_{GG} e sostituita da una frase del prodotto dove il destinatario è l'utente, secondo la distinzione fra le due specie di messaggi stabilita nella sezione 3.6.3.

5 Tracciamento dei requisiti_G

5.1 Criteri di tracciamento

5.2 Requisiti_G funzionali

5.3 Requisiti_G di qualità

5.4 Requisiti_G di vincolo_{GG}

5.5 Grafici riassuntivi